Machine Learning

BCSE0105: MACHINE LEARNING

Objective: *To introduce students to the basic concepts and techniques of Machine Learning. To develop skills of using recent machine learning software for solving practical problems. To gain experience of doing independent study and research.*

Credits: 03 **L–T–P-J: 3–0–0-0**

Module No.	Content	Teaching Hours
I	**Introduction:** Machine Learning basics, Hypothesis space and inductive bias, training and test set, and cross-validation. **Introduction to Statistical Learning:** Bayesian Method. **Machine Learning:** Supervised (Regression, Classification) vs. Unsupervised (Clustering) Learning. **Data Preprocessing:** Imputation, Outlier management, One hot encoding, Dimensionality Reduction- feature extraction, Principal Component Analysis (PCA), Singular Value Decomposition **Supervised Learning:** Regression- Linear regression, Polynomial regression, Classification- Logistic regression, k-nearest neighbor classifier,	20
II	**Supervised Learning:** Decision tree classifier, Naïve Bayes classifier Support Vector Machine (SVM)Classifier, **Unsupervised Learning:** k-means clustering, Hierarchical clustering **Underfitting vs Overfitting:** Regularization and Bias/Variance. **Ensemble methods:** Bagging, Boosting, Improving classification with Ada-Boost algorithm.	20

Text Book:

- Tom M. Mitchell, Machine Learning. Tata McGraw-Hill Education, 2013.
- Alpaydin, E. . Introduction to machine learning. MIT press, 2009.

Reference Books:

- Harrington, P. , " Machine learning in action", Shelter Island, NY: Manning Publications Co, 2012.
- Bishop, C. M. . Pattern recognition and machine learning (information science and statistics) springer-verlag new york. Inc. Secaucus, NJ, USA. 2006

Human Learning

- Learning is commonly defined as the process of acquiring knowledge through observation
- In our daily lives, we observe different things, and based on these observations, we learn.
- So more we observe more we learn.
- With more learning ,task can be performed more efficiently.
- so to do a particular task we must have observation regarding that.
- If we already observed some things then it is easy for us to perform in particular situation.

Types of Human Learning

- Learning under expert(Example: children learn under the guidance of parent, Teacher)
- Learning guided by knowledge gained from (For example, children recognize new things by connecting them with what they have learned before. This helps them understand the world better and learn new things more easily).

- Learning by self(without Expert)

When we don’t have an expert to guide us, we take action in a situation and see if it works or not. If the action is correct, we use it again in the future. If it is wrong, we avoid it and try something else

Machine Learning

- Is the science and art of programming computers so that they can learn from data.
- We are creating a program that learns from data and then produces output based on what it has learned.
- To find prime number ,to check whether the no is prime or not. generally we provide logic.
- In machine learning, we provide many numbers and label them as prime or not prime.
- The program learns from this data. After learning, if we give it a new number, it can predict whether the number is prime or not.

- Is the field of study that give Computers the ability to learn without being explicitly programmed -(**Arthur Samuel 1959)**
- In prime no. checking program we are not writing the the logic of checking whether the number is prime or not .
- It learns from data and then predicts for new data whether it is prime or not.
- A Computer program is said to learns from experience E with respect to some task T and Some performance performed measure P, if it's performed on T, as measured by P improves with experience E – **(Tom Mitchell 1997)**
- Machine Learning is the study of algorithms that
  - improve their performance P
  - at some task T

Example of Machine Learning Program

Spam Filter learning program

- Learn to flag spam given (training set) example of spam and regular Emails.

  (need to provides spam and regular Emails ,program will

  learn from data to identify which type of data is spam data).

- Each training example is called training instance (or sample)
- Task T –to flag spam for new emails (recognize spam mail)
- Experience E- training data
- Performance measure P – need to be defined for example ratio of correctly classified emails accuracy. (if there is 100 new email for prediction spam or not ,if accuracy is 90 % then program is learning from data.

traditional programming

1. Recognize what spam looks like

-it has one image (Promotional image)

-Email has link

-Email body consist sentence like “ you won a prize“

Not spam

-Email from official domain e.g. sharda.ac. in

-E mail from known source

2. Write a program :we write many condition whether spam not.

3.Test your program and repeat Step I and 2 until it is good enough .

Limitation

Since Problem will be lengthy because lot of condition hard to maintain.

If change require difficult to maintain.

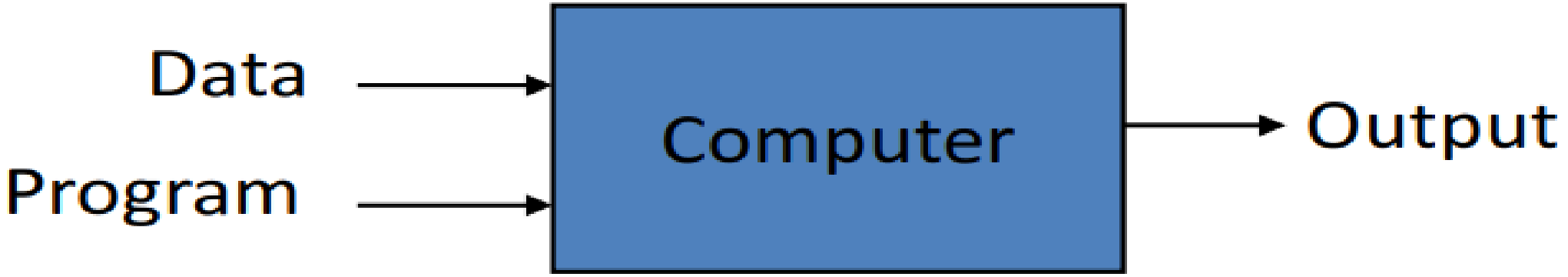
Takes time

Machine Learning Program : much shorter

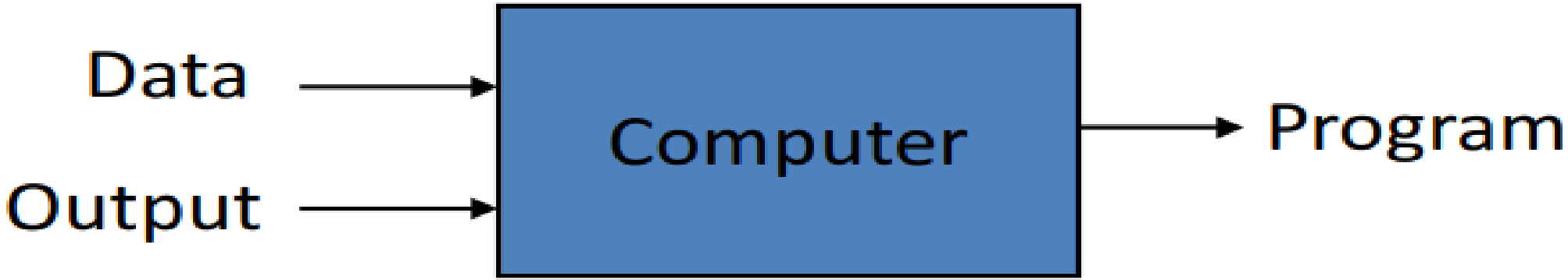
Easy to maintain

more accurate

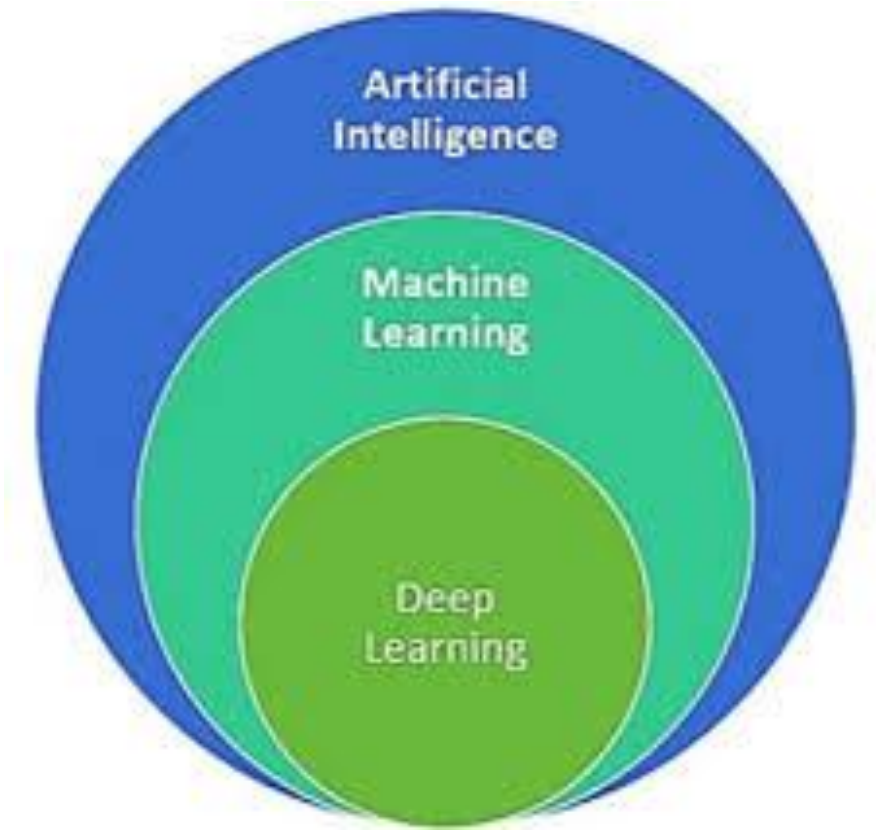
Traditional Programming



Machine Learning



Machine Learning



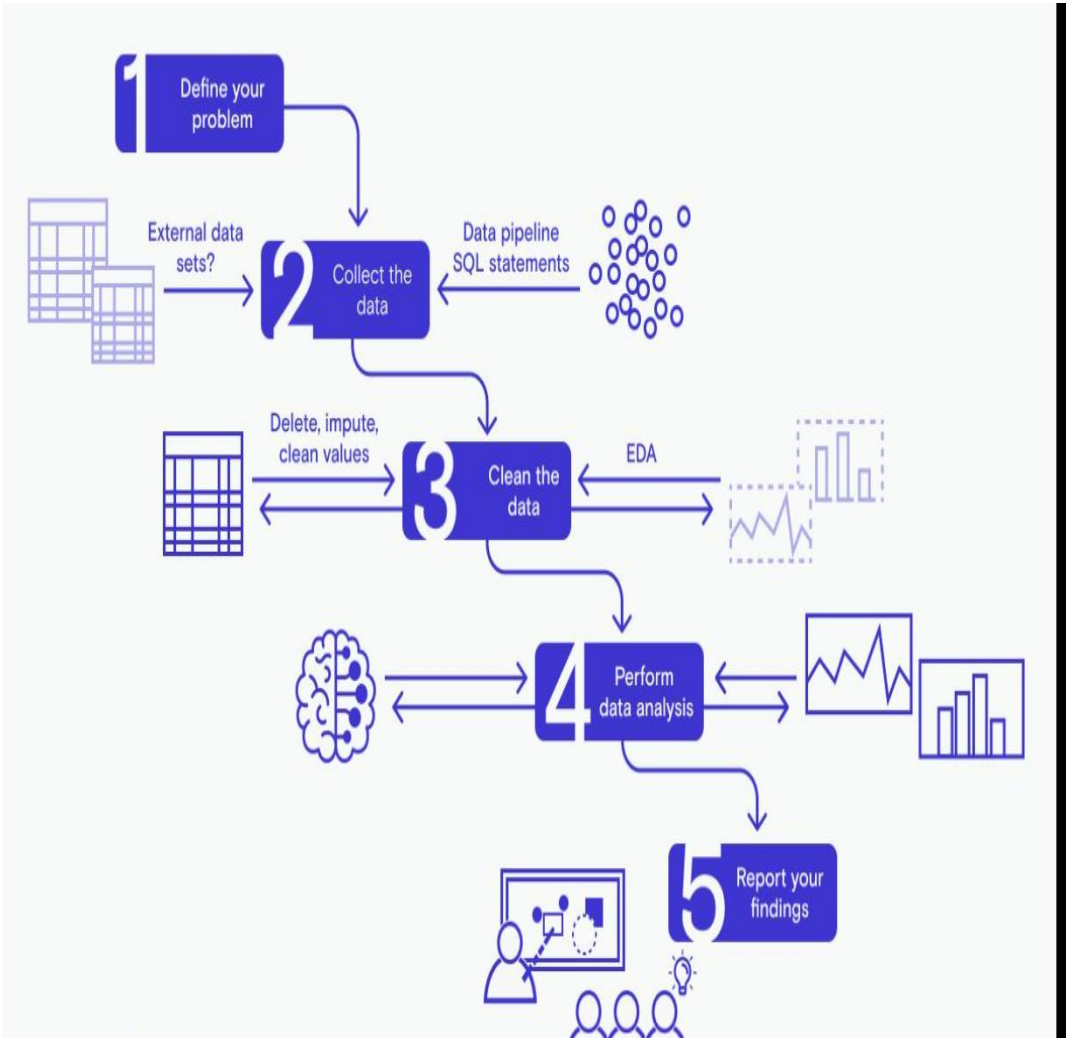
Machine Learning is a subset of Artificial Intelligence that enables machines to learn automatically from data and improve with experience, without being explicitly programmed.

Machine Learning is a field of Artificial Intelligence that enables machines to analyze data, visualize patterns, make predictions, and forecast future trends without being explicitly programmed.

Data Analysis is the process of examining, cleaning, and transforming data to discover useful information and patterns.

Tools for Data Analysis

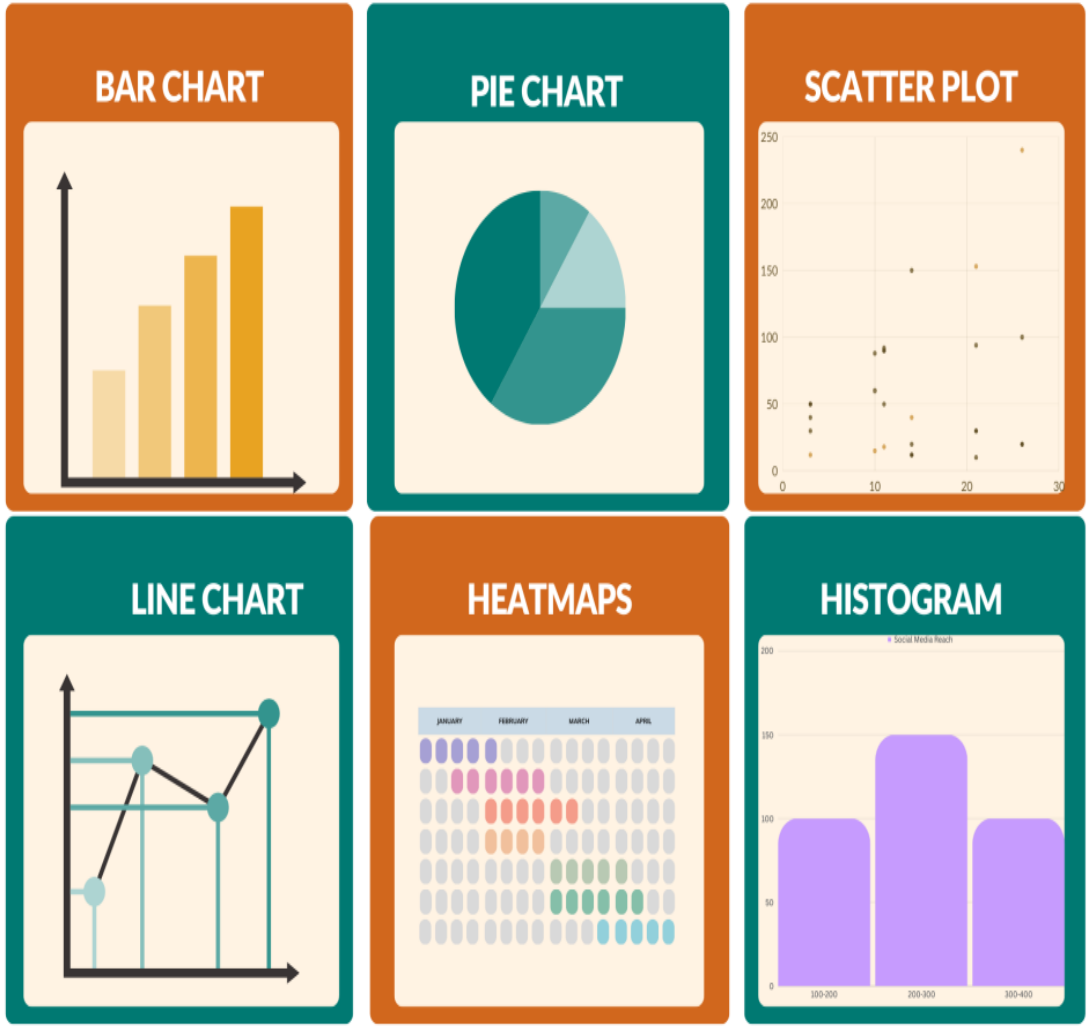
- **NumPy** – Numerical computation, arrays, matrices
- **Pandas** – Data handling, cleaning, filtering, aggregation
- **SQL** – Querying structured data
- **SciPy** – Statistical analysis, scientific computations



Data Visualization is the graphical representation of data using charts and graphs to understand patterns and trends easily.

Tools for Data Visualization

- **Matplotlib** – Basic plots (line, bar, scatter)
- **Seaborn** – Statistical & attractive visualizations
- **Tableau / Power BI** – Dashboard & business visualization



Prediction is the process of using historical data and machine learning models to estimate unknown or future outcomes.

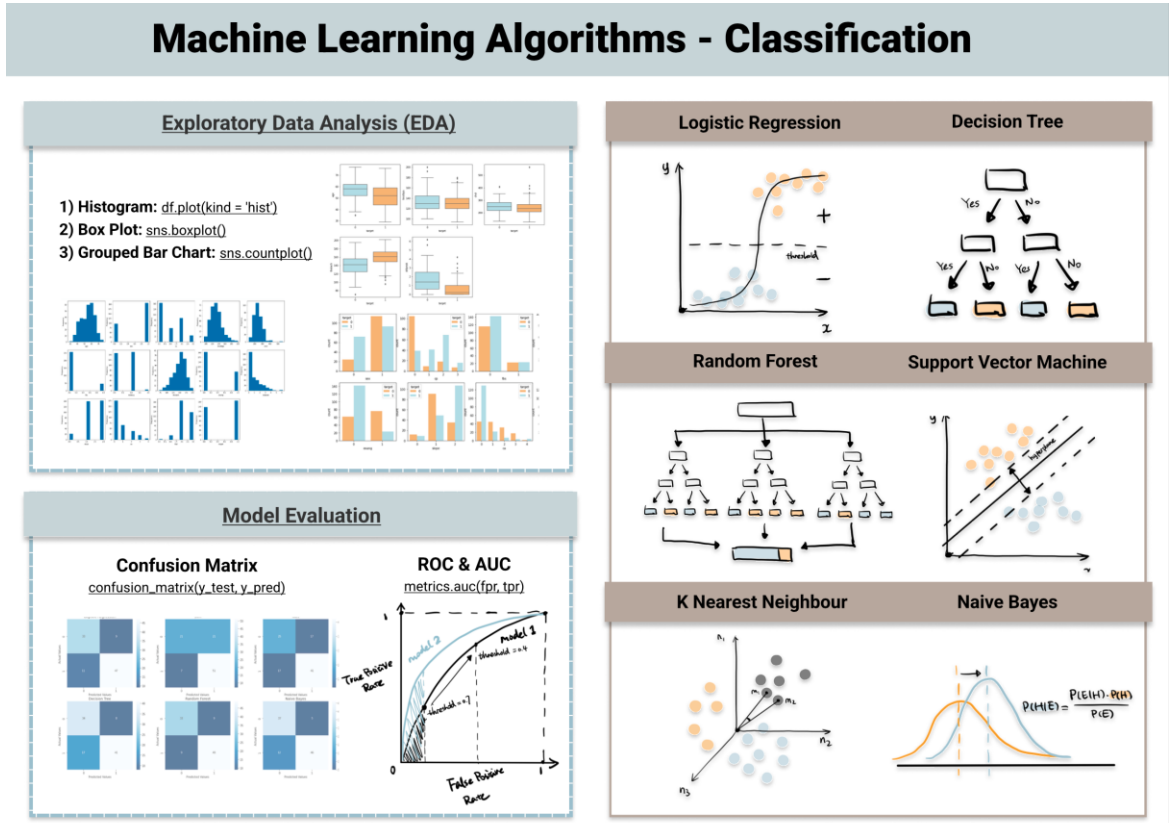
Tools for Prediction

Scikit-Learn –

- Linear Regression
- Logistic Regression
- KNN
- Decision Tree
- Random Forest

XGBoost / LightGBM – High-performance prediction

TensorFlow / PyTorch – Deep learning models



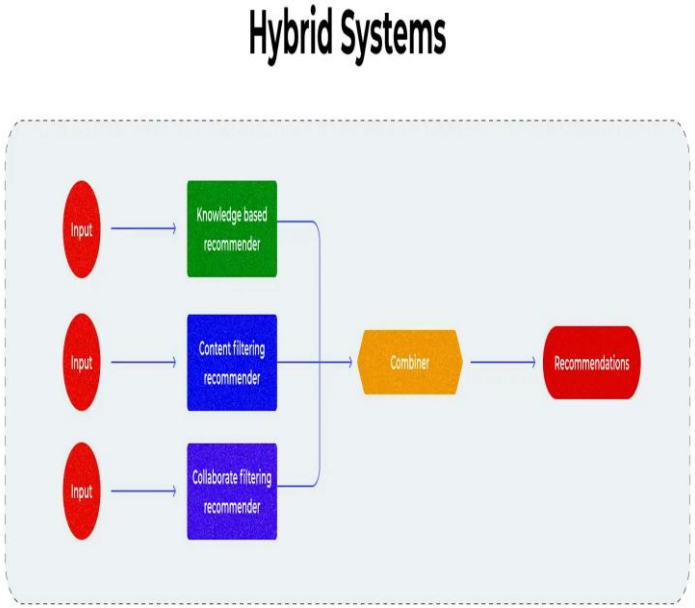
Applications of Machine Learning

Recommendation Systems (Netflix, Amazon, YouTube)

Machine Learning analyzes user behavior such as:

- What you watch
- What you search
- What you buy
- What you like or rate

Using this information, ML learns your preferences and recommends similar content or products.



Examples

Netflix

- If you watch action movies
- If you search for thriller or adventure films
- Netflix will recommend more action and thriller movies

Amazon

- If you buy an iPhone

  Amazon may recommend:
  - Headphones
  - Mobile cover
  - Screen protector

This is called personalized recommendation.

YouTube

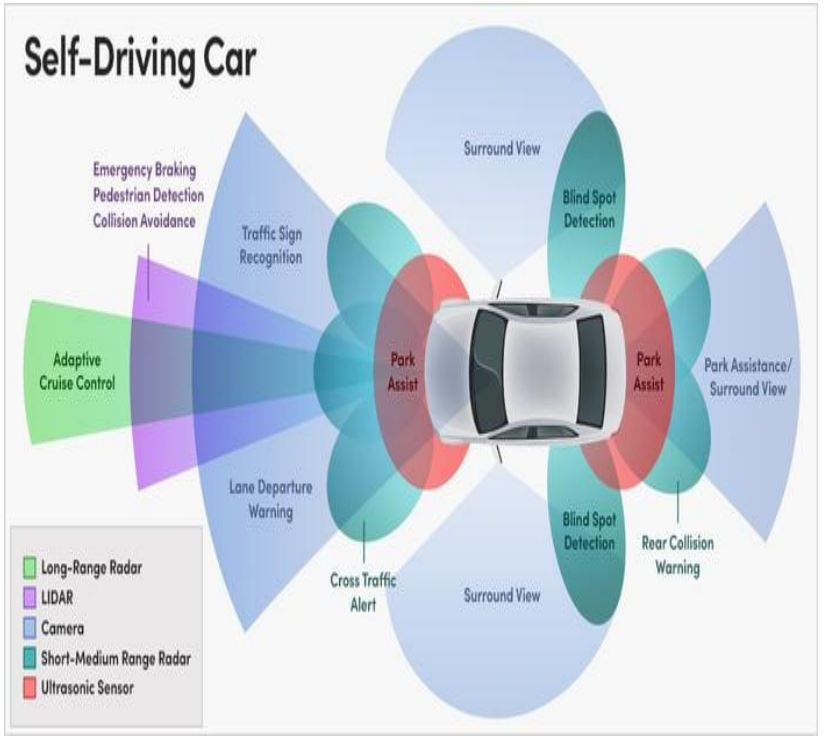
- If you watch coding tutorials
- If you like machine learning videos

  YouTube recommends similar educational videos

Self-Driving Cars

Machine Learning helps cars to:

- Detect pedestrians
- Recognize traffic signs
- Stay in lane
- Avoid obstacles

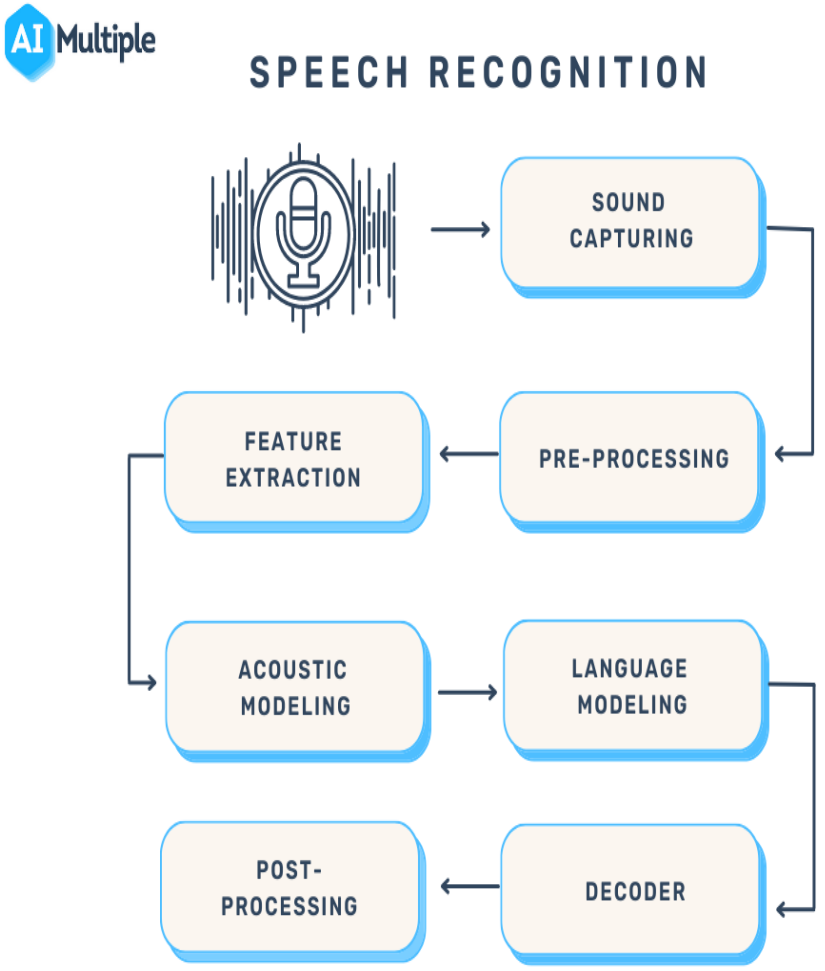


The car learns from camera, radar, and sensor data

Makes driving decisions automatically

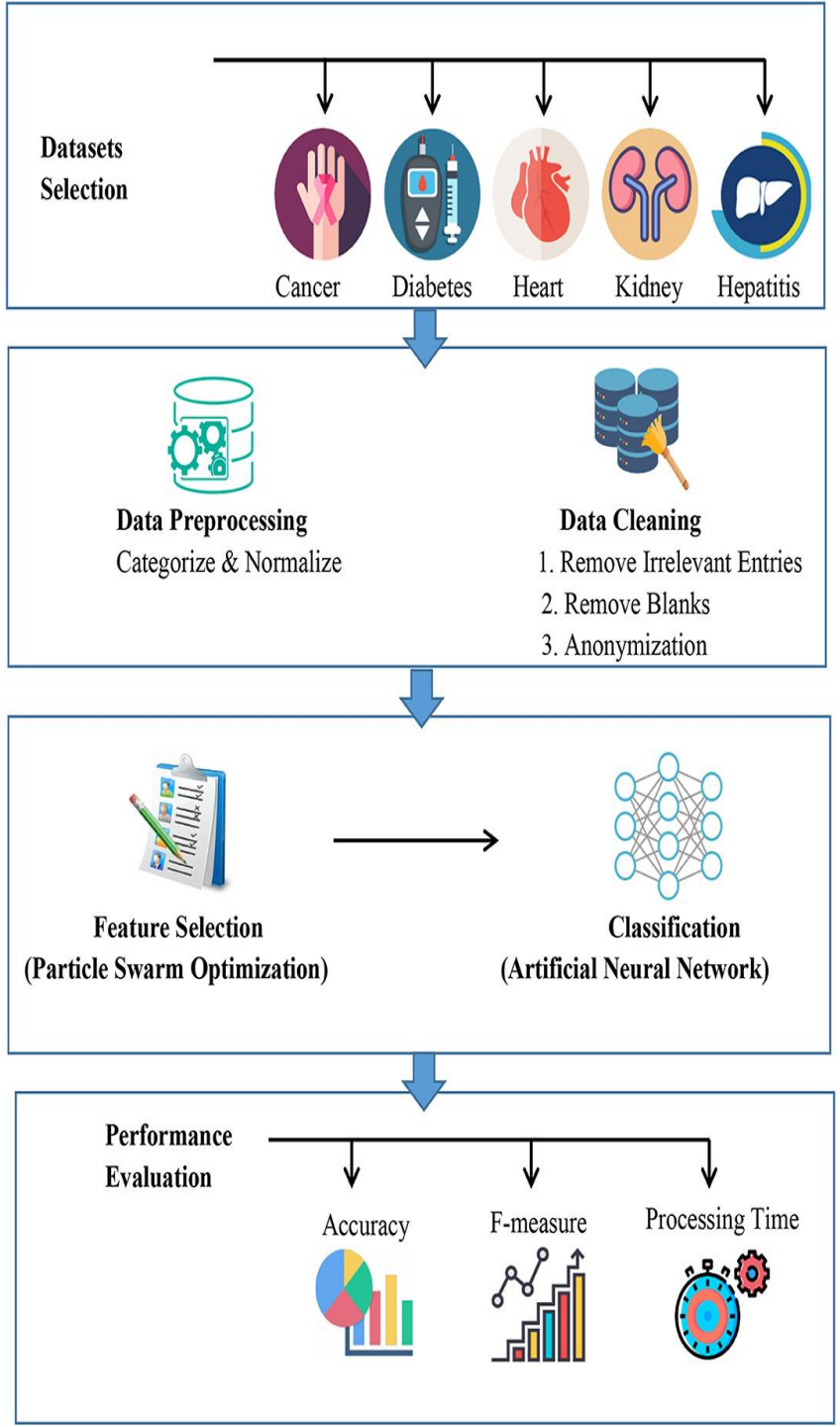
Voice Assistants (Siri, Alexa, Google Assistant)

- Converts speech to text
- Understands user intent
- Responds intelligently
- Example:
- “Play music”
- “What is the weather today?”



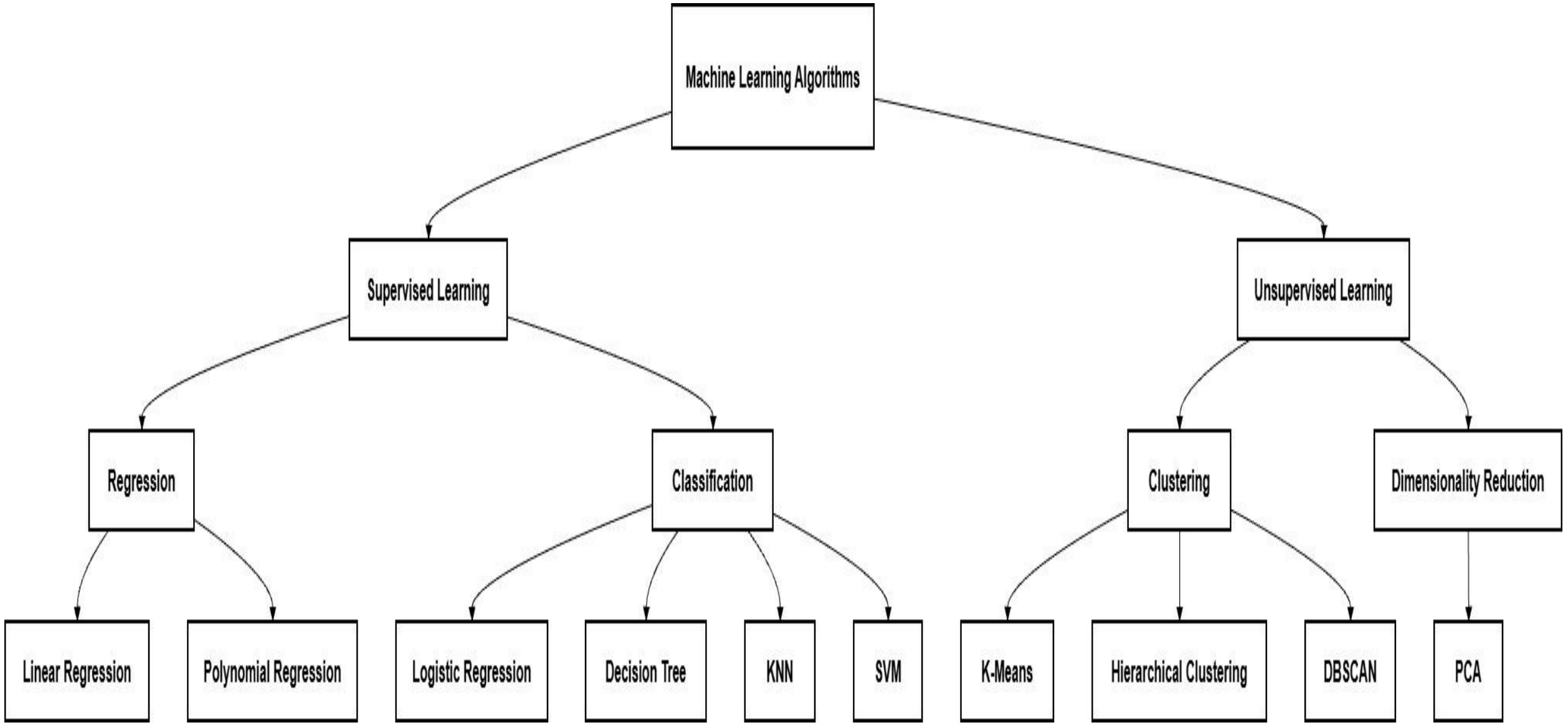
Healthcare & Disease Prediction

- Analyzes medical data
- Predicts diseases early
- Assists doctors in diagnosis
- Example:
- Predicting heart disease or diabetes
- Analyzing X-ray or MRI images

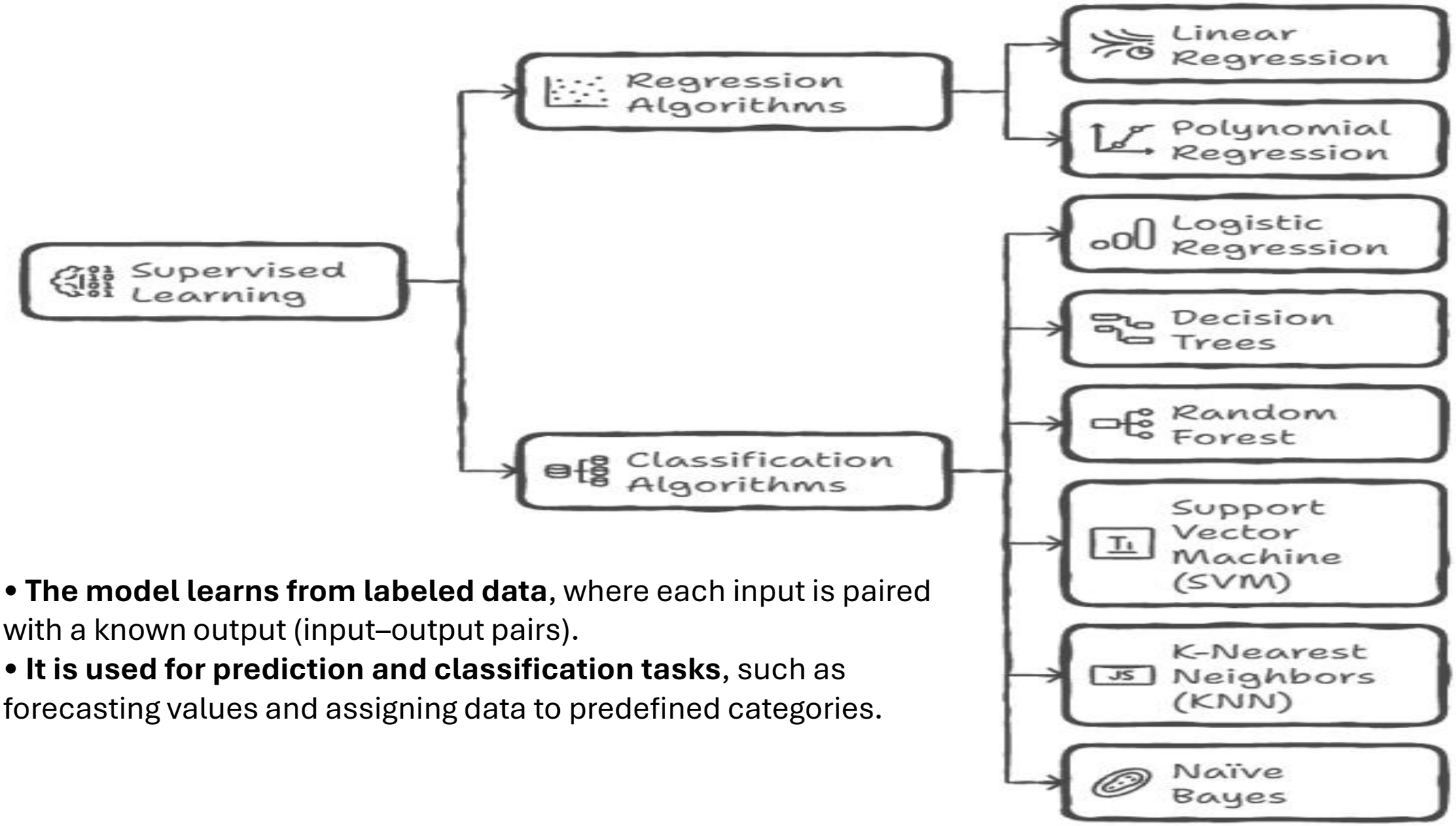


Types Machine learning algorithms

Machine learning algorithms are broadly classified into Two types based on how they learn from data:



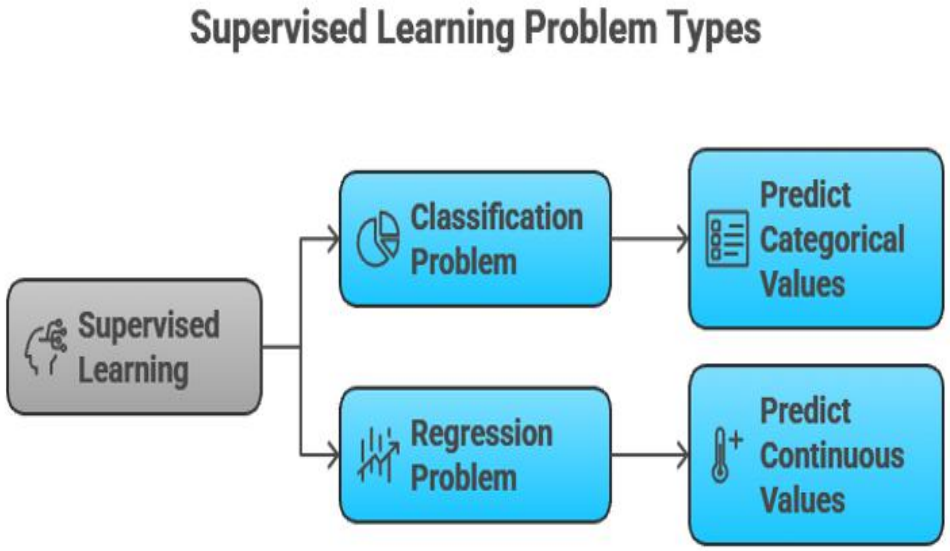
Supervised Learning



- **The model learns from labeled data**, where each input is paired with a known output (input–output pairs).
- **It is used for prediction and classification tasks**, such as forecasting values and assigning data to predefined categories.

Supervised Learning

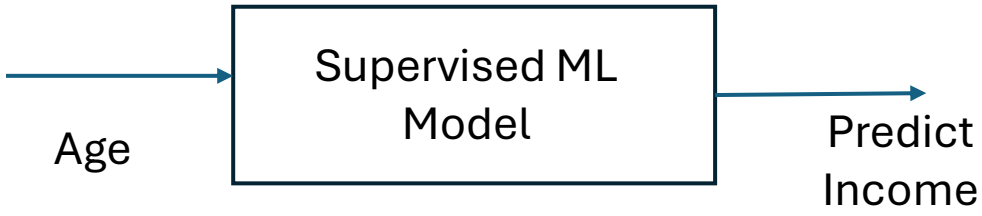
- Supervised Learning is a type of machine learning where historical and labeled data is applied and machine learning algorithm predict a value.
- Historical data means known data from the past.(for example on what price house have been sold in past).
- Machine learning algorithm learns from labeled data (i.e., input-output pairs).
- Labeled data means desired output is known.
- The model is trained using historical data, where each input (features) has a corresponding correct output (label).
- The goal is to learn a mapping function that can make accurate predictions on new, unseen data.



Customer	Age (Years)	Income (₹)
1	22	25,000
2	25	30,000
3	30	45,000
4	35	50,000
5	40	60,000
6	28	32,000
7	45	80,000
8	50	90,000
9	23	27,000
10	38	52,000

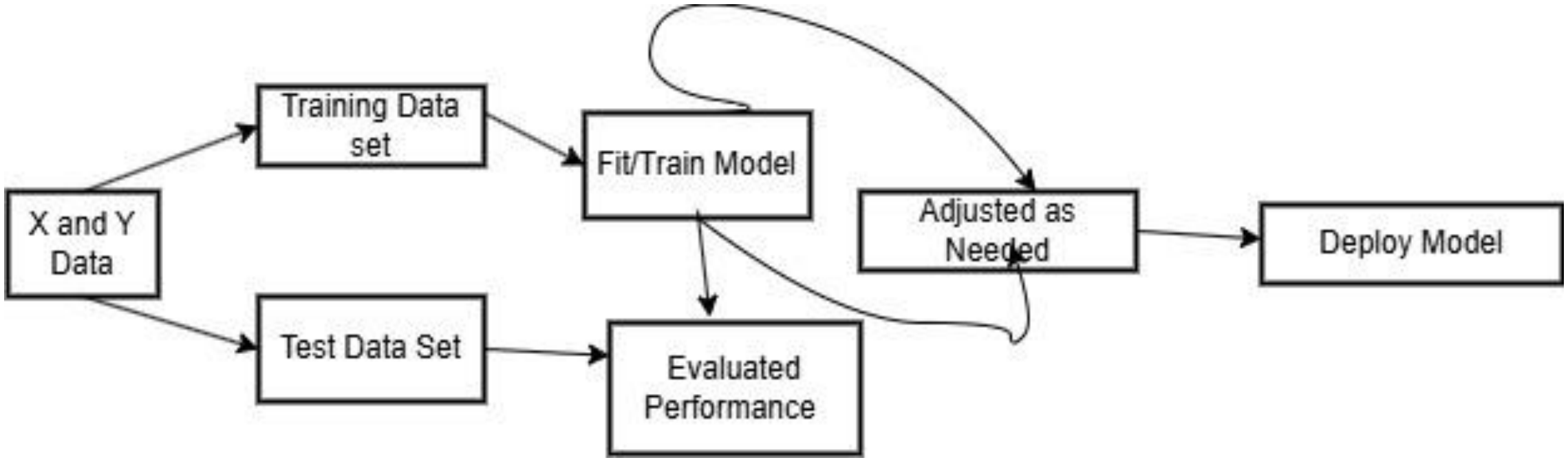
Customer record

- First, we will train our ML model with given dataset



Independent Feature (Input Feature): Age(input are the independent feature that is used to predict the output).

Dependent Feature (Target Variable): Income(A dependent feature is the output variable whose value depends on independent features)



Workflow of Supervised Machine Learning Model

Classification Problem:

Customer	Age	Income (₹)	Buy Computer
1	22	25000	No
2	25	30000	No
3	30	45000	Yes
4	35	50000	Yes
5	40	60000	Yes
6	28	32000	No
7	45	80000	Yes
8	50	90000	Yes
9	23	27000	No
10	38	52000	Yes

- In this data set Age and Income are the Independent features or input variables used to make predictions.
- Buy computer is Dependent Feature
- We will train our model which will predict output based on training data set.

Supervised Learning can solve two types of problem

1. Classification Problem: A classification problem involves predicting categorical (class) values, where the output belongs to a predefined category or label (such as Yes/No, Spam/Not Spam, or Disease/No Disease).Example: Tumor Prediction (Cancerous or Benign) in Supervised Learning

In this classification problem, we use historical medical data to train a machine learning model to predict whether a tumor is cancerous (malignant) or benign based on various features.

Handwriting Recognition Using Machine Learning (Supervised Learning Example)

Handwriting recognition is a classification problem where a model learns to recognize handwritten characters or digits from images. The most common dataset for this task is MNIST (Modified National Institute of Standards and Technology dataset), which contains 28×28 pixel grayscale images of handwritten digits (0-9).

Algorithms used to solve classification problem

- **Decision Tree –** Uses rule-based tree structure for decisions

  Example: Loan approval
- **Random Forest –** Combines many decision trees for better accuracy

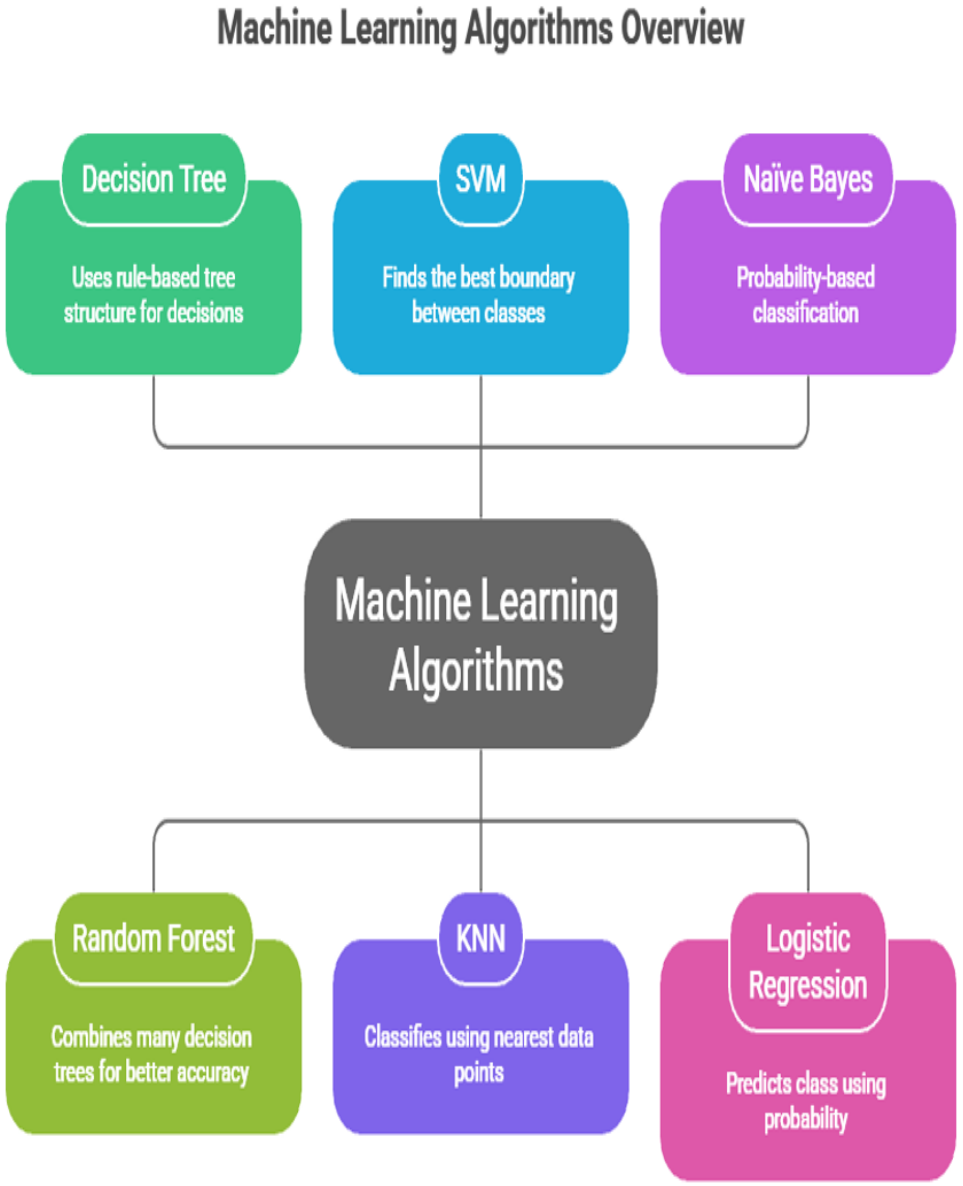
  Example: Disease diagnosis
- **SVM (Support Vector Machine)–** Finds the best boundary between classes

  Example: Handwritten digit recognition
- **KNN(K-Nearest Neighbors)** – Classifies using nearest data points

  Example: Recommender systems
- **Naïve Bayes –** Probability-based classification

  Example: Sentiment analysis
- **Logistic Regression –** Predicts class using probability

  Example: Email spam detection

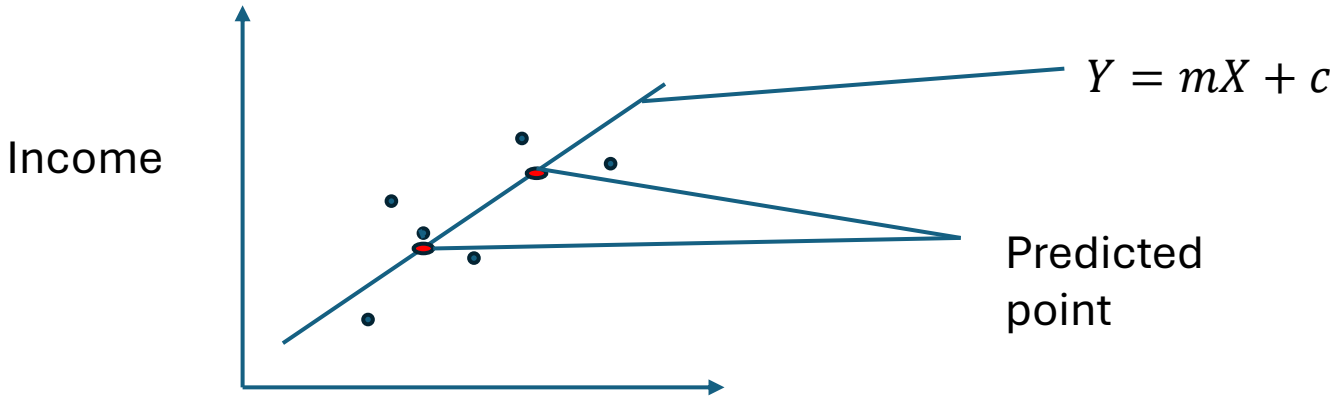


Regression Problem (Predict Continuous Values)

Customer	Age (Years)	Income (₹)
1	22	25,000
2	25	30,000
3	30	45,000
4	35	50,000
5	40	60,000
6	28	32,000
7	45	80,000
8	50	90,000
9	23	27,000
10	38	52,000

If we have a dataset with Age and Income, and we want to predict income using Linear Regression.
Then here:
Independent feature (X) → Age
Dependent feature (Y) → Income
This is a Supervised Learning – Regression problem.

The equation of **Simple Linear Regression** is:



If the output (dependent feature) is a continuous variable, then the problem can be solved using regression algorithms.

Regression Problem (Predict Continuous Values):

Regression is a **supervised learning technique** used to predict continuous numerical values, such as house prices, stock prices, temperature, or test scores.

House Price Prediction: A regression model is trained on past house sale prices (based on square footage, number of rooms, location) and predicts future house prices.

- Data set have feature(X) and label(Y)
- Data set is divided in four component X Train,Y Train,X test,Y Test for for evaluation.
- So if a new house comes on the market with a known area, number of bedrooms, and number of washrooms, the model can predict its expected price based on historical data and learned patterns.
- So for this model input is house feature and output is prediction of selling price.

	Features			Label	
	Area m^2	Bedrooms	Washroom	Price	
X Train	200	3	2	500000	X Test
	190	2	1	450000	
	230	3	3	650000	
Y Train	180	1	1	400000	Y Test
	210	2	2	550000	

House Price Prediction Dataset

Algorithms used to solve Regression problem

Linear Regression – Fits a straight line to predict values.

Example: House price prediction

Polynomial Regression – Fits a curve to model non-linear relationships in data.

Example: Curve fitting for growth trends

Unsupervised Learning

In **Unsupervised Learning**, there is no labeled output (such as house prices). Instead, the model identifies patterns or groups within the data.For housing data, unsupervised learning can be used for:

1. **Clustering (Grouping Similar Houses)**

- **K-Means Clustering**: Groups houses into clusters based on features like area, number of bedrooms, and location. For example, it can segment houses into "Luxury," "Affordable," and "Mid-range" categories.
- **Hierarchical Clustering**: Builds a tree-like structure to show relationships between different house categories.
- **DBSCAN**: Identifies housing price anomalies and clusters based on density.

2. **Anomaly Detection (Identifying Outliers)**

- Helps detect houses with abnormally high or low prices compared to similar properties.
- Techniques: **Isolation Forest, One-Class SVM, Autoencoders**

3. **Dimensionality Reduction (Feature Reduction)**

- If there are many features (e.g., location, area, amenities), **Principal Component Analysis (PCA)** can reduce complexity while preserving important patterns.

Clustering

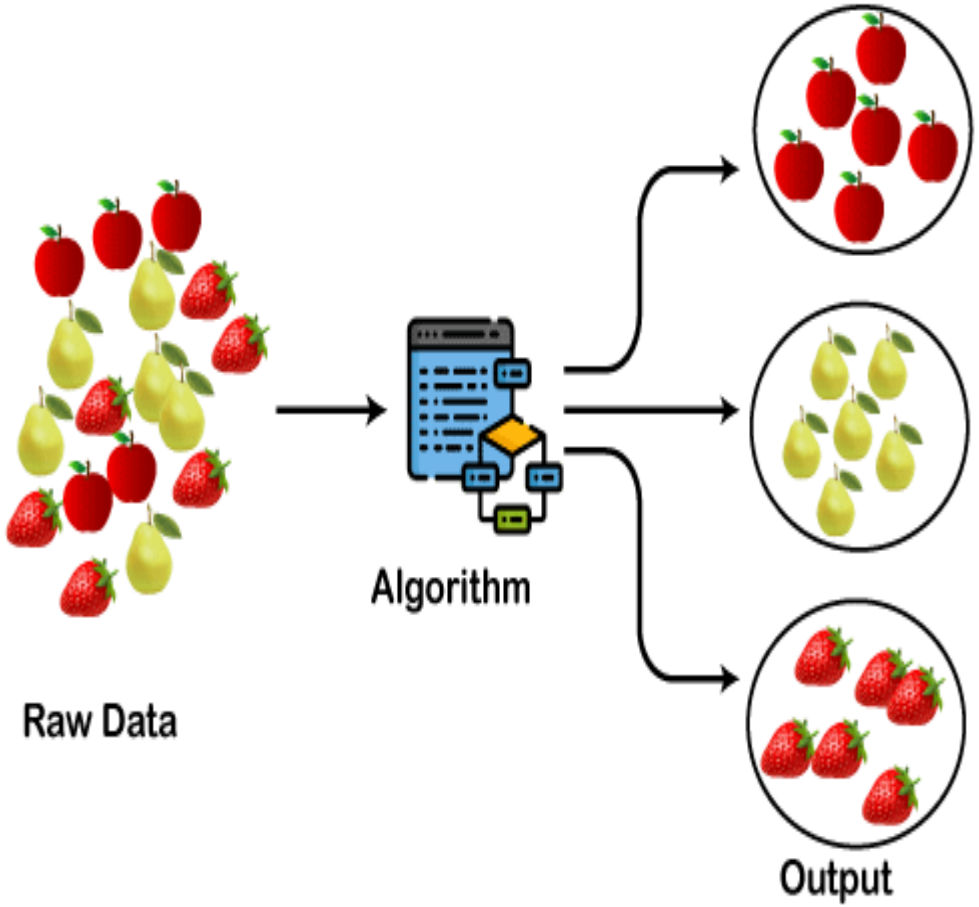
- Clustering is an unsupervised learning technique that groups similar data points into clusters without using labeled data.
- Clustering is a way of grouping data points into different clusters such that data points within the same cluster are similar to each other, while data points belonging to different clusters have less or no similarity with one another.
- Clustering is somewhat similar to the classification algorithm, but the key difference lies in the type of dataset used.
    - In classification, we work with a labeled dataset.
    - In clustering, we work with an unlabeled dataset.

Applications of Clustering:

- Market segmentation
- Statistical data analysis
- Social network analysis
- Image segmentation
- Anomaly detection

Example of Fruit Clustering (Unlabeled Data)

- In clustering, fruits are grouped based on similar features such as weight, sweetness, and color without giving labels in advance.



Fruit No.	Weight (g)	Sweetness (1–10)	Color Code
1	150	9	Red
2	160	8	Red
3	120	7	Yellow
4	130	6	Yellow
5	200	5	Green
6	210	4	Green

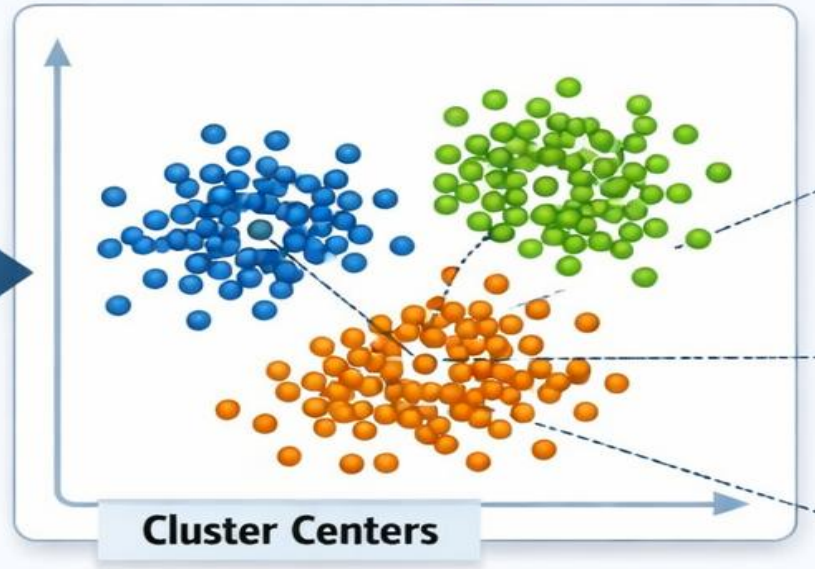
K-Means Clustering

House Clustering Based on Area, Bedrooms & Location

Housing Data



K-Means Algorithm

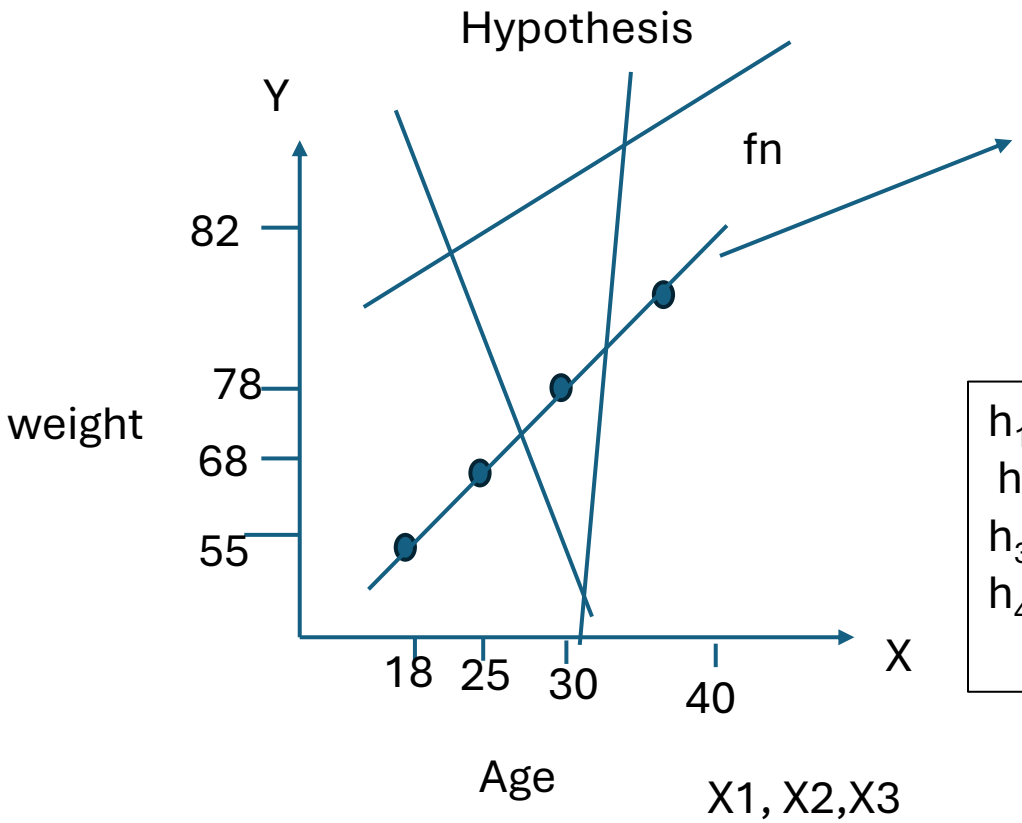


Segmented Clusters



High-End "Luxury" Medium "Mid-Range" Low-Cost "Affordable"

Hypothesis Space



Y=mx+c

Function/Hypothesis

Age	Height (cm)	BMI	Activity	Weight (kg)
18	165	21	2	55
25	170	23	3	68
30	175	26	1	78
40	168	28	1	82

h_1 : Weight is calculated only from age
h_2 : Weight depends only on height
h_3 : Weight depends only on Activity
h_4 : Weight depends only on BMI

The set of all hypotheses is called the hypothesis space

$h(x) = \hat{y}$

where:

- h(x) = hypothesis function
- x = input/features
- $\hat{y}$ = predicted output

A hypothesis function is a mathematical function used to make predictions or represent an assumption about the relationship between input variables and an output variable.

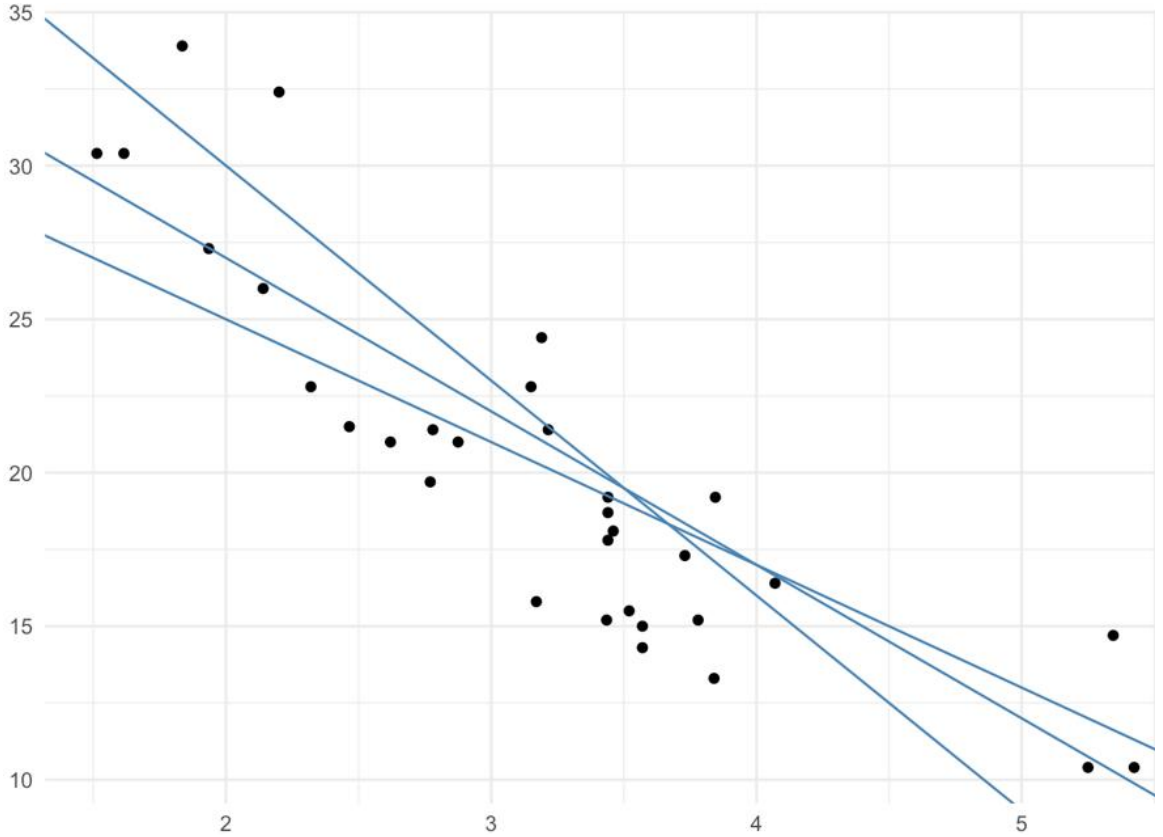
In Machine Learning, it is often represented as:

- When we train a machine learning model, the goal is to find a function that best maps input to output.
- This function is called a hypothesis
- The hypothesis space (H) is the set of all possible hypothesis that a model can consider while learning from data.

Hypothesis Space(H)

if x=input space and y=output space then H={h:X → Y}

This means that H is a set of functions that map input(x) to output (Y)



- If the hypothesis space is too large, it can make learning difficult because there are too many possibilities to choose from.
- If it's too small, the model might not capture the complexity of the data and could perform poorly.

Decision Tree Hypotheses

Let

$$x = (\text{Age}, \text{Height}, \text{BMI})$$

h1(x)

$$h_1(x) = \begin{cases} \text{High} & \text{if BMI} > 25 \\ \text{Normal} & \text{otherwise} \end{cases}$$

h2(x)

$$h_2(x) = \begin{cases} \text{High} & \text{if Age} > 30 \text{ and BMI} > 26 \\ \text{Normal} & \text{otherwise} \end{cases}$$

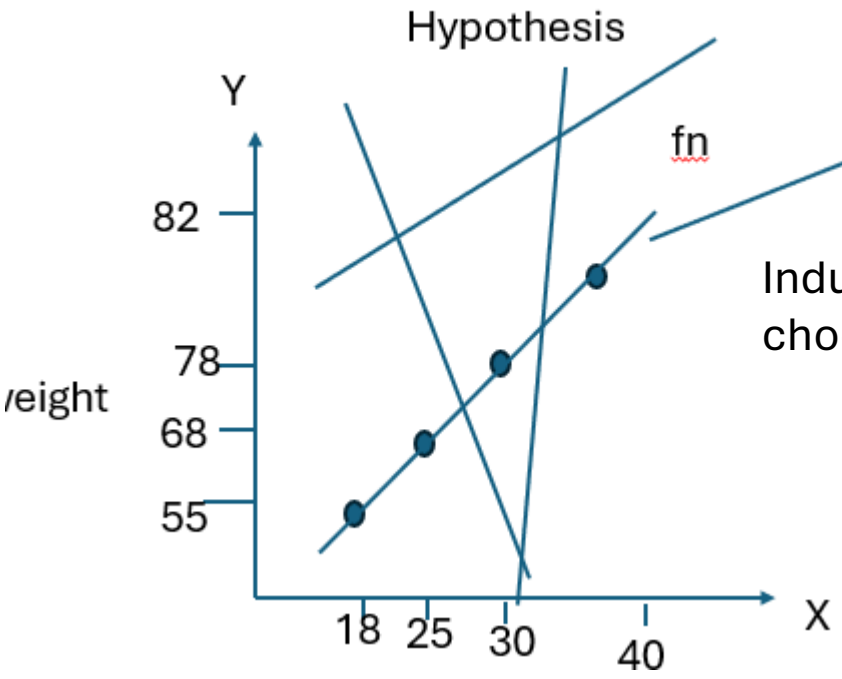
h3(x)

$$h_3(x) = \begin{cases} \text{High} & \text{if Height} < 160 \\ \text{Normal} & \text{otherwise} \end{cases}$$

Hypothesis Space(H)

H={h1,h2,h3}

Inductive Bias



Inductive Bias

Inductive bias is the set of assumptions used by a machine learning model to choose a suitable hypothesis from a large hypothesis space.

- The hypothesis space is usually very large. If a model is allowed to randomly choose any hypothesis, it may lead to overfitting (overly complex models).
- To address this, machine learning models apply certain rules to narrow down the search for a suitable hypothesis. These rules or assumptions are called inductive bias.

Age	Weight
10	30
20	50
30	70

Now you want to predict weight for age = 40.

Possible models:

- Straight line
- Curve

Which one will the algorithm choose?

- It chooses a straight line, because it assumes the relation is linear

This assumption is the inductive bias

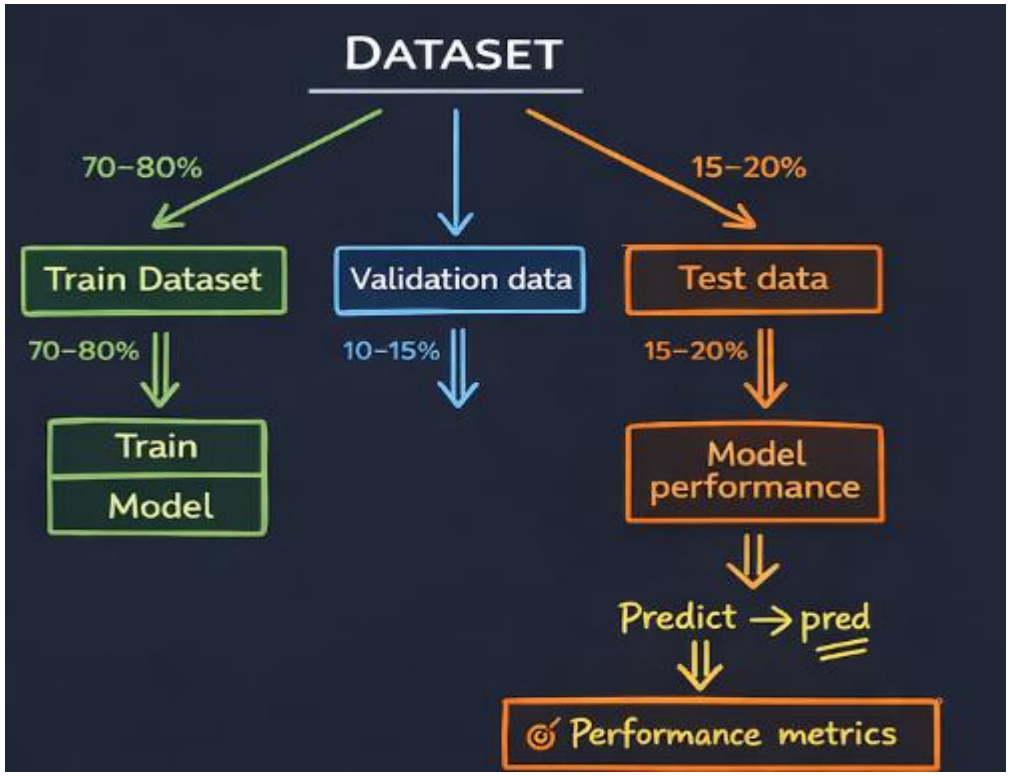
Training Testing and Validation Dataset

When training a Machine Learning model, we typically split the dataset into three parts

- Training Set: Used to train the model by learning pattern from the dataset
- Validation Set:Used to fine tune the model by selecting the best hyperparameters and preventing overfitting.
- Testing : Used to evalute the final performance of the trained model on completely unseen dataset.

Training Dataset (Learning Phase)

- The training set is used to train the model
- The model learns patterns, relationships, and rules
- Model parameters are adjusted here
- Usually 70–80% of the data use for this purpose



Testing Dataset (Evaluation Phase)

- The test set is used to evaluate the model performance
- Model has never seen this data before
- Used to calculate accuracy, error, precision, etc.
- Usually 20–30% of the data use for this purpose

Validation: After training a model, we do not accept it directly; instead, we use validation data to fine-tune it by adjusting model settings to improve performance and ensure better generalization.

Why Do We Split Data

- If we test the model on training dataset only:
- Model may memorize data
- Accuracy looks high but fails on new data
- This is called Overfitting
- So, testing data must be separate

Common Train–Test Split Ratio

Training	Testing
70%	30%
80%	20%
75%	25%

Most common: 80% Training – 20% Testing

	Features			Label
	Area m^2	Bedrooms	Washroom	Price
Training	200	3	2	500000
	190	2	1	450000
	230	3	3	650000
Testing	180	1	1	400000
	210	2	2	550000

Main Challenges of Machine Learning

Is the model perfect:

- When we train a model using a given dataset ,it is difficult to say whether the model is perfect or not.
- Because we evaluate the performance of the model using the training and testing dataset.
- But this shows how well it performs on the specific data.
- The real test of the model accuracy happens when it is used on new ,unseen data in the future.
- Only then we can determine ,model is working well or not.

Which Model to Choose:

- As we have seen , different ML task ,like classification or regression task there are many models to choose.
- However, selecting the best model is challenging there is no single criterion for determining the most suitable one.

Sufficient/Insufficient Data:

- It is often unclear whether a dataset is sufficient.
- While we have data , we may not know if it is enough to train a reliable model and make an accurate prediction.

What is Data Quality:

- Data Quality may be checked by looking at
  - Error in Data
  - Missing Value
  - Noise in the Dataset
- If the model has too many issues and model learns from it,the model may not be able to learn the best pattern.

How good are result?

- If a model is trained on a given dataset , it is difficult to determine whether it is good or not.
- This is because evaluation is based on the available dataset,but its performance on future dataset remain uncertain.
- A model can only be considered as good if it perform well on unseen or future dataset.

Are the data represented Correctly:

- Are height and weight enough? Should we look at age also
- How should we represent age-As a number or as a categories like young , Middle- aged and old?
- Data can be represented in different ways , making it difficult to determine whether it is in the correct format.

How to ensure generalization:

- An ML model may perform well on the given data , but its performance on future data in uncertain.
- This Concept is known as generalization
- Means how much our model is general
- Which refer to how well the model performs on new,unseen data
- Predicting generalization error is difficult during training and testing,as it only becomes clear when the model encounters real world data.

Example of Bad Data

- Insufficient quantity of training data
- Non representative training data
- Poor quality data
- Irrelevant Feature

Example of Machine Learning Model Problem

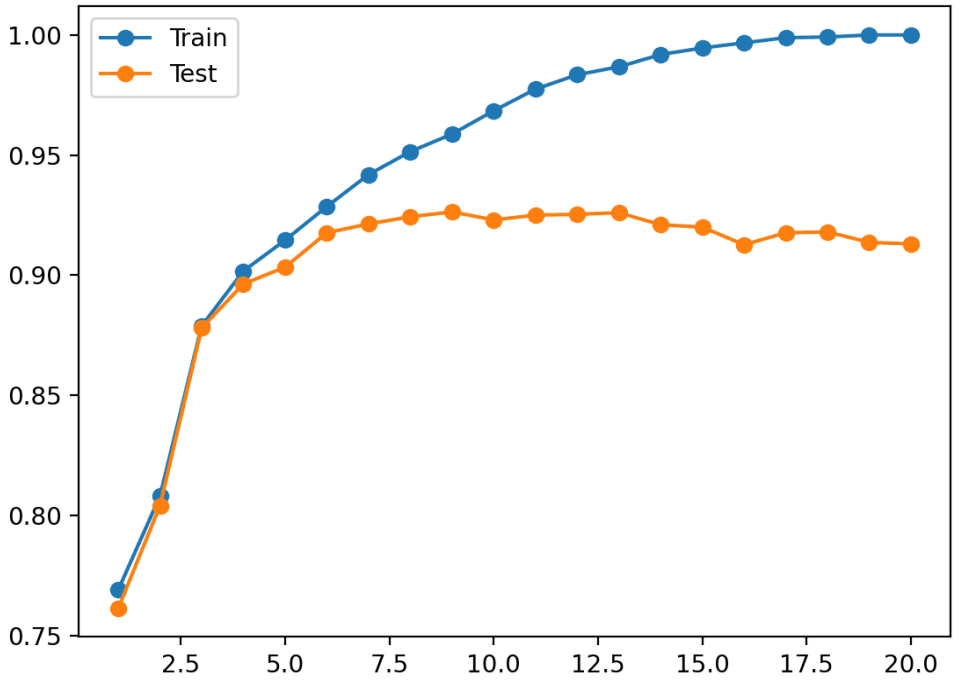
- **Overfitting**
- **Underfitting**

Overfitting

- Overfitting happens when a machine learning model learns the training data too much, including noise and small details, but fails to perform well on new (unseen) data.
- Training accuracy is very high but Testing accuracy is low.

For Example:

- A student memorizes answers for a question paper.
- He/She scores 90% in the practice test (training data).
- In the final exam (new questions), he/she scores 40%.
- This is called overfitting .



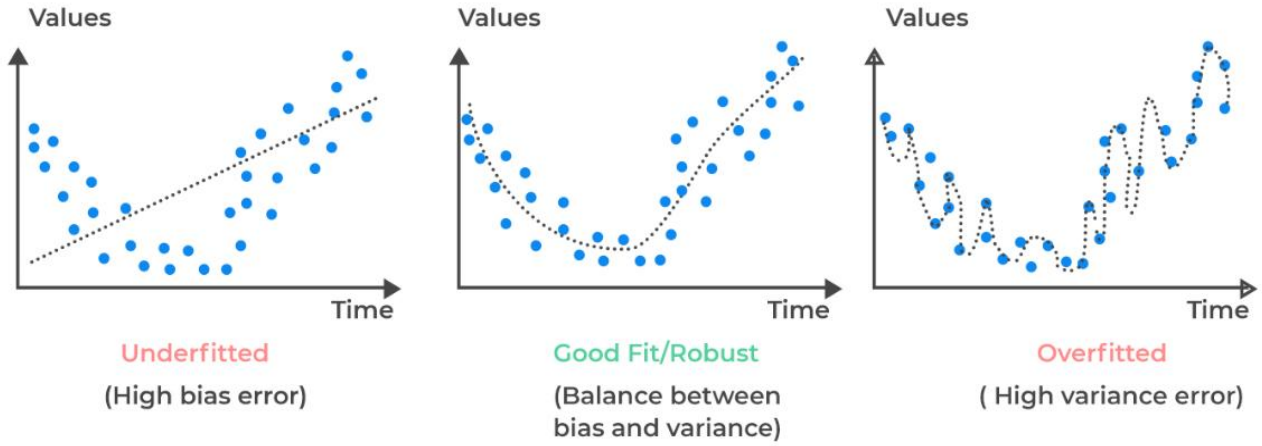
Why Overfitting Happens?

Model Performs well on training data , but it does not generalize well(high test error):

- When a model performs well on training data but not working with test Dataset
- This means model has learned pattern , including noise , from the training set but fails to generalize to new data.

Happens When:

- Model is too complex but dataset is too simple



For Example:

- Suppose we have data of 10 students showing hours studied vs marks.
- If we apply a high-degree polynomial model, the curve will pass through every training point.
- The model works perfectly for training data, but when a new student comes, the prediction is incorrect.
- This happens because the model has learned data points, not the pattern.

A high-degree polynomial model is a very complex model that fits all training data points exactly, including noise.

Noisiness of the training data:

Training data may contain noise, such as:

- Wrong values
- Measurement errors
- Outliers(Outliers are data points that are very different from the rest of the data and do not follow the normal pattern)

Noise does not represent the actual trend in data.

For Example:

- While predicting salary based on experience, one employee may have a very high salary due to a special case.
- If the model tries to learn this value, it learns incorrect information.
- A complex model tries to fit this noise, which changes the learned pattern and reduces performance on new data.

Possible Solutions

- Simplify the model by selecting one or fewer parameter

Complex Model (Many Parameters)

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 x^4$$

- Many parameters
- Fits training data very closely
- High risk of overfitting

Simplified Model (Fewer Parameters)

$$y = \beta_0 + \beta_1 x$$

- Only two parameters
- Captures general trend
- Works well on new data

For Example: Writing a simple answer instead of a complex explanation. Easier explanation can apply in any new situations

- By reducing the number of attribute in the training data
- By gathering more training data
- By reducing noise in the training dataset

Constraining a model to make it simpler and reduce the risk of overfitting is called regularization

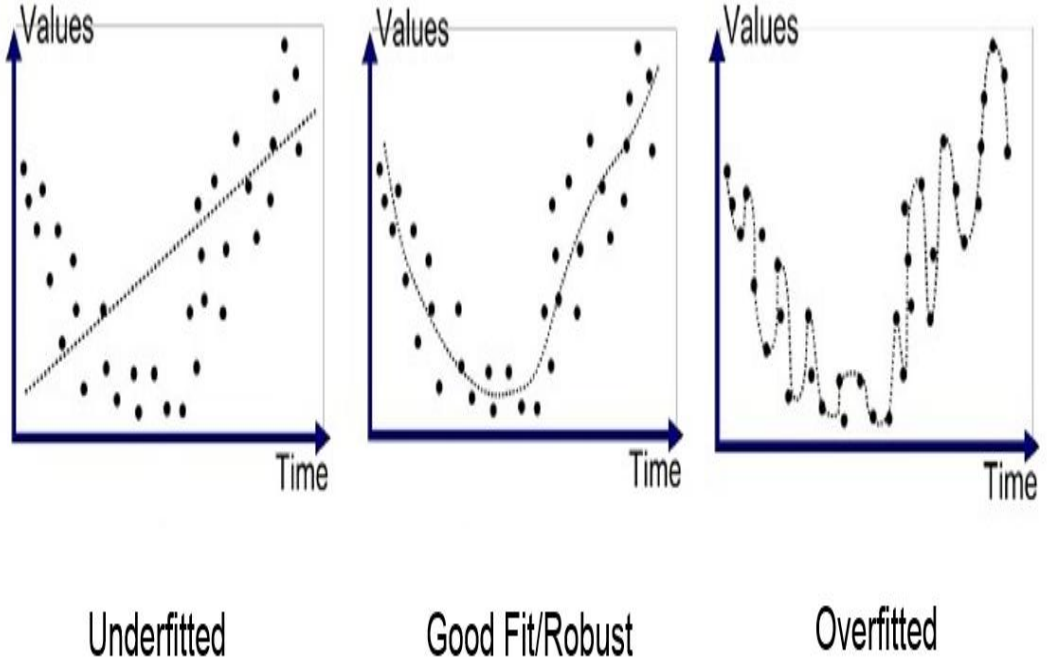
Underfitting

Underfitting happens when a machine learning model is too simple to learn pattern in the data , leading to poor performance.

How do we ensure that a model is Underfitting?

High Training Error and high test Error:

- When we train our model on the training data ,there may be a high training error.
- After training when we make prediction ,there may be many error in prediction.
- If the model does not perform well on the training data,it will also not perform well on test data.
- Therefore, if a model is underfitted to the traing data,it will definitely not work well on test data.



Underfitting Occurs when:

Model is too simple to learn the underlying structure of the data

Real-life Example

- Predicting house prices using only house size, location
- while ignoring : number of rooms ,facilities
- The model lacks important information so underfitting occur.

We have very less amount of data.

- When the house price dataset is too small:The model does not see enough examples
- Important patterns (location, demand, amenities) are missing
- Model parameters are poorly learned
- As a result, the model fails to learn the underlying structure of house prices.

If model is underfitted , adding more training data will not help.

- If the model is very simple,so with more data it will fail to learn relationship.

How to fix underfitting

Use a more powerful model.

- Polynomial regression instead of linear regression.
- Deep decision trees with greater depth instead of simple tree.
- CNN instead of Basic ML Model.

Feature Selection & Engineering

- Remove irrelevant or highly corelated feature that might introduce noise.
- Use Principal Component Analysis (PCA) to reduce dimensionality.

Gather More Training Data

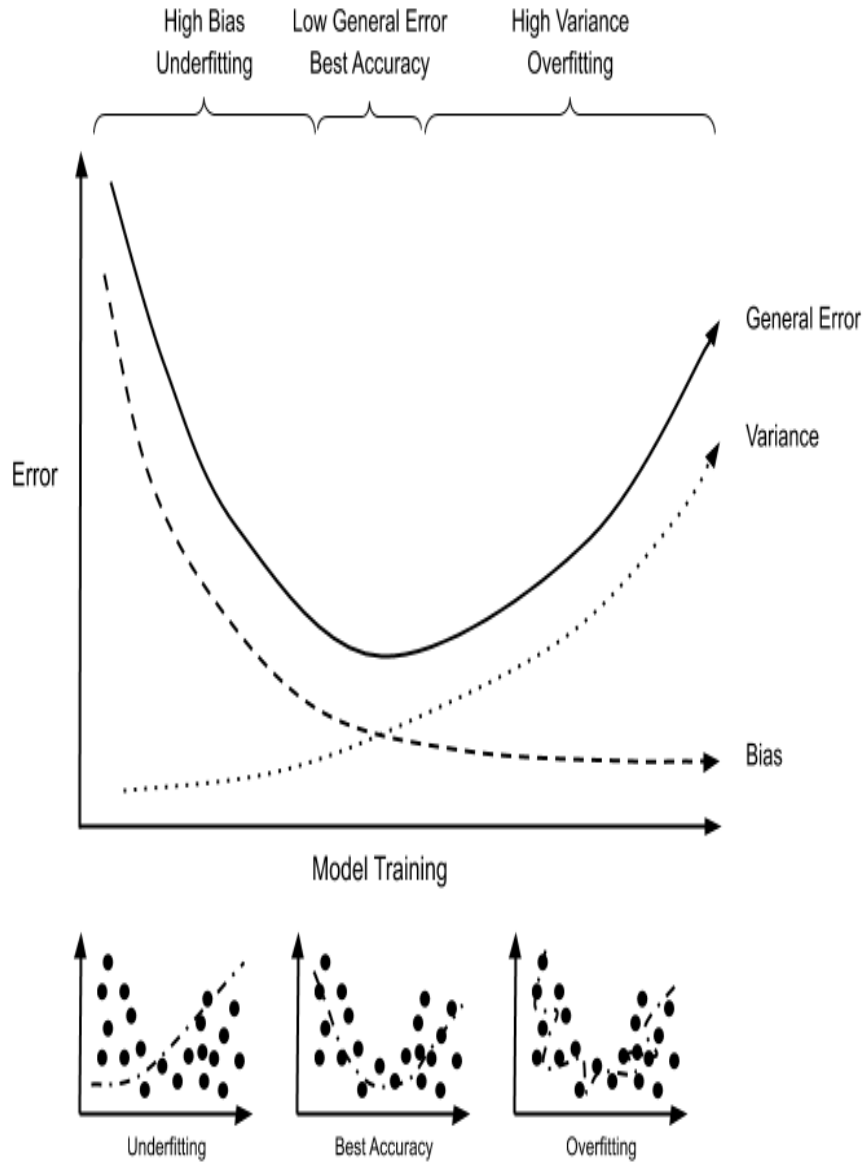
- If possible ,collect more training data to help the model learn better general patterns instead of memorizing specific example.

Reduce Noise

- By handling Missing Values,Fill using:Mean / Median / Mode
- Remove outliers
- Correct Wrong Labels: Re-check mislabeled data,Especially important in classification
- Smooth the Data, Apply:Moving average,Binning
- Apply proper data preprocessing

Bias-Variance Tradeoff

- The Bias–Variance Tradeoff describes the balance between bias(underfitting) and variance(overfitting) in machine learning model
- **Prediction Error:** There is two types of errors that affect a machine learning model's performance on unseen data.
    - Bias Error
    - Variance Error
- A model with high bias is too simple and underfits the data , while a model with high variance is too complex and overfits.
- The goal is to find a balance where the model captures pattern without memorizing noise, ensuring good generalization ,cross validation and feature selection help to achieve this balance.



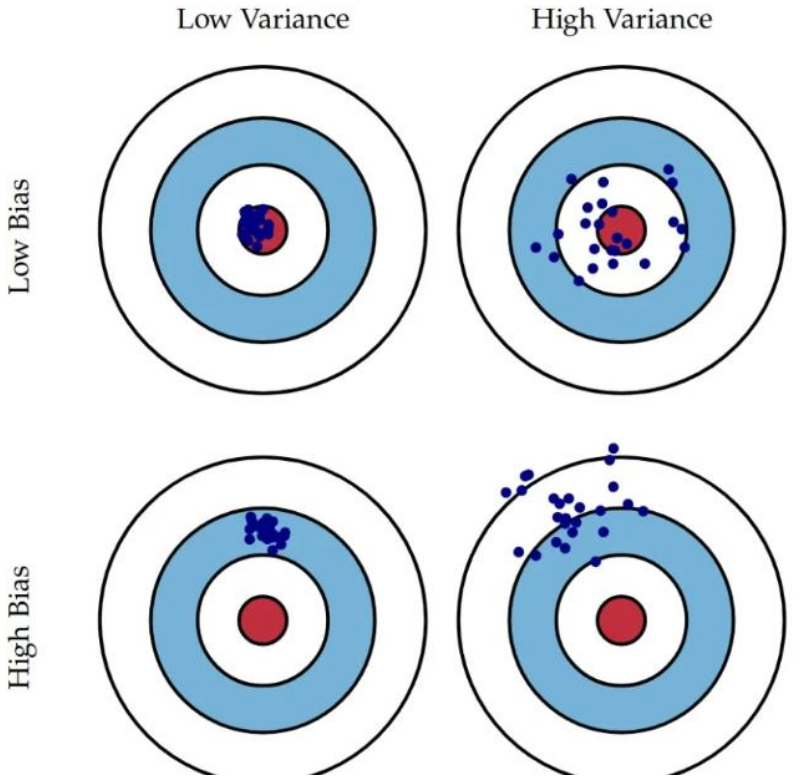
Bias Error

- Bias model occurs when a model is too simple to capture patterns in the data , leading to underfitting
- It result in high errors on both training and test data
- To reduce bias, use a more complex model, add relevant feature or decrease regularization.
- Bias is difference between average prediction of our model and the correct value which we are trying to predict.
- Model with high bias pays very little attention to the training data and oversimplified the model.
- It always leads to high error on training and test data
- A high level of bias can lead to underfitting, Which occur when the algorithm is unable to capture relevant between feature and target output.
- A linear Algorithm often has high bias, A linear model assumes that if input increases by a fixed amount, the output also changes by a fixed amount.
- Relationship is a straight line

$$y = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

For Example:

- Predicting house price using only *house size* with a straight line, while price also depends on *location, age, amenities*.
- The model cannot learn the complexity → high bias.

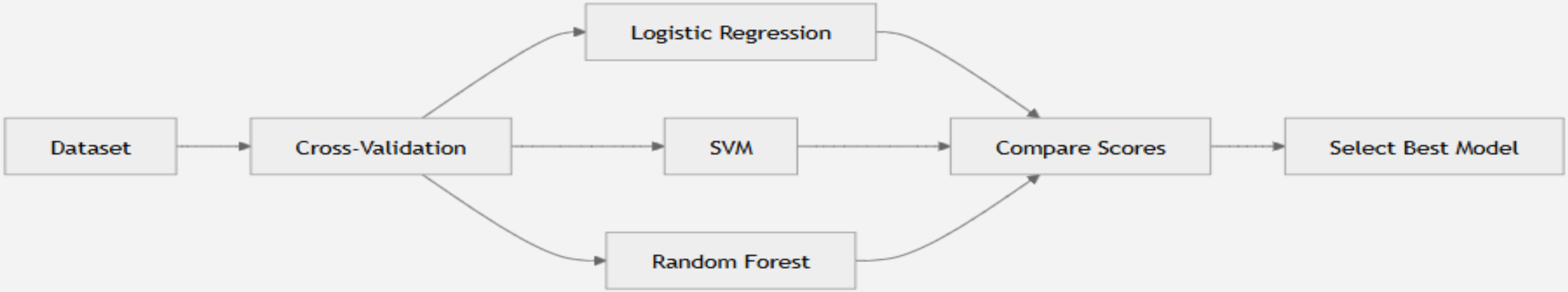


Variance(Changing in Dataset)

- Variance in machine learning refer to how much a model's prediction change when training on different dataset
- High variance means the model is too sensitive to small changes in data.
- Leading to overfitting , while low variance helps in better generalization
- Variance indicates how much a model's prediction change when trained on different dataset.
- High variance means the model is too sensitive to training data , leading to overfitting , while low variance improves generalization
- A model with high variance focuses too much on the training data
- Learning noise and fails to generalize to new data , leading to overfitting
- This can be fixed using regularization , more training data.
- If high variance , as a result model perform well on training data but has high error on test data

Cross Validation

- Cross-validation is a technique used to evaluate the performance of a machine learning model by dividing the dataset into different training and validation sets multiple times.
- We can use cross-validation to compare multiple models and select the model that performs best on unseen data.



Learn (Train)

Practice

Understand Concepts

Solve Problems

100 math questions
used to train the student

Learn / Train →



Student

- Understands concepts clearly
- Practices well
- Learns effectively

← Test

Q1	100/100
Q2	100/100
Q3	100/100
...	
Q100	100/100

Few questions from same 100 math questions that student was trained on

↓

FULL MARKS!
Student will get
100/100

What happens if we use the full dataset for both training and testing?

- The model is evaluated only on the training data and is not tested on unseen data.
- The model may learn or memorize specific training examples instead of learning generalizable patterns.
- Training accuracy becomes very high, while performance on unseen/test data may remain poor.
- The model has poor generalization ability and may perform poorly on real-world data.
- The difference between training and real-world situations performance becomes large.
- This is known as overfitting.

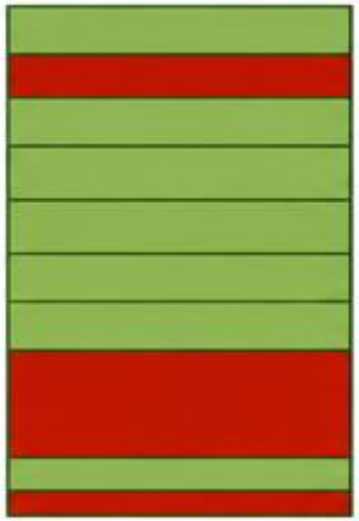
So, testing on the same data is not a reliable evaluation method.

There is two solution for this problem

Solution 1: Train–Test Split

Suppose we have 100 math questions.

Training Data
(70 Questions)



70%

- Used to train the model
- The student learns from these questions
- Model finds patterns and concepts

Train →



← Test

Test Data
(30 Questions)

Q1
Q2
Q3
...
Q30

30%

- Not seen during training
- Used to evaluate the model
- Checks how well the model performs on unseen data
- Gives real performance of the model

Goal

Learn the concepts from training data and perform well on unseen questions.

Purpose

To evaluate how well the model (or student) generalizes to new, unseen questions.

Solution 1: Train–Test Split

To solve this problem, we divide the dataset into two parts:

Training data → used to train the model

Testing data → used to evaluate the model on unseen data

```
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.3, random_state=42)
```

If test_size=0.3

- 70% of data is used for training
- 30% of data is used for testing

Example:

- Total samples = 100
- Training samples = 70
- Testing samples = 30

If test_size=0.2

- 80% → Training data
- 20% → Testing data

Example:

- Total samples = 100
- Training samples = 80
- Testing samples = 20

Limitations of Train–Test Split

- **Depends on one random split**

  Performance may reduce.

- **Unequal data distribution possible**
- **Not all data is tested**

  Some samples are never evaluated

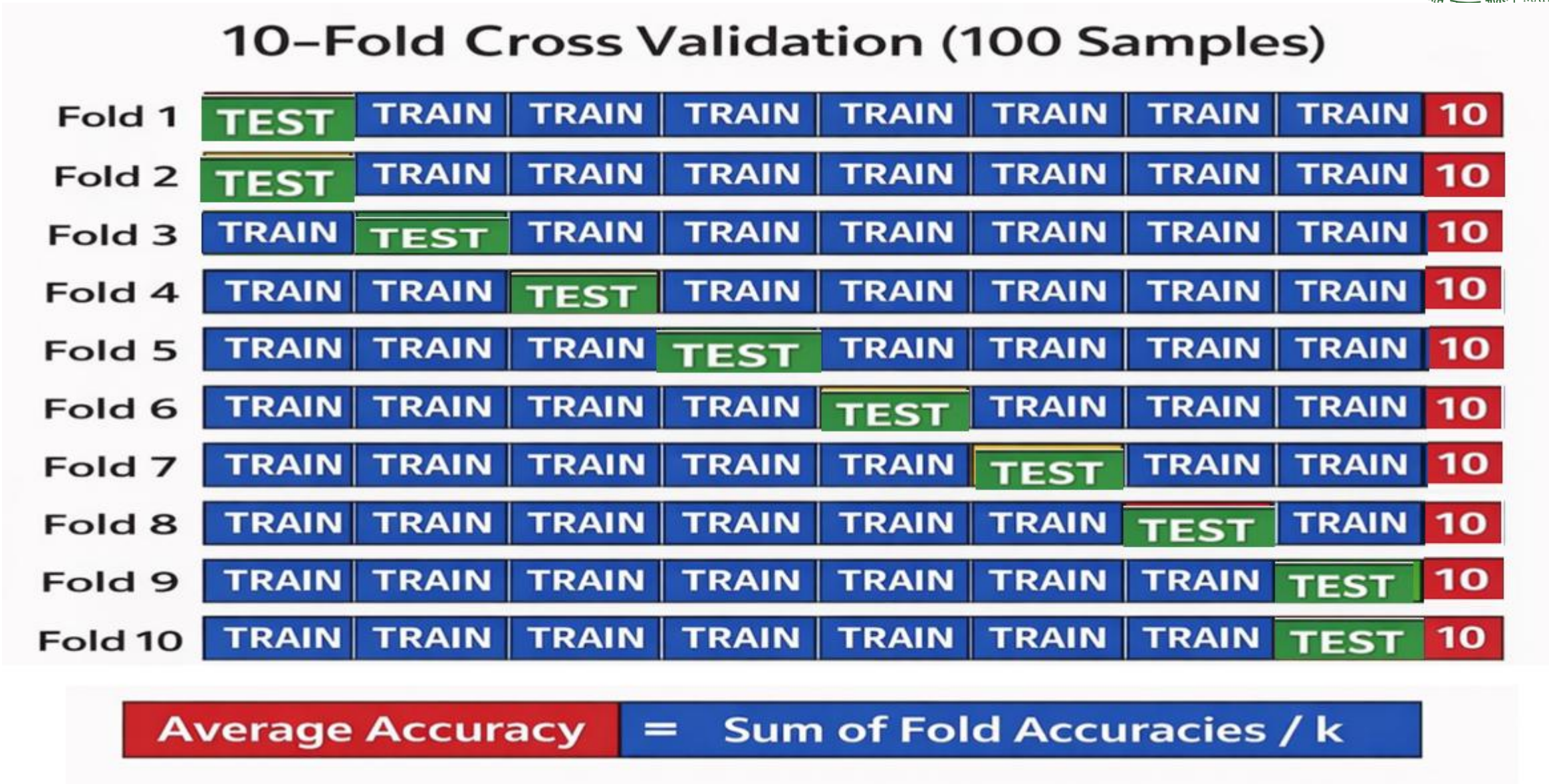
- **Unstable accuracy**

  Changing random_state can change accuracy

  Therefore, the result may be biased and unreliable.

Solution 2: Cross-Validation

K-Fold Cross-Validation



Average Accuracy = Sum of Fold Accuracies / k

- Cross validation is a technique used to evaluate a model's performance by training and testing it on different subsets of the same dataset.

Why cross validation is needed

- Single train–test split may give biased results
- Helps detect overfitting and underfitting
- Gives more reliable and stable accuracy

How it perform

- Divide data into K equal folds
- Model is trained and tested K times
- Each fold is used once as test data
- Repeat the process
- Final accuracy = average of all folds

```
from sklearn.model_selection import KFold
kf = KFold(n_splits=3)
for train_index, test_index in kf.split([1,2,3,4,5,6,7,8,9]):
    print(train_index, test_index)
```

```
[3 4 5 6 7 8] [0 1 2]
[0 1 2 6 7 8] [3 4 5]
[0 1 2 3 4 5] [6 7 8]
```

Why K-Fold is Better

- **Every sample is tested**

  No data is wasted

- **Balanced evaluation**

  Each fold gets a chance to be test data

- **Reduced bias**

  Performance does not depend on a single split

- **More reliable accuracy**

  Average of K results gives stable performance

- **Best for small datasets**

  Uses data efficiently

K-Fold Cross-Validation is better than train–test split because it evaluates the model on multiple data splits, reducing bias and providing more reliable accuracy.

Stratified K-Fold Cross-Validation

Stratified K-Fold ensures that each fold has the same target class distribution as the original dataset, giving reliable classification results.

Why this is important

- In classification problems, datasets are often imbalanced.
- If folds are created randomly, some folds may contain very few samples of a class, leading to biased accuracy.

For Example

Diabetes Original dataset (100 samples):

- Class 0 (Non-Diabetic) = **80**
- Class 1 (Diabetic) = **20**

5-Fold Stratified Cross-Validation:

Each fold will contain:

- Class 0 = **16**
- Class 1 = **4**

- So Same class ratio in every fold, due to this fair training and fair testing

What happens in each fold

- In Stratified K-fold in every fold training and testing on balanced dataset

What is the train–test ratio in K-Fold Cross-Validation?

Solution:

$$\text{Train} : \text{Test} = (K-1) : 1$$

If K = 10

Fold size = 100 ÷ 10 = 10

- Training samples = 100-10=90
- Testing samples = 10

$$\text{Train} : \text{Test} = 90 : 10 = 9 : 1$$

A dataset contains 240 samples. You apply 8-Fold Cross-Validation.

- Number of samples in each fold
- Training samples per iteration
- Testing samples per iteration
- Total number of models trained

Solution

Number of fold=8

Size of fold= 240 ÷ 8 = 30

Training = 240 − 30 = 210

Testing = 30

No. of times Models trained = 8

A dataset has 120 samples. You use 6-Fold Cross-Validation.

1. Samples per fold?
2. Training samples per iteration?
3. Testing samples per iteration?
4. Total models trained?

Solution:

Number of fold:6

- Fold size = 120 ÷ 6 = 20
- Training = 120 − 20 = 100
- Testing = 20
- Models = 6

Section – A

Attempt All Questions 3 X 5 = 15 Marks

No.	Detail of Question	Marks	CO	BL	KL
1	Explain the advantages of using k-fold cross-validation over a traditional train-validation-test split in this scenario. How does cross-validation help in model evaluation and selection?	3	2	A	C

No.	Detail of Question	Marks	CO	BL	KL
1	Building a churn prediction model with a highly imbalanced dataset. The team suggests using k-fold cross-validation for model evaluation. How would you set up cross-validation to handle class imbalance, and why might you prefer stratified k-fold cross-validation in this scenario to improve performance?	4	1	U	F

2	Discuss the role of cross-validation in improving model generalization by preventing overfitting and guiding optimal model selection.	4	3	An	C

Bayesian Method

- The Bayesian method is a probability-based reasoning method.
- The Bayesian method is based on Bayes' Theorem, which describes the probability of an event based on prior knowledge of conditions related to the event.
- It is used to solve the classification problem
- There is two types of Event
    - Independent Event
    - Dependent Event
- It is a way to calculate the value of P(B|A) with the knowledge of P(A|B)
- It is a statistical principle for combining prior knowledge of class with new evidence gathered from data

Independent Event

- Two events are called independent events if the occurrence of one event does not affect the probability of the other event.

Example :Rolling a Dice {1,2,3,4,5,6}

$$P(1) = \frac{1}{6} \ , P(2) = \frac{1}{6}$$

Dependent Event (Most Important for Bayesian Method)

- Two events are called dependent events if the occurrence of one event affects the probability of the other event.
- Drawing Marbles from a Bag (Without Replacement)

Consider a bag containing:

- **3 Red marbles**
- **2 Green marbles**

Total marbles = 5

$$P(R) = \frac{3}{5}, \quad P(G) = \frac{2}{4}$$

Conditional probability

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} \quad = P(A \cap B)=P(B)*P(A|B)$$

$$P(B \mid A) = \frac{P(B \cap A)}{P(A)} \quad = P(B \cap A)=P(A)*P(B|A)$$

$$P(A \cap B)= P(B \cap A)$$

$$P(B)*P(A|B)= P(A)*P(B|A)$$

Bayes Theorem

$$P(A|B)=\frac{P(B|A)*P(A)}{P(B)}$$

Prior

Evidence

Posterior

Likelihood

P(A | B) Posterior Probability: Which we need to calculate, and it will be read as probability of hypothesis A ,when we have occurred an evidence B

P(B|A) Likelihood: (probability of observing evidence B, given that hypothesis is true)

- we consider that hypothesis is true, then we calculate the probability of evidence.
- This is the probability of observing the new data under the assumption that the prior and the model are true.

P(A) Prior : (Prior Probability of event before seeing new data)

- Probability of hypothesis before considering the evidence
- This is the initial judgment or belief about the probability of an event before new data is introduced.

P(B) Evidence : pure probability of evidence

A card is drawn at random from a standard deck of 52 playing cards. It is given that the card drawn is a face card. Find the probability that the card drawn is a King, using Bayes' Theorem.

Solution:

A = Event that the card drawn is a King

B = Event that the card drawn is a Face card

Total no. of card in a deck (n=52)

Total no. of face card:4(king)+4(Queens)+4(Jacks)=12

Let's find the probability of king

According to Bayes theorem : $P(\text{King}|\text{Face})=\frac{P(\text{Face}|\text{King})*P(\text{King})}{P(\text{Face})}$

There are 4 Kings in a deck of 52 cards $P(King) = \frac{4}{52}$ =1/13

Probability of face card given King : $P(Face \mid King) = 1$

Probability that the card is a face card $P(face) = \frac{12}{52}$ =3/13

$P(king \mid face) = \frac{1 * 1/13}{3/13}$ =1/3

$\boxed{P(\text{King} \mid \text{Face Card}) = \frac{1}{3}}$`

Example: Picnic Day



You are planning a picnic today, but the morning is cloudy

- Oh no! 50% of all rainy days start off cloudy!
- But cloudy mornings are common (about 40% of days start cloudy)
- And this is usually a dry month (only 3 of 30 days tend to be rainy, or 10%)

We have to find What is the probability that it will rain given that the morning is cloudy?

A=Rain = It rains during the day

B=Cloud = Morning is cloudy

P(Rain) = 10% = 0.1

P(Cloud | Rain) = 50% = 0.5

P(Cloud) = 40% = 0.4

$$P(\text{Rain} \mid Cloud) = \frac{P(Rain) \times P(\text{Cloud} \mid Rain)}{P(Cloud)}$$

$$P(\text{Rain} \mid Cloud) = \frac{0.1 \times 0.5}{0.4} = 0.05/0.4 = 0.125 = 12.5\%$$

$$\boxed{P(\text{Rain} \mid \text{Cloud}) = 12.5\%}$$

Example:

- dangerous fires are rare (1%)
- but smoke is fairly common (10%) due to barbecues,
- and 90% of dangerous fires make smoke



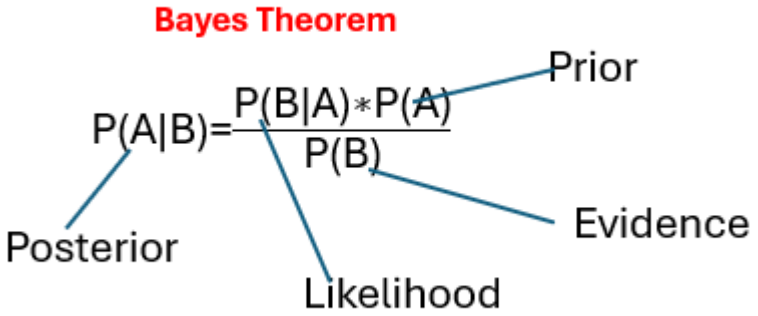
Home Bonfire

We can then discover the **probability of dangerous Fire when there is Smoke**:

$$P(\text{Fire}|\text{Smoke}) = \frac{P(\text{Fire})\ P(\text{Smoke}|\text{Fire})}{P(\text{Smoke})}$$

$$= \frac{1\% \times 90\%}{10\%}$$

Q3. Imagine you receive 100 emails, and 20 of them are spam. The word "free" appears in 10 of the spam emails and in 4 of the non-spam emails. If an email contains the word "free," what is the probability that it is spam? Use Bayes' Theorem with the following: solve



Solution:
Total emails = 100
Spam emails = 20
Non-spam emails = 80
Word “free” appears =10 spam emails
Number of free in non-spam emails=4
events
email is spam
email contains the word free

Prior probability of spam

$$P(spam) = \frac{20}{100} = 0.2$$

Probability of free given spam

$$P(free \mid spam) = \frac{10}{20} = 0.5$$

Probability of free given non-spam

$$P\left(\text{free} \mid \text{non}\overline{spam}\right) = \frac{4}{80} = 0.05$$

$$P(\text{nonspam}) = \frac{80}{100} = 0.8$$

Total probability of free

$P(free)$ =0.5*0.2+.05*0.8=0.14

$$P(\text{spam} \mid free) = \frac{P(\text{free} \mid spam) \times P(spam)}{P(free)}$$

$$= \frac{0.5*0.2}{0.14} \approx 0.714$$

P(spam|free) ≈ 0.714 0r 71.4%

Imagine there are 1000 people in a town, and 10 of them have a particular disease. There is a test for the disease that correctly identifies the disease in 9 out of 10 people who have it. However, it also incorrectly identifies the disease in 1 out of 100 people who do not have it. If a person tests positive, what is the probability that they actually have the disease?

Imagine there are 1000 people in a town, and 10 of them have a particular disease. There is a test for the disease that correctly identifies the disease in 9 out of 10 people who have it. However, it also incorrectly identifies the disease in 1 out of 100 people who do not have it. If a person tests positive, what is the probability that they actually have the disease?

- Total people = 1000
- People with disease = 10
- People without disease = 990

1. Person has the disease
2. Test result is positive

P(disease)=10/1000=0.01
P(nondisease)=990/1000=0.99
P(positive|disease)=9/10=0.9
P(positive|nondisease)=10/990 =0.0101

$$P(\text{disease} \mid positive) = \frac{P(\text{positive}|disease)\times P(disease))}{P(positive)}$$

P(positive)= P(positive|disease)* P(disease)+P(positive|nondisease)* P(nondisease)
P(positive)=0.9*.01+.0101*0.99=.019
P(disease|positive)=0.9*.01/0.019=0.474

Out of 2000 people in a city, 10 have a certain disease. A test is available that correctly identifies 8 out of 10 people who have the disease and incorrectly identifies 1 out of 50 people who do not have the disease. If someone tests positive, what is the probability that they actually have the disease?

You have two coins. One is a fair coin, and the other lands heads 3 out of 4 times. You pick one coin at random and flip it, getting heads. What is the probability that you picked the biased coin?

Out of 2000 people in a city, 10 have a certain disease. A test is available that correctly identifies 8 out of 10 people who have the disease and incorrectly identifies 1 out of 50 people who do not have the disease. If someone tests positive, what is the probability that they actually have the disease?

Hint: Use Bayes' Theorem with the following:

- Number of people with the disease: 10
- Number of people without the disease: 1990
- Number of true positives: 8
- Number of false positives: 40

- **Solution:**

1. **Prior probability of having the disease:**

$$P(\text{Disease}) = \frac{10}{2000} = 0.005$$

2. **Probability of testing positive given the disease:**

$$P(\text{Positive}|\text{Disease}) = \frac{8}{10} = 0.8$$

3. **Probability of testing positive given no disease:**

$$P(\text{Positive}|\text{No Disease}) = \frac{40}{1990} \approx 0.0201$$

4. **Posterior probability:**

$$P(\text{Positive}) = 0.8 \cdot 0.005 + 0.0201 \cdot 0.995 \approx 0.004 + 0.0200 = 0.024$$

Now, calculate:

$$P(\text{Disease}|\text{Positive}) = \frac{0.8 \cdot 0.005}{0.024} \approx 0.167$$

Answer: The probability that a person actually has the disease if they test positive is approximately 0.167, or 16.7%.

- **Q4. You have two coins. One is a fair coin, and the other lands heads 3 out of 4 times. You pick**
- **one coin at random and flip it, getting heads. What is the probability that you picked the**
- **biased coin?**
- Hint: Use Bayes' Theorem with the following:
- Number of fair coins: 1
- Number of biased coins: 1
- Number of heads with the fair coin: 1 out of 2
- Number of heads with the biased coin: 3 out of 4
- Number of heads with the biased coin: 3 out of 4

- Solution:

1. Prior probability of picking either coin:

$$P(\text{Fair Coin}) = P(\text{Biased Coin}) = \frac{1}{2} = 0.5$$

2. Probability of getting heads with the fair coin:

$$P(\text{Heads}|\text{Fair Coin}) = \frac{1}{2} = 0.5$$

3. Probability of getting heads with the biased coin:

$$P(\text{Heads}|\text{Biased Coin}) = \frac{3}{4} = 0.75$$

4. Posterior probability:

$$P(\text{Heads}) = 0.5 \cdot 0.5 + 0.75 \cdot 0.5 = 0.25 + 0.375 = 0.625$$

Now, calculate:

$$P(\text{Biased Coin}|\text{Heads}) = \frac{0.75 \cdot 0.5}{0.625} = 0.6$$

Answer: The probability that you picked the biased coin after flipping heads is 0.6, or 60%.

Applications

- Credit card fraud detection
- Spam Filtering
- Medical Diagnosis
- Patterns in customer dataset/marketing campaign performance
- Help robots make decisions
- Reconstructing clean images from noisy images
- Weather Prediction

Data Preprocessing

- Large datasets often contain various types of data, including structured tables, images, audio files, and videos.
- However, machine learning algorithms cannot directly process raw text, images, or videos, as they only understand numerical representations (1s and 0s).
- Therefore, it is essential to transform or encode the dataset into a suitable format before applying machine learning techniques.
- By converting the data into meaningful numerical features, the algorithm can effectively interpret and learn patterns, enabling accurate predictions and analysis.

Imputation

- Data Imputation is the process of handling missing values or null values in a dataset for the purpose of enhancing efficiency and accuracy in the model training process
- Imputation replace suitable substitute values so that machine learning algorithms can work properly.
- Data Imputation plays a crucial role in Data pre-processing for efficient model training and results.

The main goals of Data Imputation are listed below:

- Maintaining Dataset completeness
- Enhancing model performance, efficiency, and accuracy
- Data imputation helps resolve data inconsistency
- Helps in reducing data loss and bias, i.e., helps retain valuable data patterns
- Ensures model compatibility for performing predictions

Glucose	BloodPres	SkinThicki	Insulin	BMI	DiabetesP	Age	Diabetes
148	72	35	0	33.6	0.627	50	1
85	66	29	0	26.6	0.351	31	0
183	64	0	0	23.3	0.672	32	1
89	66	23	94	28.1	0.167	21	0
137	40	35	168	43.1	2.288	33	1
116	74	0	0	25.6	0.201	30	0
78	50	32	88	31	0.248	26	1
115	0	0	0	35.3	0.134	29	0
197	70	45	543	30.5	0.158	53	1
125	96	0	0	0	0.232	54	1
110	92	0	0	37.6	0.191	30	0
168	74	0	0	38	0.537	34	1
139	80	0	0	27.1	1.441	57	0
189	60	23	846	30.1	0.398	59	1
166	72	19	175	25.8	0.587	51	1
100	0	0	0	30	0.484	32	1
118	84	47	230	45.8	0.551	31	1
107	74	0	0	29.6	0.254	31	1
103	30	38	83	43.3	0.183	33	0
115	70	30	96	34.6	0.529	32	1
126	88	41	235	39.3	0.704	27	0
99	84	0	0	35.4	0.388	50	0
196	90	0	0	39.8	0.451	41	1
119	80	35	0	29	0.263	29	1
143	94	33	146	36.6	0.254	51	1
125	70	26	115	31.1	0.205	41	1
147	76	0	0	39.4	0.257	43	1
97	66	15	140	23.2	0.487	22	0
145	82	19	110	22.2	0.245	57	0
117	92	0	0	34.1	0.337	38	0
109	75	26	0	36	0.546	60	0
158	76	36	245	31.6	0.851	28	1
88	58	11	54	24.8	0.267	22	0
92	92	0	0	19.9	0.188	28	0

GLA UNIVERSITY MATHURA

Missing Value

- Remove Missing value
- Imputation
  - Univariate Imputation
  - Multivariate Imputation

Student	Marks (Maths)	Height (cm)	Gender
S1	78	165	Male
S2	85	170	Female
S3	NaN	168	Male
S4	90	NaN	Female
S5	88	172	NaN

- Univariate imputation replaces missing values using only the same variable (single feature).
- Common method is Mean and median use for numerical value.
- Mode also use for categorical data

Student	Marks (Maths)	Height (cm)	Gender
S1	78	165	Male
S2	85	170	Female
S3	NaN	168	Male
S4	90	NaN	Female
S5	88	172	NaN

- Multivariate imputation replaces missing values using multiple related variables.
- KNN imputation
- Iterative imputation

Complete Case Analysis (CCA)

- Complete Case Analysis (CCA) is a method of handling missing data in which only those records (rows) that have no missing values are used for analysis.
- All rows containing at least one missing value are discarded.

Patient_ID	Age	BP	Gender	Diabetes
P001	45	120	M	No
P002	52	NaN	F	Yes
P003	NaN	135	M	No
P004	60	150	F	Yes
P005	48	NaN	M	No

Advantages

- Simple to implement
- No need to impute values
- Preserves original data without introducing artificial values

Disadvantages

- It can exclude a large fraction of the original dataset
- Excluded observation could be informative for the analysis
- Not suitable when missing data is high
- When using our models ,the model will not know how to handle missing data

- If the dataset have missing value can be remove
- But percentage of missing data should be<5%
- Data is Missing Completely At Random (MCAR)
- If dataset is too large ,so removing some value does not effect

Advantage and Disadvantage of Univariate

- Simple Method

Disadvantage
Outlier-sensitive

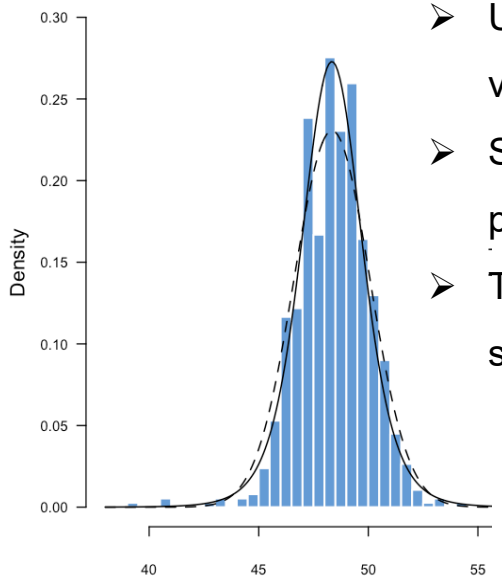
Marks
40
45
50
500
NaN

500 is an outlier
Mean = (40 + 45 + 50 + 500) / 4 = 158.75
Missing value=158.75
Which is not realistic

Reduce correlations

Correlation Between	Correlation Before Imputation	Correlation After Imputation
Age and Fitness Score	-0.9817252	-0.9035089

Change in data shape



- Univariate imputation fills missing values with one constant.
- So many values pile up at the same point.
- This change the original distribution shape and reduces variance.

- Correlation shows how two variables change together.
- Univariate imputation fills missing values using one variable only (mean/median).
- This inserts constant values.
- Constants do not vary, so they reduce correlation with other variables.

Advantage and disadvantage of Multivariate Imputation

Advantages

- **Preserves relationships:** It uses many features together, so their connection is not lost.
- **More realistic values:** Missing values are filled using similar cases, not just averages.
- **Better model performance:** Good data helps the model predict better.
- **Best for medical/research data:** Medical values are related (e.g., glucose depends on BMI).

Disadvantage

- **Computationally expensive:** Needs more time and resources.
- **Complex for beginners:** Harder to understand and tune.
- **Risk of overfitting:** Models used for imputation may fit noise if not careful.

Common Imputation Techniques

Mean Imputation

- The mean is the average value of a dataset, obtained by adding all observations and dividing by the number of observations.

$$\text{Mean} = \frac{\text{Sum of all values}}{\text{Number of values}}$$

Data: 10, 20, 30, 40

$$\text{Mean} = \frac{10 + 20 + 30 + 40}{4} = 25$$

With an Outlier

Data: 10, 20, 30, 200

$$\text{Mean} = \frac{10 + 20 + 30 + 200}{4} = 65$$

Advantages

- Easy to calculate and understand
- Uses all values in the dataset
- Suitable for normally distributed data

Disadvantages

- Highly sensitive to outliers
- Not suitable for skewed data
- May give unrealistic results

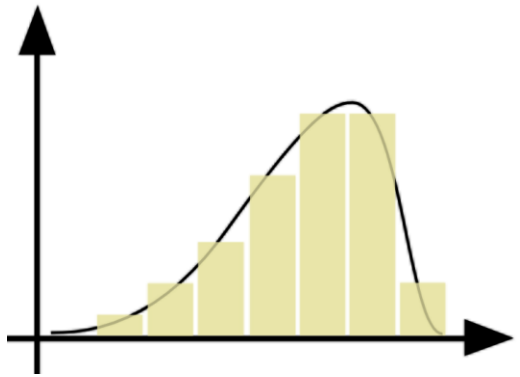
Median Imputation

- The median is the middle value of a dataset when arranged in ascending or descending order.

$$\text{Median} = \frac{\text{Middle two values}}{2}$$

Data: 10, 20, 30, 40

Median = (20 + 30) / 2 = 25



Advantages

- Not affected by outliers
- Good for skewed data

Disadvantages

- Does not use all values
- Harder for large data

Mode Imputation

- Mode is the value that occurs most frequently.

$$\text{Mode} = \text{Most frequent value}$$

For example:

Data: 2, 4, 4, 6
Mode = 4

Advantages

- Best for categorical data
- Easy to identify

Disadvantages

- May be no mode or many modes
- Not always representative

How to Choose the Correct Imputation Method

Check how many values are missing
If <5% use Complete Case Analysis (CCA)

↓

If Many missing values, then use imputation method

↓

Check if Numerical data(Age, BMI, Insulin, BP)
Use Mean or Median
If Categorical data(Gender)
Use Mode

```
# Percentage of missing values in each column
missing_percent = (X_train.isnull().sum() / len(X_train)) * 100


print(missing_percent)
```

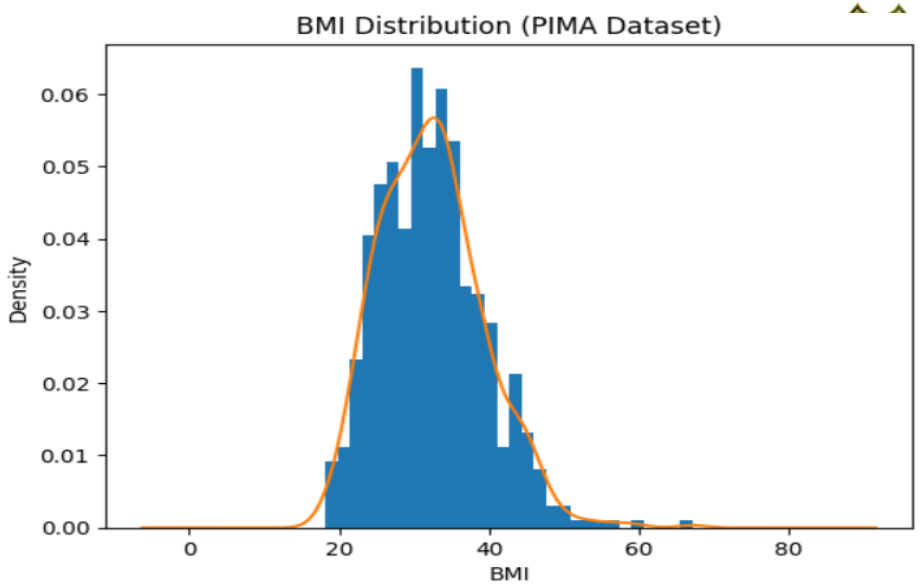
```
Pregnancies              0.000000
Glucose                  0.814332
BloodPressure            3.908795
SkinThickness           28.664495
Insulin                 47.231270
BMI                      1.140065
```

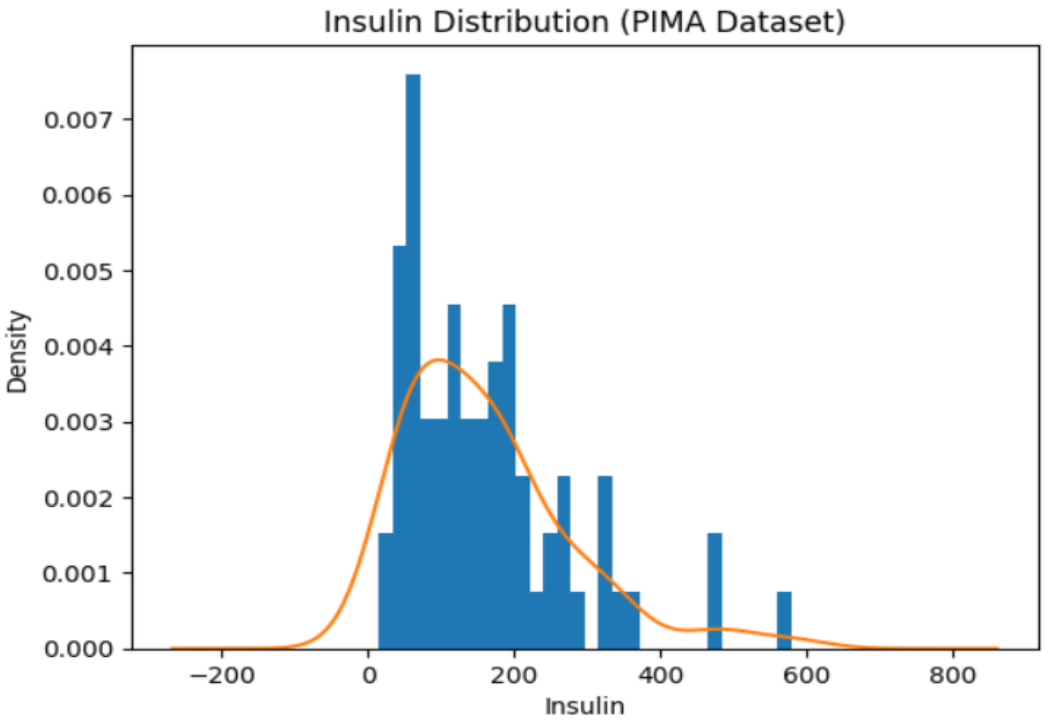
Check the distribution of data
If it is normal distribution means bell-shaped curve (Gaussian distribution)
Use Mean Imputation method

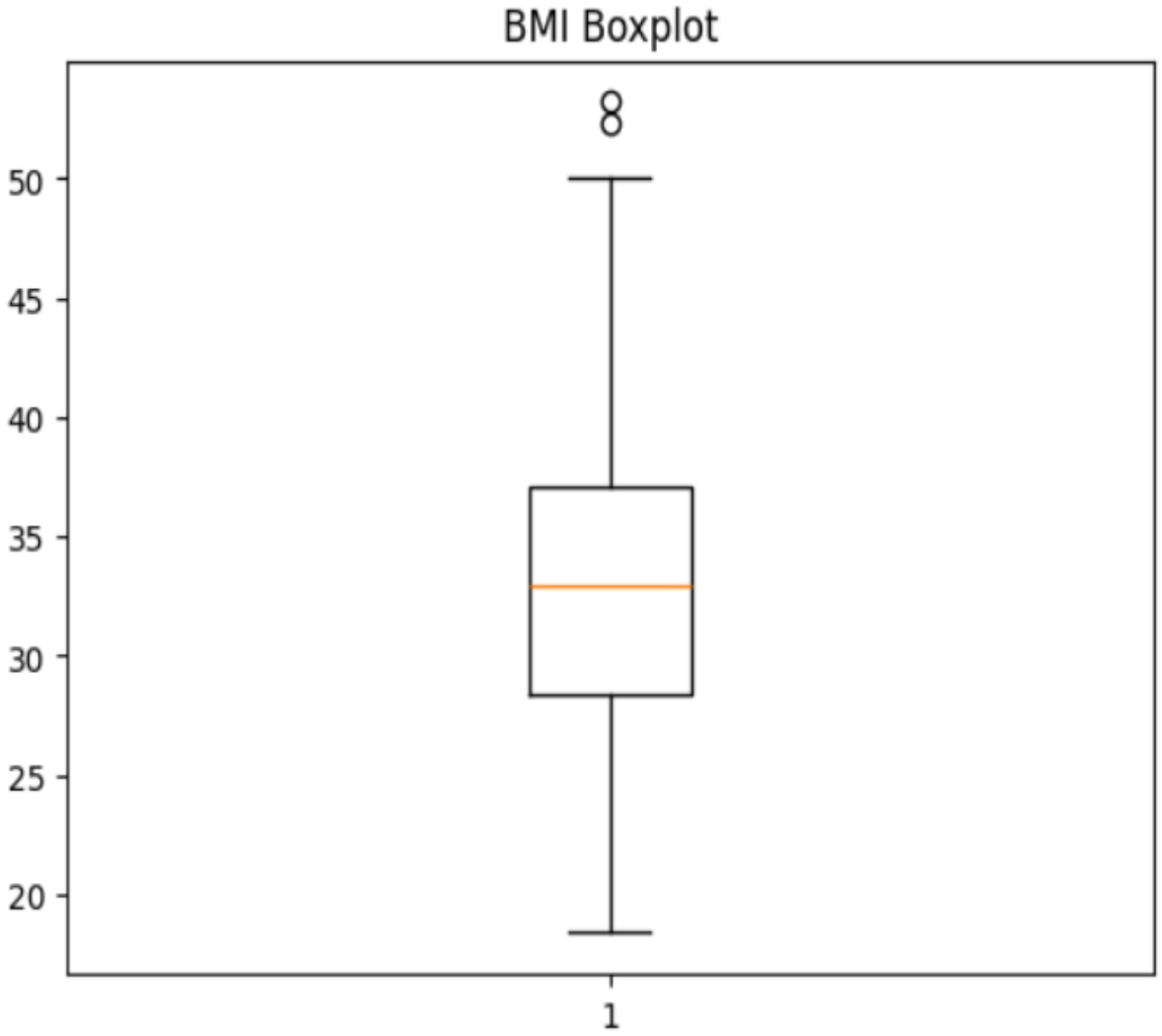
Check if Skewed distribution (tail on one side)
Use Median

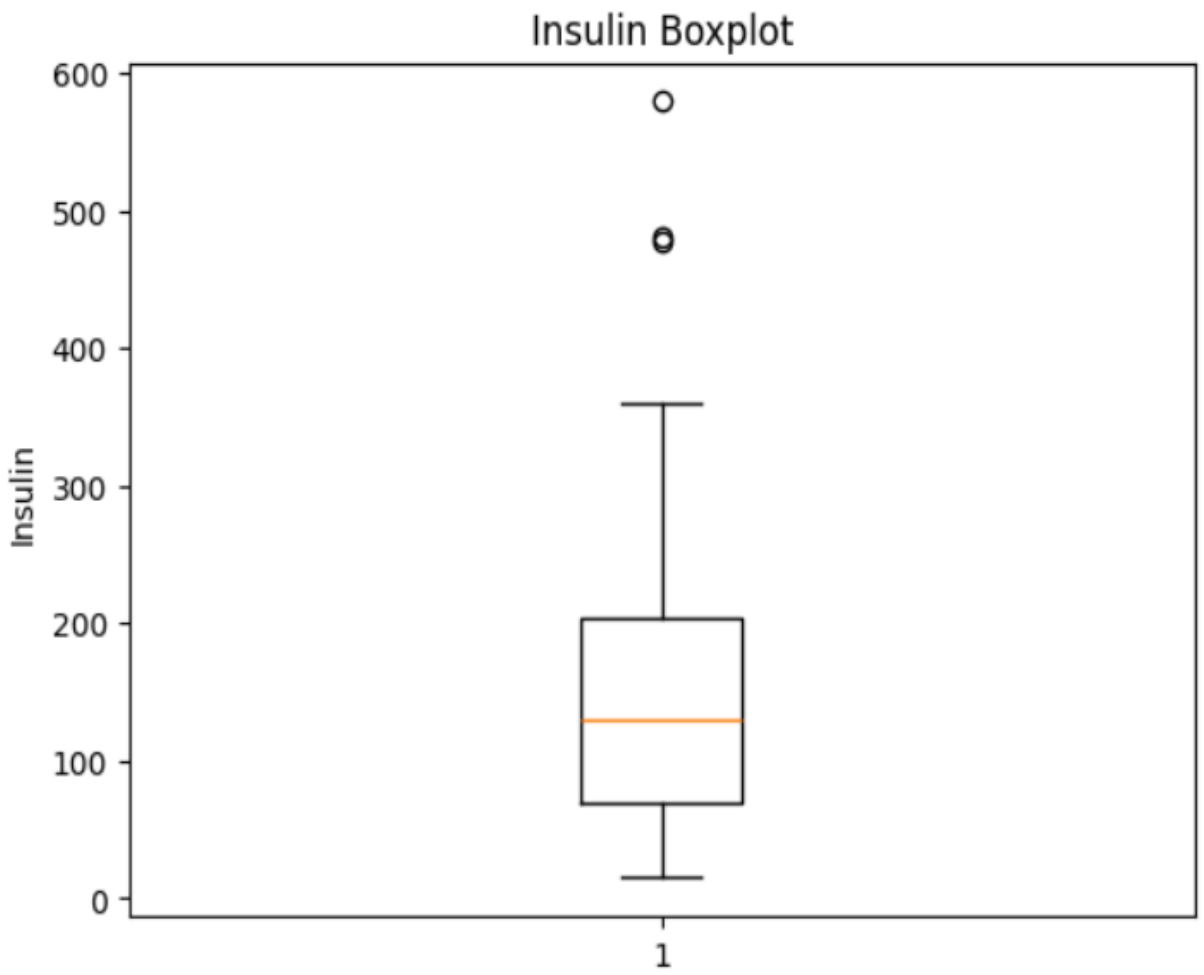
Check If outliers then Median
Else Mean

Median imputation is preferred for healthcare datasets because medical data is usually skewed and contains outliers, while mean imputation is suitable only for normally distributed data.






BMI Boxplot
50
45
40
35
30
25
20
1


Insulin Boxplot
Insulin
600
500
400
300
200
100
0
1

You are working with a healthcare dataset that contains patient information, but several rows have missing values for the Age and Blood Pressure columns.

What are some strategies for handling missing values in these columns, and how would you decide whether to use mean imputation, median imputation, or deletion of missing data points?

2	What do you mean by data imputation? Explain any two data imputation methods with example.	4	1	U	F

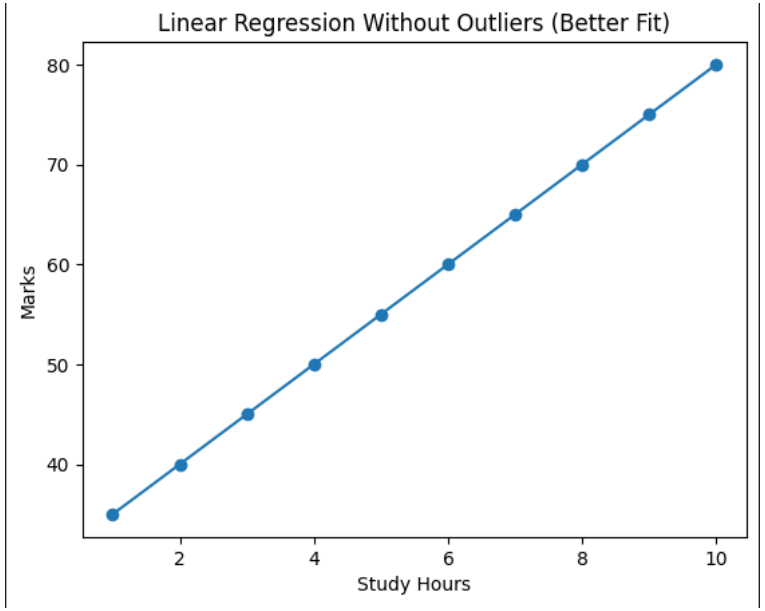
2	In a healthcare dataset, you observe that many rows are incomplete because the Age and Blood Pressure attributes are missing. What preprocessing methods can be used to handle these missing values? Explain when it is more appropriate to replace values with the mean, use the median, or completely drop the records.	3	2	A	C

Outliers

- An outlier is a data value that is very different from most of the other values in a dataset.

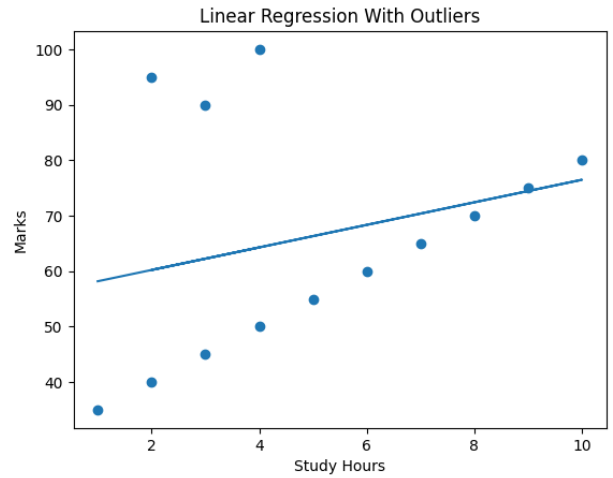
For example, in student marks and study hours dataset

- As study hours increase, marks increase.
- All points follow one straight trend.
- Linear regression can easily draw a best-fit straight line.



Study Hours (X)	Marks (Y)
1	35
2	40
3	50
4	60
5	65
6	75

Study Hours	Marks
1	35
2	40
2	**95** ← Outlier
3	50
4	60
5	65
6	75

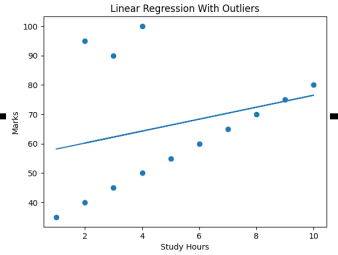


How Outliers Occur

- **Data Entry Error**
  1. Wrong typing while entering data
  2. Example: Marks entered as 95 instead of 45
- **Measurement / Instrument Error**
  1. Faulty sensors or devices
  2. Example: Wrong readings from machines
- **Rare but Valid Cases**
  1. Exceptional or unusual behavior
  2. Example: A student studies only 2 hours but scores 95 marks
- **Mixing Data from Different Sources**
  1. Data collected from different groups
  2. Example: Average students + toppers together
- **Natural Extreme Events**
  1. Rare events that occur naturally
  2. Example: Fraud transactions, cyber attacks

Impact of Outliers on Machine Learning Algorithms

Linear Regression
Outliers pull the regression line



K-Nearest Neighbors (KNN)

- Outliers increase distance
- Wrong neighbors are selected

K-Means Clustering
Outliers shift the cluster centroid

Support Vector Machine (SVM)
Outliers affect the margin

Algorithms Less Affected by Outliers

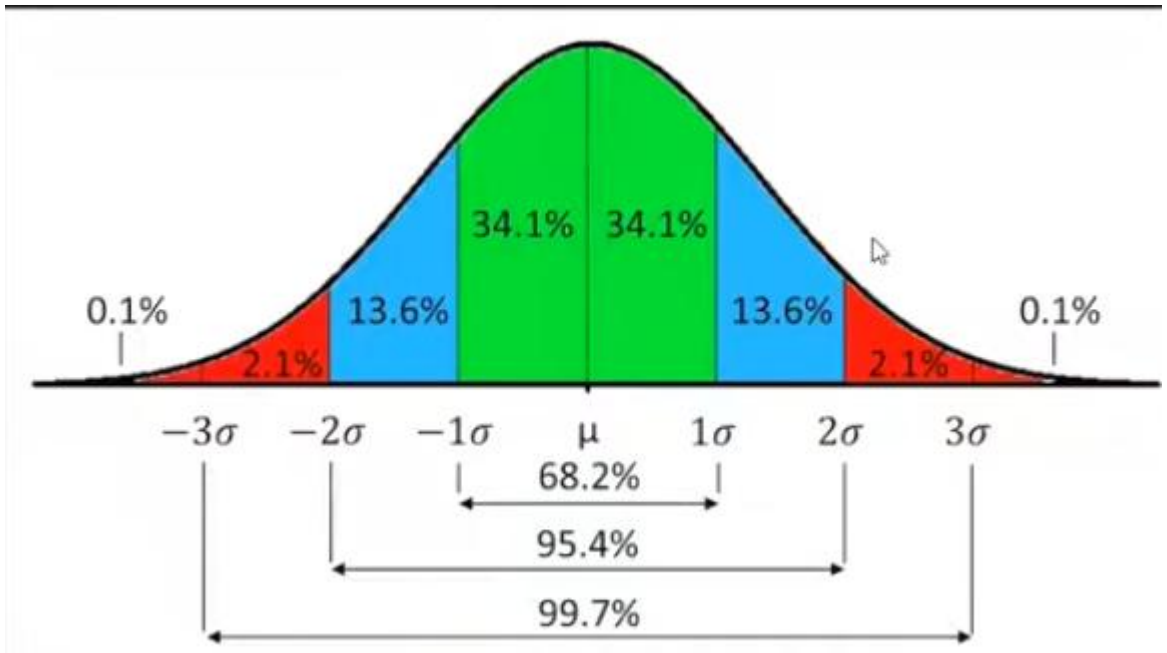
1. Decision Tree
2. Random Forest
3. Gradient Boosting
4. DBSCAN (detects outliers as noise)

Tree-based algorithms and density-based clustering methods are less affected by outliers because they do not rely on distance or mean calculations, and DBSCAN explicitly identifies outliers as noise.

Outlier Detection Technique

- Box Plot
- Z score
- Inter Quantile Range(IQR)

Z score



- μ is the mean (average) of the data
- It lies at the center of the bell curve
- Half the data lies on the left, half on the right

For example

If average marks of a class = 60, then μ = 60

•σ is the standard deviation

•It tells how much data is spread around the mean

this image is showing the Normal Distribution (Bell Curve) with μ (mean) and σ (standard deviation) and the 68–95–99.7 rule.

μ ± σ (Green Region – 68%)

$\mu - \sigma$ to $\mu + \sigma$

•Covers 68% of the data

μ ± 2σ (Green + Blue Region – 95%)

$\mu - 2\sigma$ to $\mu + 2\sigma$

•Covers 95% of the data

μ ± 3σ (Entire Curve – 99.7%)

$\mu - 3\sigma$ to $\mu + 3\sigma$

•Covers 99.7% of the data

•Only 0.3% values lie outside

Values outside this range are Outliers.

Steps to Calculate the Z-score:

1. **Calculate the Mean (μ):** The mean is the average value of all data points in the dataset.

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

where x_i is each data point and n is the number of data points.

2. **Calculate the Standard Deviation (σ):** The standard deviation is a measure of how spread out the data points are from the mean.

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$$

3. **Compute the Z-score for Each Data Point (Z):** The Z-score is calculated using the formula:

$$Z = \frac{x_i - \mu}{\sigma}$$

Here, x_i is the value of the data point, μ is the mean, and σ is the standard deviation.

Z-Score

$$Z = \frac{X - \mu}{\sigma}$$

Z| ≤ 1 → Normal
|Z| ≤ 2 → Acceptable
|Z| ≤ 3 → Rare
|Z| > 3 → **Outlier**

Suppose in a class:
•Mean marks (μ) = 60
•Standard deviation (σ**) =** 10
Now calculate Z-score for different students.

Student scored 70 marks

$$Z = \frac{70 - 60}{10} = \frac{10}{10} = 1$$

Z = 1
Normal no outlier

Student scored 95 marks

$$Z = \frac{95 - 60}{10} = \frac{35}{10} = 3.5$$

Z > 3 Outlier

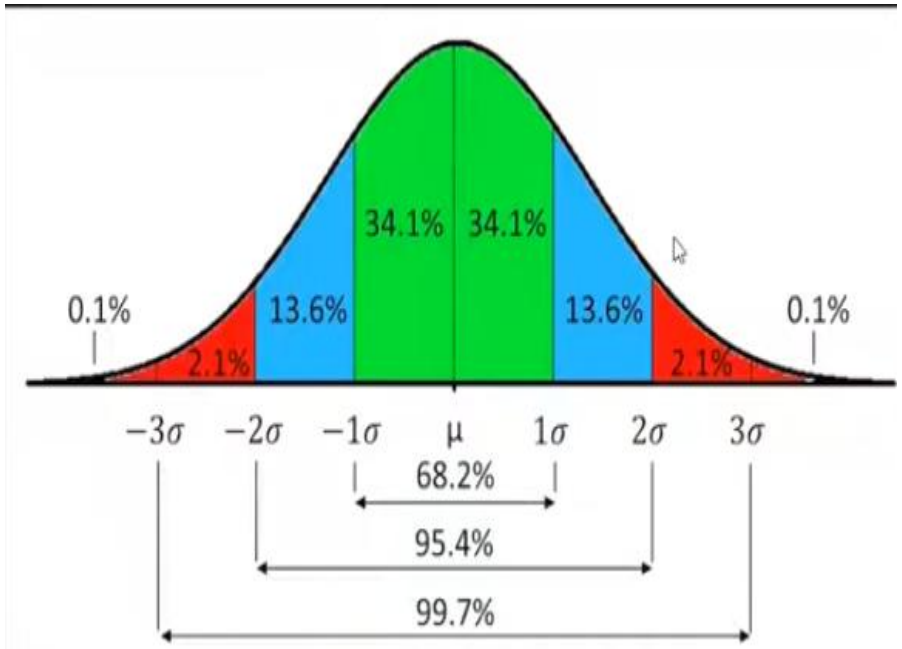
Steps to Handle Outliers Using Z-Score

1. First Check Whether Data Is Normally Distributed : Z-score method works only when data follows a normal (bell-shaped) distribution.

How to check normality : Histogram looks bell-shaped

2. If Data Is Normally Distributed : Use Z-Score $Z = \frac{X-\mu}{\sigma}$

If |Z| > 3, the value is an outlier



How to Handle the Outlier

1: Trimming (Remove Outlier): Completely delete the outlier value

Marks: 35, 40, 50, 60, 70, 95

Remove 95

2: Capping : Replace outlier with boundary value (μ ± 3σ)

Use capping when:

Value is rare but valid

You don't want to lose data

Upper limit = μ + 3σ = 90

95 → replaced by 90

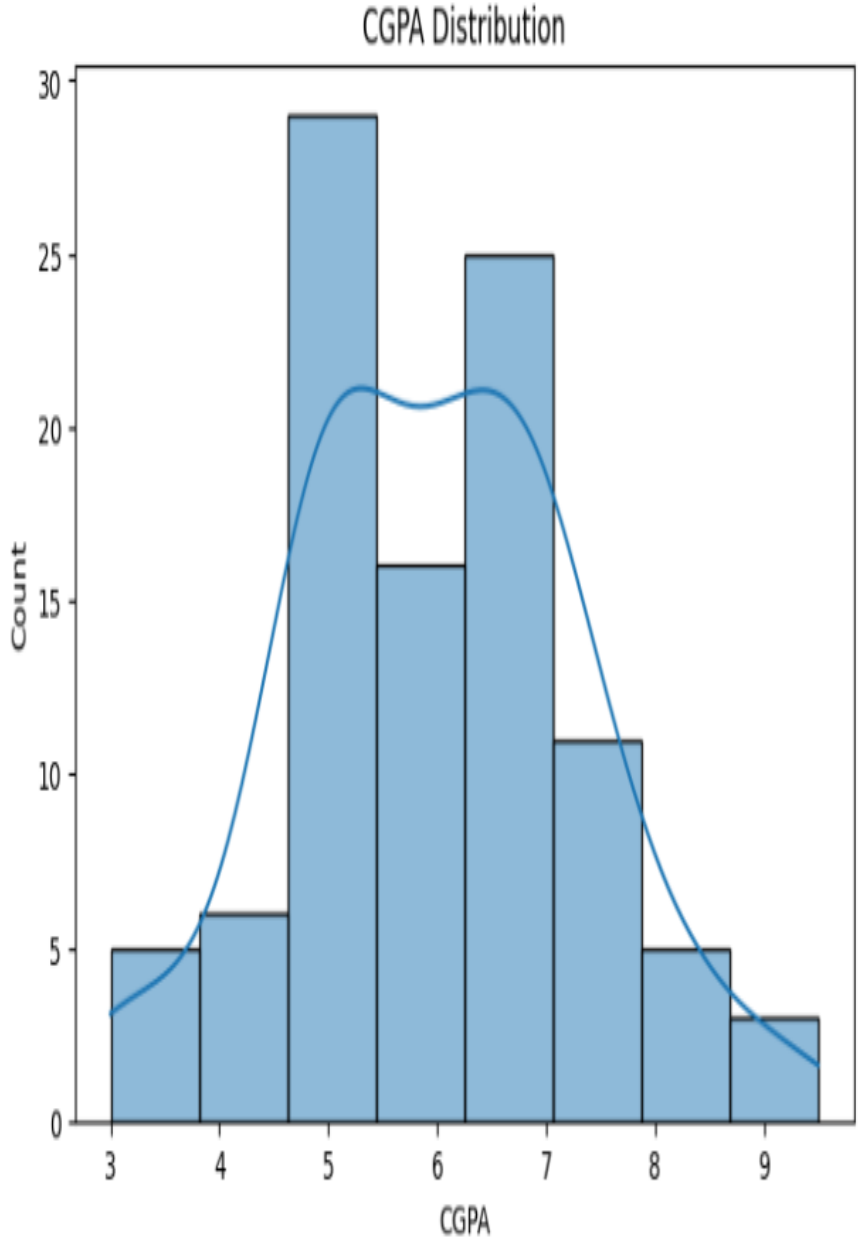
Use: **Trimming for wrong data**

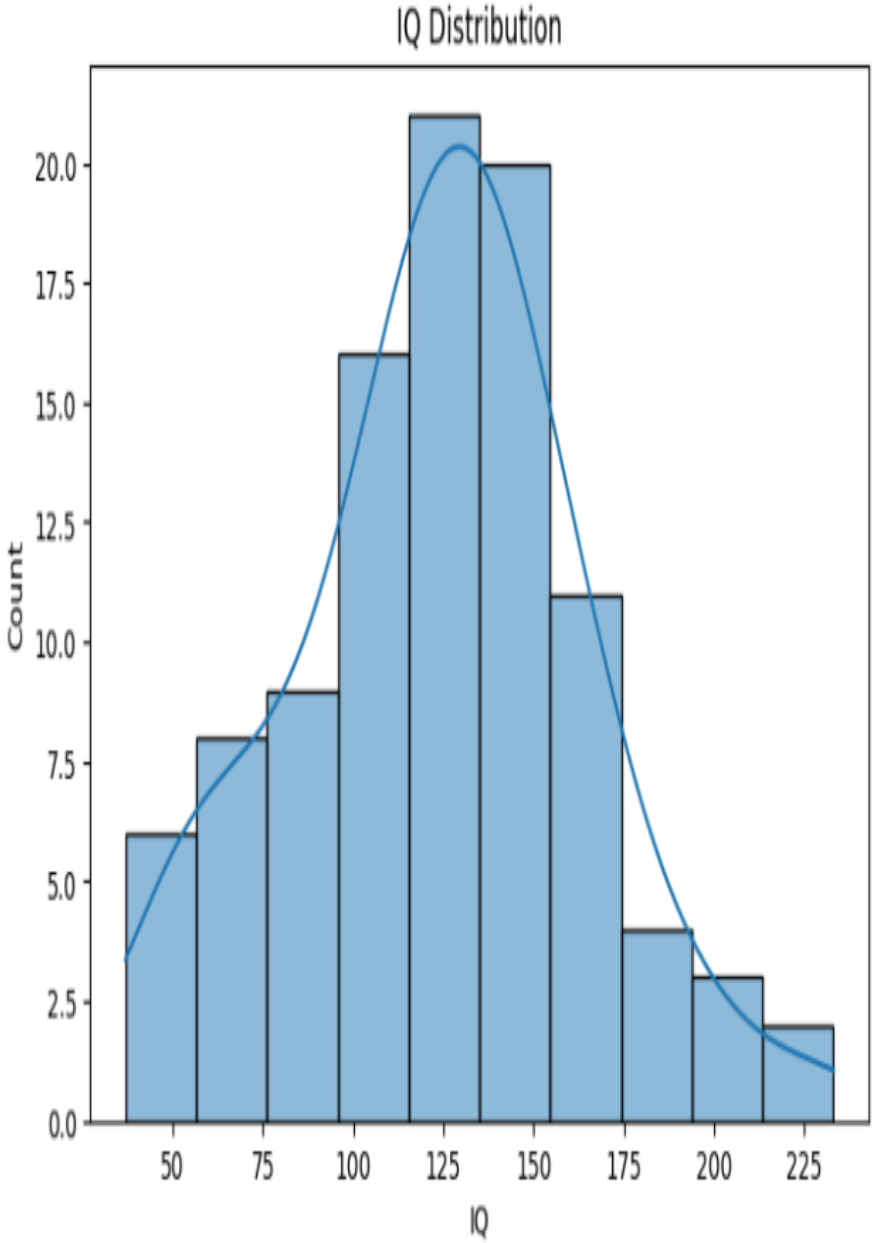
Capping for extreme but valid data

	Student_ID	CGPA	IQ	Placement
0	1	6.8	123	1
1	2	5.9	106	0
2	3	5.3	121	0
3	4	7.4	132	1
4	5	5.8	142	0

```
df.shape
```

```
(100, 4)
```





```
print("mean value of cgpa",df['CGPA'].mean())
print("mean value of cgpa",df['CGPA'].std())
print("mean value of cgpa",df['CGPA'].min())
print("mean value of cgpa",df['CGPA'].max())
```

```
mean value of cgpa 5.996
mean value of cgpa 1.3399773856074566
mean value of cgpa 3.0
mean value of cgpa 9.5
```

```
print("Highest allowed", df['CGPA'].mean()+3*df['CGPA'].std())
print("Lowest allowed", df['CGPA'].mean()-3*df['CGPA'].std())
```

```
Highest allowed 10.01593215682237
Lowest allowed 1.9760678431776304
```

```
#finding outliers
df[(df['CGPA']>10.05) |(df['CGPA']<1.97) ]
```

	Student_ID	CGPA	IQ	Placement	CGPA_ZScore
69	70	8.5	120	1	1.868688
77	78	9.0	50	1	2.241829
91	92	9.0	158	1	2.241829
99	100	9.5	113	1	2.614970

```
#trimming
new_df=df[(df['CGPA']<8.42) &(df['CGPA']>1.97) ]
new_df
```

	Student_ID	CGPA	IQ	Placement
0	1	6.8	123	1
1	2	5.9	106	0
2	3	5.3	121	0
3	4	7.4	132	1
4	5	5.8	142	0
...	...	...	...	...
94	95	4.7	52	0
95	96	4.3	200	0
96	97	4.4	42	0
97	98	6.7	182	1
98	99	6.3	103	1

96 rows × 4 columns

```
new_df.shape
```

```
(96, 5)
```

```python
import numpy as np

# Calculate limits using Mean and Standard Deviation
upper_limit = df['CGPA'].mean() + 3 * df['CGPA'].std()
lower_limit = df['CGPA'].mean() - 3 * df['CGPA'].std()

print("Upper Limit:", upper_limit)
print("Lower Limit:", lower_limit)

# Apply Capping
df['CGPA'] = np.where(
    df['CGPA'] > upper_limit,
    upper_limit,
    np.where(
        df['CGPA'] < lower_limit,
        lower_limit,
        df['CGPA']
    )
)
```

```
Upper Limit: 10.015921568223 7
Lower Limit: 1.9760678431776304
```

```
df.shape
```

```
(100, 5)
```

```
df['CGPA_ZScore']=(df['CGPA']-df['CGPA'].mean())/df['CGPA'].std()
```

```
df.head()
```

	Student_ID	CGPA	IQ	Placement	CGPA_ZScore
0	1	6.8	123	1	0.600010
1	2	5.9	106	0	-0.071643
2	3	5.3	121	0	-0.519412
3	4	7.4	132	1	1.047779
4	5	5.8	142	0	-0.146271

```
df[(df['CGPA_ZScore'] < -3) | (df['CGPA_ZScore'] > 3)]
```

Student_ID	CGPA	IQ	Placement	CGPA_ZScore

Boxplot and IQR

Box Plot

A box plot (also called a box-and-whisker plot) is a graphical tool used to: Show data distribution , Identify outliers

A box plot shows normal data and highlights outliers clearly

Q1 (First Quartile – 25%)
25% of data lies below Q1
Left edge of the box

Median (Q2 – 50%)
Middle value of the dataset
Shown as a line inside the box

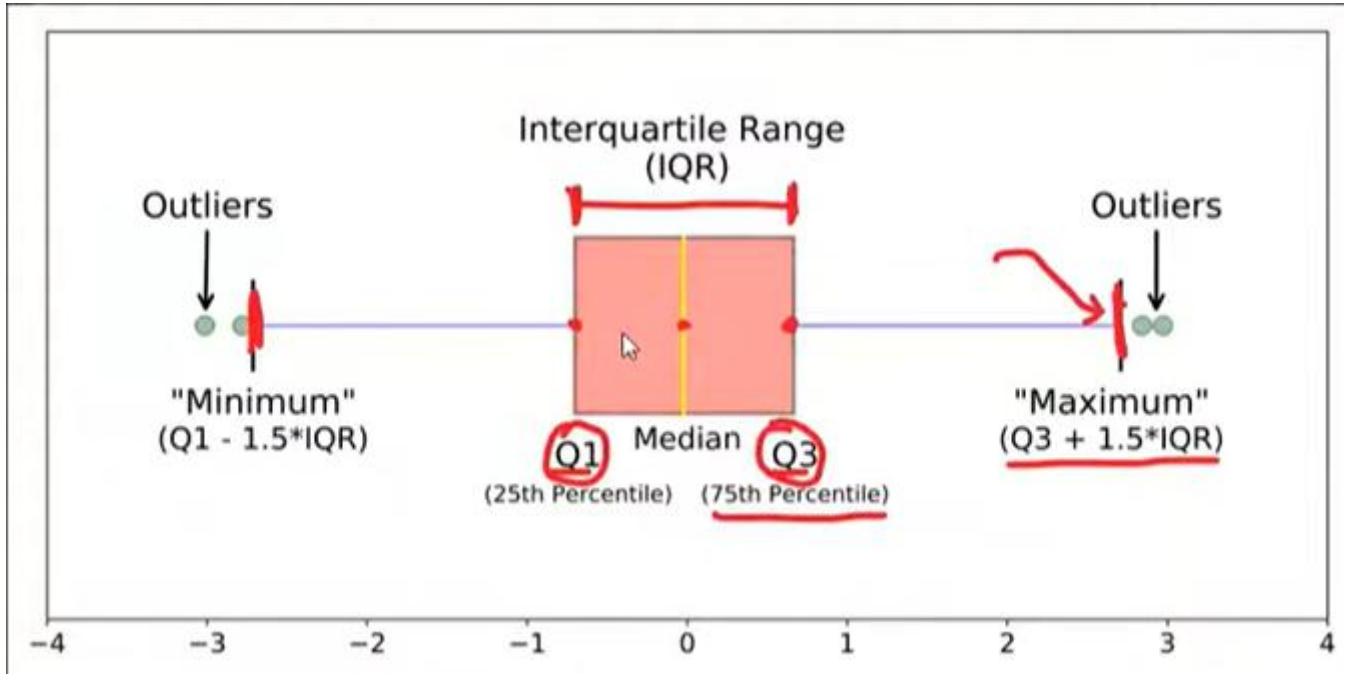
Q3 (Third Quartile – 75%)
75% of data lies below Q3

Whisker
•Largest and smallest value within the allowed range
$Q3 + 1.5 \times IQR$/Q1-1.5*IQR

Interquartile Range (IQR)

$$IQR = Q3 - Q1$$

•Represents the middle 50% of the data
•Length of the box = IQR



How Box Plot Detects Outliers

Outliers

•Points below

$$Q1 - 1.5 \times IQR$$

•Points above

$$Q3 + 1.5 \times IQR$$

These are shown as dots outside the whiskers

Example

Marks:

35, 40, 50, 60, 65, 75, 95

Q1 = 40

Q3 = 75

IQR = 35

Upper bound:$Q3 + 1.5 \times IQR$

$$75 + 1.5 \times 35 = 127.5$$

If marks = 150, it will appear as a dot outside the whisker → Outlier

Lower Bound=Q1−1.5×IQR

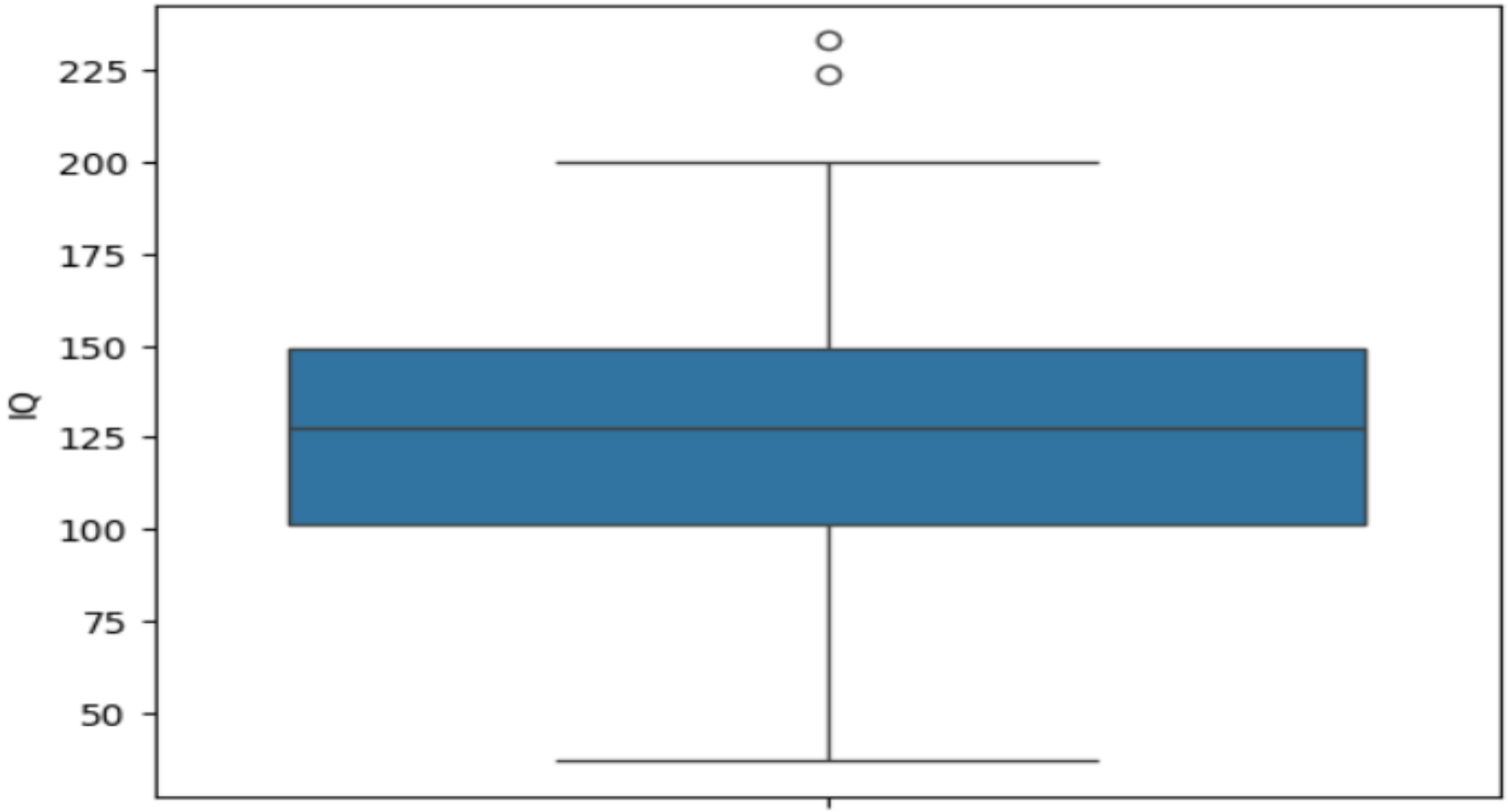
Substitute values:

$$40 - 1.5 \times 35$$

$40 - 52.5 = -12.5$ if marks <-12.5 is outlier

```
sns.boxplot(df['IQ'])
```

```
<Axes: ylabel='IQ'>
```




```
# Finding the IQR
percentile25 = df['IQ'].quantile(0.25)
percentile75 = df['IQ'].quantile(0.75)
```

```
percentile75
```

```
np.float64(149.0)
```

```
IQR=percentile75-percentile25
```

```
IQR
```

```
np.float64(47.5)
```

```
upper_limit = percentile75 + 1.5 * IQR
lower_limit = percentile25 - 1.5 * IQR
```

```
print("Upper limit", upper_limit)
print("Lower limit", lower_limit)
```

```
Upper limit 220.25
Lower limit 30.25
```

```
df[df['IQ'] > upper_limit]
```

	Student_ID	CGPA	IQ	Placement	CGPA_ZScore
16	17	5.2	224	0	-0.594040
50	51	3.5	233	0	-1.862718

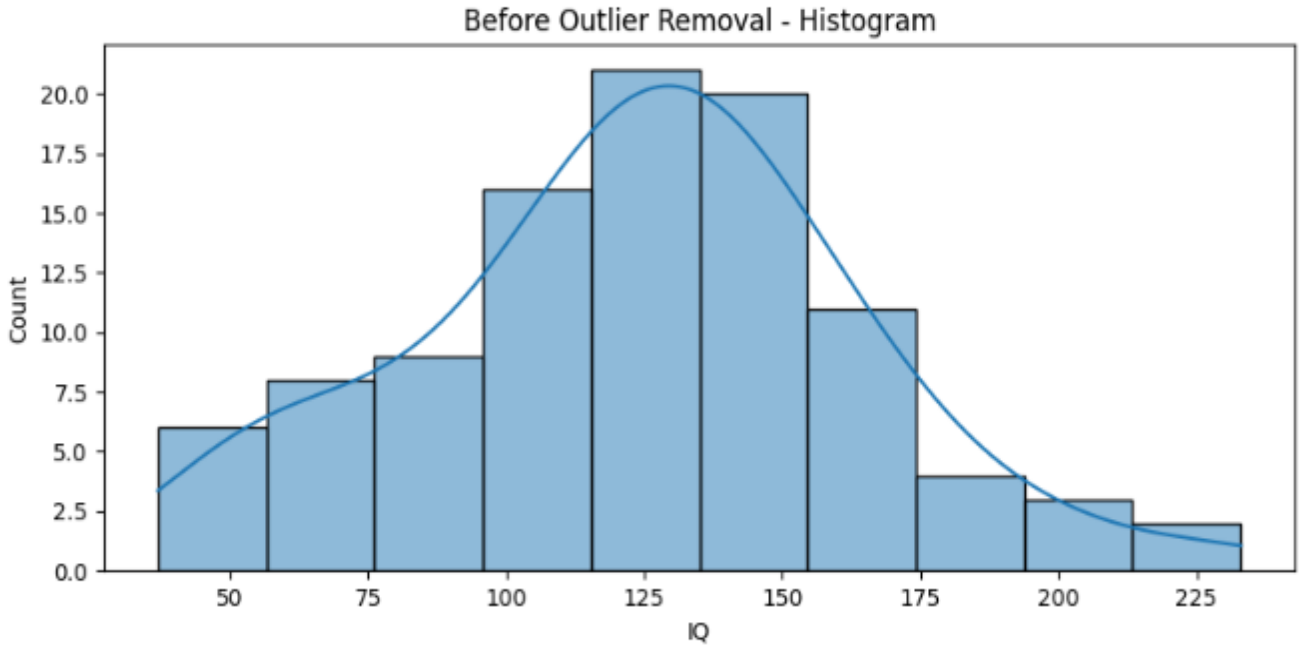
```
df[df['IQ'] < lower_limit]
```

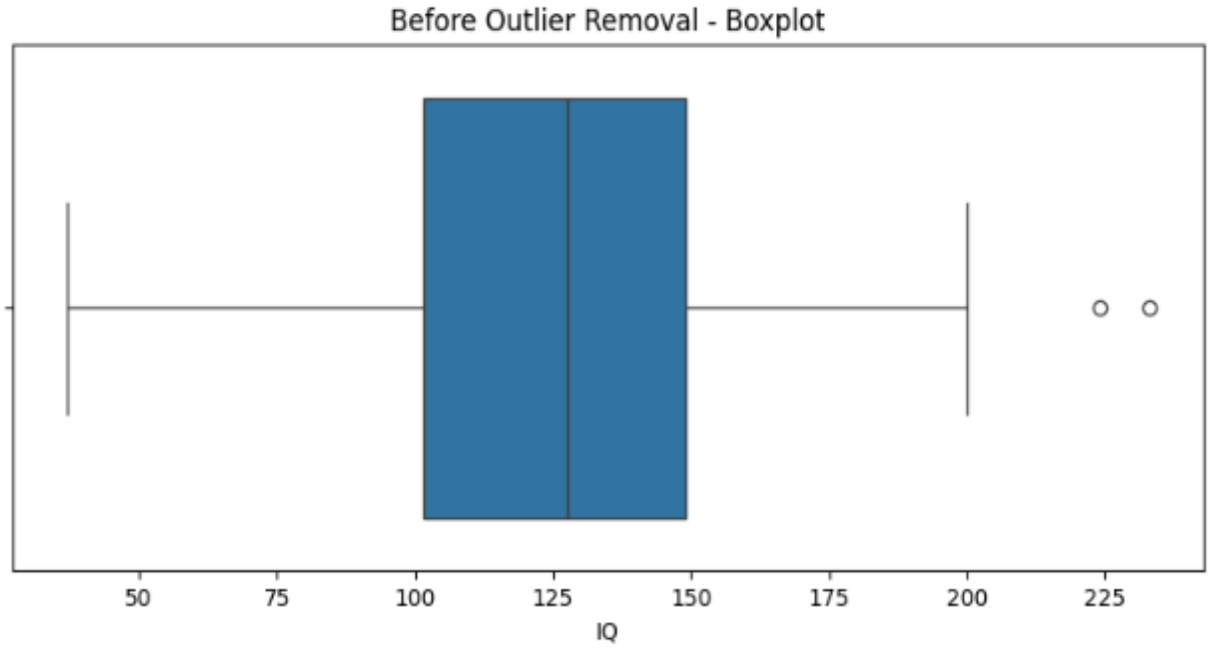
	Student_ID	CGPA	IQ	Placement	CGPA_ZScore

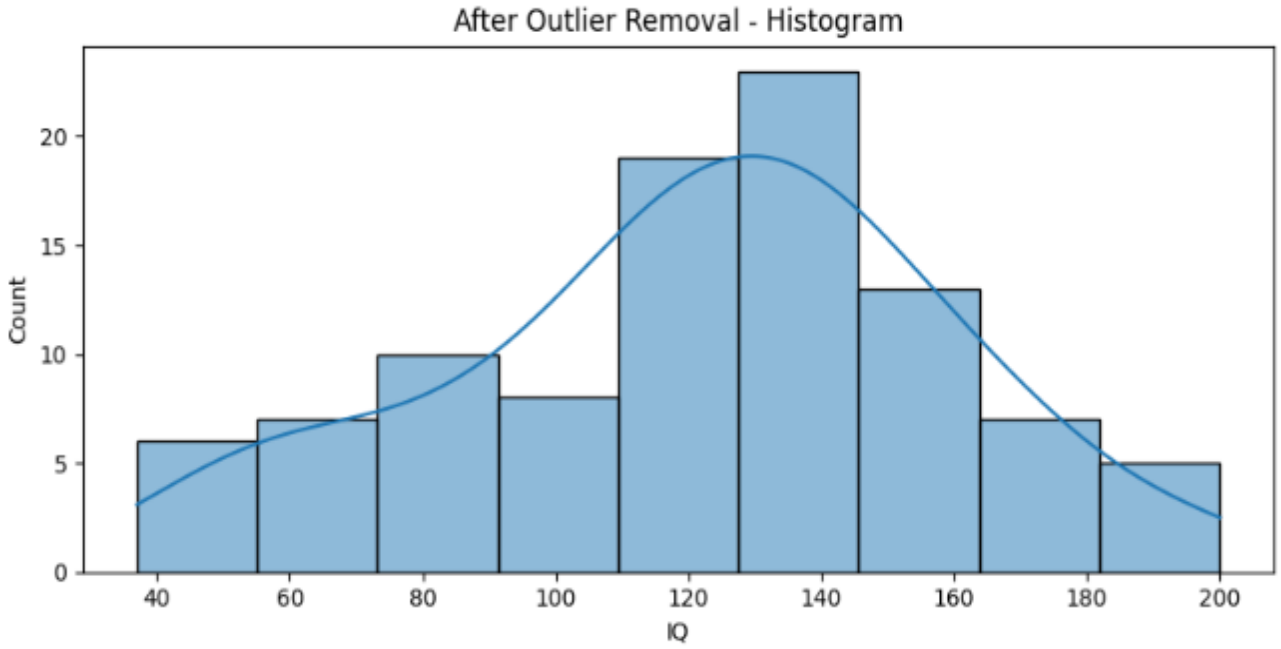
```
#trimming
new_df = df[
    (df['IQ'] >= lower_limit) &    (df['IQ'] <= upper_limit)
]
```

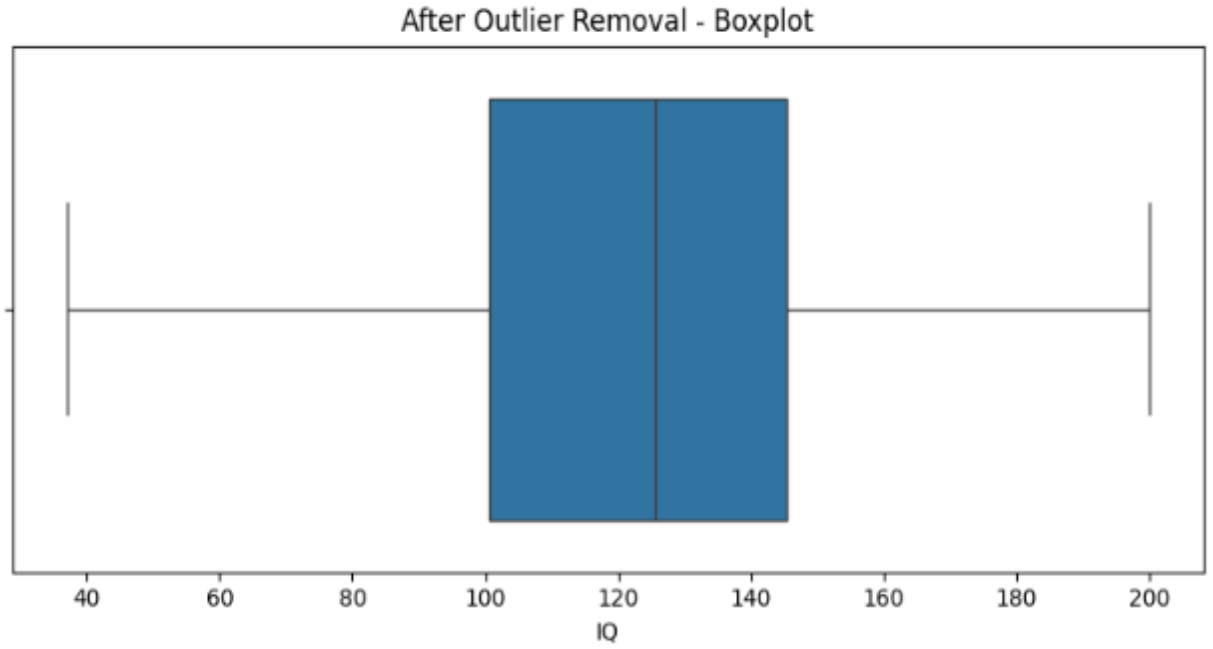
```
new_df.shape
```

```
(98, 5)
```


Before Outlier Removal - Histogram
Count
IQ
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
50
75
100
125
150
175
200
225


Before Outlier Removal - Boxplot
IQ
50
75
100
125
150
175
200
225


After Outlier Removal - Histogram
Count
IQ
0
5
10
15
20
40
60
80
100
120
140
160
180
200


After Outlier Removal - Boxplot
IQ
40
60
80
100
120
140
160
180
200

Capping

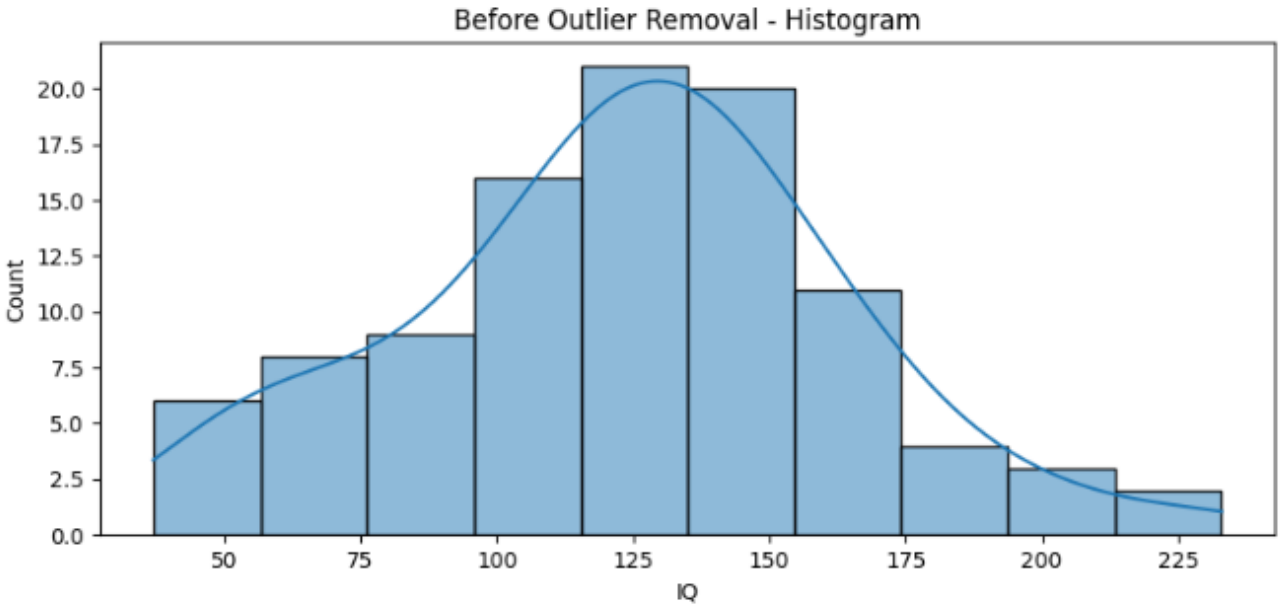
```
#capping
import numpy as np

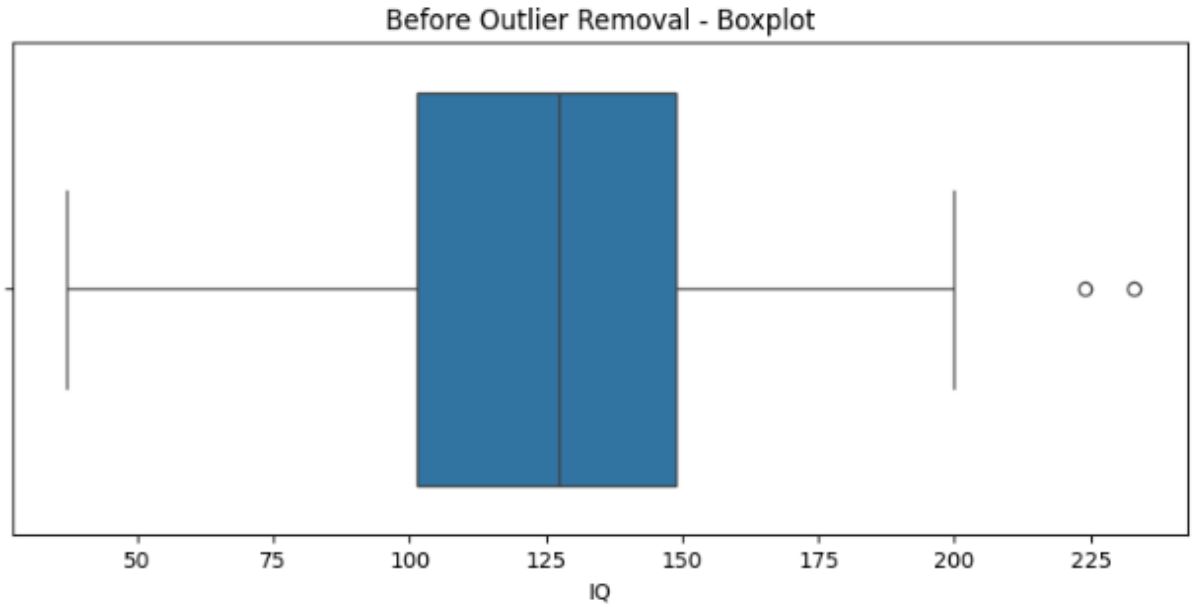
# Copy dataframe
new_df_cap = df.copy()

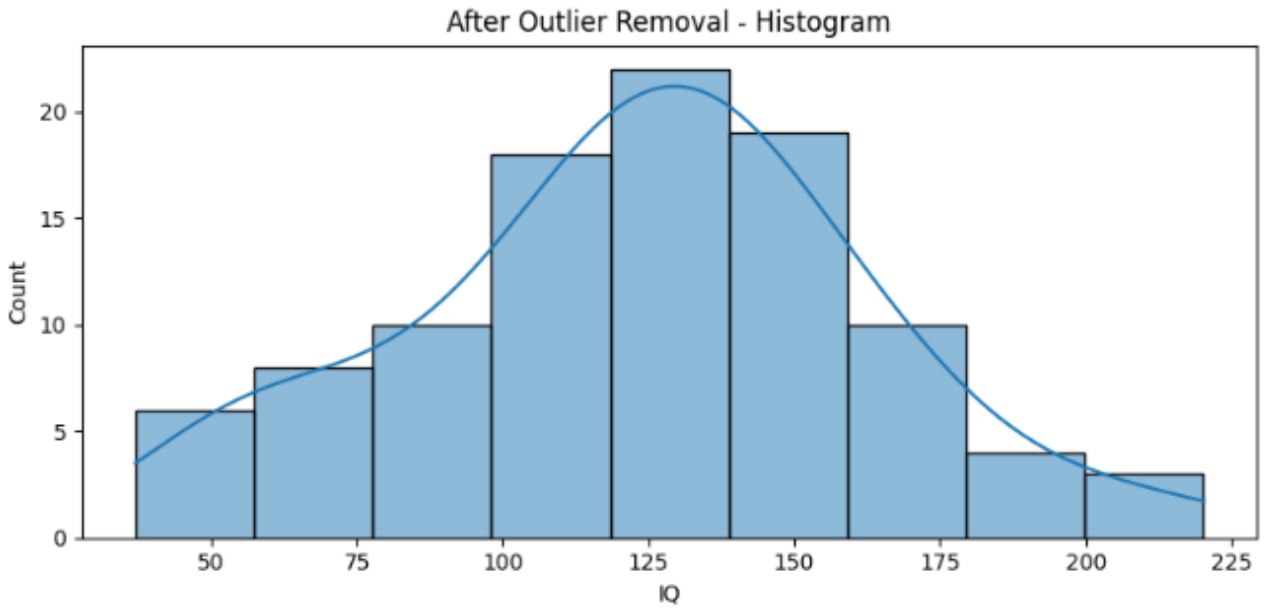
# Apply capping
new_df_cap['IQ'] = np.where(
    new_df_cap['IQ'] > upper_limit,
    upper_limit,
    np.where(
        new_df_cap['IQ'] < lower_limit,
        lower_limit,
        new_df_cap['IQ']
    )
)
```

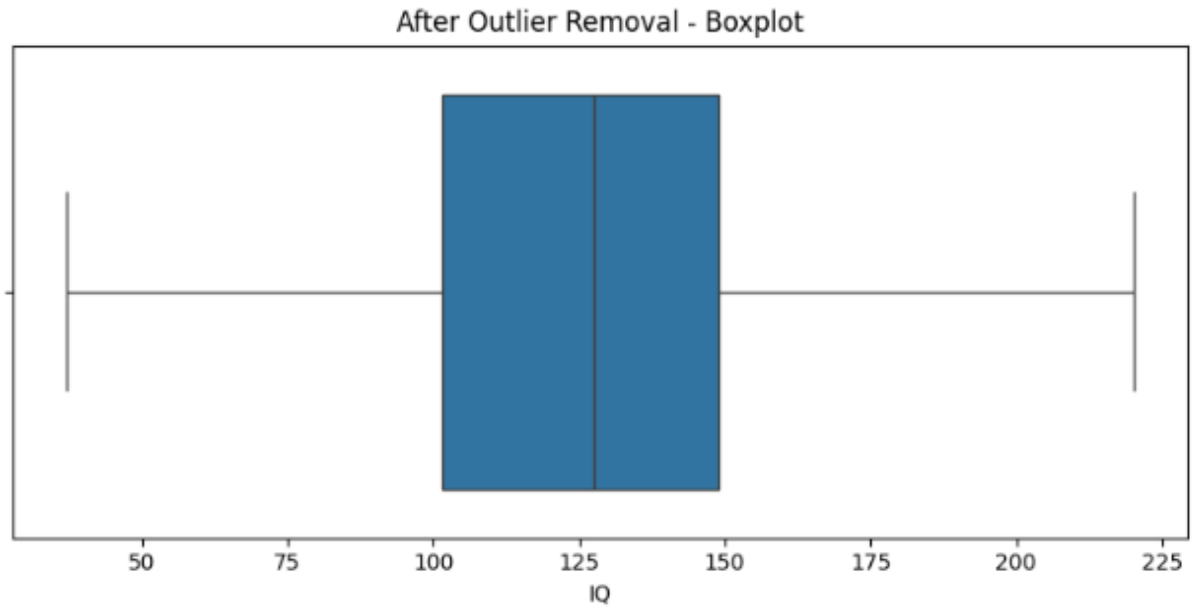
```
new_df_cap.shape
```

```
(100, 5)
```

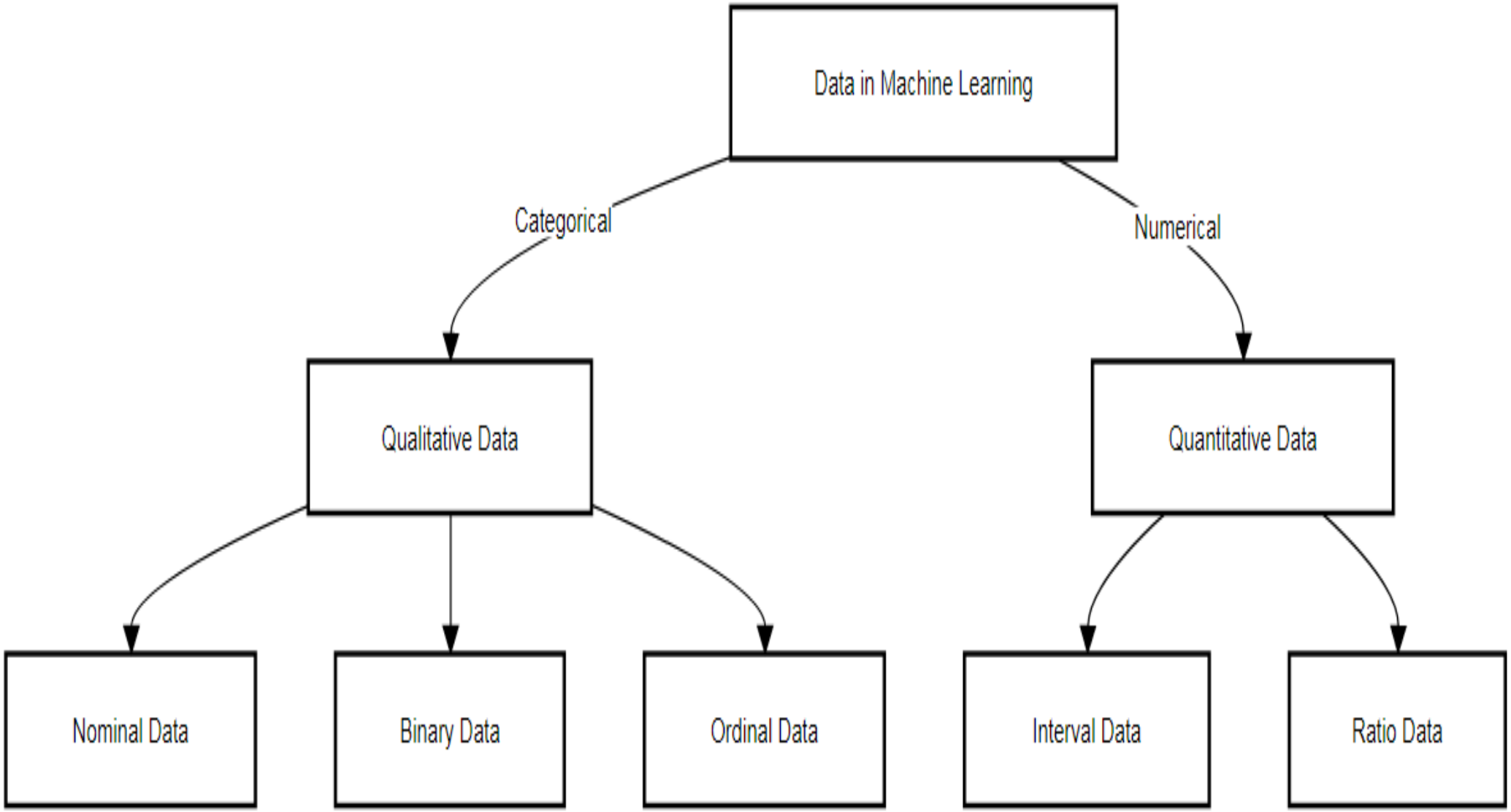

Before Outlier Removal - Histogram
Count
IQ
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
50
75
100
125
150
175
200
225


Before Outlier Removal - Boxplot
IQ
50
75
100
125
150
175
200
225


After Outlier Removal - Histogram
Count
IQ
0
5
10
15
20
50
75
100
125
150
175
200
225


After Outlier Removal - Boxplot
IQ
50
75
100
125
150
175
200
225

One Hot Encoding


Data in Machine Learning
Categorical
Numerical
Qualitative Data
Quantitative Data
Nominal Data
Binary Data
Ordinal Data
Interval Data
Ratio Data

ORDINAL DATA

- **Ordinal data** in addition to possessing the properties of nominal data, can also be naturally ordered.
- they can be arranged in a sequence of increasing or decreasing value so that we can say whether a value is better than or greater than another value.
- Examples of ordinal data are
  - Customer satisfaction: 'Very Happy', 'Happy', 'Unhappy', etc.
  - Grades: A, B, C, etc.
  - Hardness of Metal: 'Very Hard', 'Hard', 'Soft', etc.

Nominal Data

Nominal data is a type of **qualitative (categorical) data** that represents **labels or names** without any inherent order or ranking. The values in nominal data are used only for identification and classification purposes.

Characteristics of Nominal Data:

- **No natural ordering** (e.g., Red is not greater than Blue).
- **Cannot perform arithmetic operations** like addition or averaging.
- **Can only be compared for equality or inequality** (e.g., Male ≠ Female).

Examples of Nominal Data:

- **Colors:** Red, Blue, Green
- **Gender:** Male, Female, Other
- **Blood Group:** A, B, AB, O
- **Nationality:** American, Indian, Chinese
- **Modes of Transport:** Car, Bike, Bus, Train
- Statistical function such as mean, variance etc can also not be applied on nominal data
- However basic count is possible. For example, how many times B+ frequently occurring.

Categorical data

Nominal

- It has no order

Examples:

Fuel_Type → Petrol, Diesel, CNG

Brand → Honda, Bajaj, Hero

City → Delhi, Mumbai, Agra

- So we can not level it because
  - •Label encoding assigns numbers like:
  - •Petrol → 0, Diesel → 1, CNG → 2
  - •This creates a false order
- One hot encoding convert categorical data into numerical data
- because Machine Learning algorithms can not understand text

Ordinal

- It has a order
- Rating
  Poor:0,Average:1,Good:2,Excellent:3

- One-Hot Encoding is a data preprocessing technique used to convert categorical variables into numerical form so that machine learning algorithms can process them.
- It represents each category as a binary (0/1) feature.

brand	Year	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmissio	Owner	Selling_Price
ritz	2014	5.59	27000	Petrol	Dealer	Manual	0	3.35
sx4	2013	9.54	43000	Diesel	Dealer	Manual	0	4.75
ciaz	2017	9.85	6900	Petrol	Dealer	Manual	0	7.25
wagon r	2011	4.15	5200	Petrol	Dealer	Manual	0	2.85
swift	2014	6.87	42450	Diesel	Dealer	Manual	0	4.6
vitara brezza	2018	9.83	2071	Diesel	Dealer	Manual	0	9.25
ciaz	2015	8.12	18796	Petrol	Dealer	Manual	0	6.75
s cross	2015	8.61	33429	Diesel	Dealer	Manual	0	6.5
ciaz	2016	8.89	20273	Diesel	Dealer	Manual	0	8.75
ciaz	2015	8.92	42367	Diesel	Dealer	Manual	0	7.45
alto 800	2017	3.6	2135	Petrol	Dealer	Manual	0	2.85
ciaz	2015	10.38	51000	Diesel	Dealer	Manual	0	6.85
ciaz	2015	9.94	15000	Petrol	Dealer	Automatic	0	7.5
ertiga	2015	7.71	26000	Petrol	Dealer	Manual	0	6.1
dzire	2009	7.21	77427	Petrol	Dealer	Manual	0	2.25
ertiga	2016	10.79	43000	Diesel	Dealer	Manual	0	7.75
ertiga	2015	10.79	41678	Diesel	Dealer	Manual	0	7.25
ertiga	2016	10.79	43000	Diesel	Dealer	Manual	0	7.75
wagon r	2015	5.09	35500	CNG	Dealer	Manual	0	3.25
sx4	2010	7.98	41442	Petrol	Dealer	Manual	0	2.65
alto k10	2016	3.95	25000	Petrol	Dealer	Manual	0	2.85
ignis	2017	5.71	2400	Petrol	Dealer	Manual	0	4.9
sx4	2011	8.01	50000	Petrol	Dealer	Automatic	0	4.4
alto k10	2014	3.46	45280	Petrol	Dealer	Manual	0	2.5
wagon r	2013	4.41	56879	Petrol	Dealer	Manual	0	2.9
swift	2011	4.99	20000	Petrol	Dealer	Manual	0	3
swift	2013	5.87	55138	Petrol	Dealer	Manual	0	4.15
swift	2017	6.49	16200	Petrol	Individual	Manual	0	6
alto k10	2010	3.95	44542	Petrol	Dealer	Manual	0	1.95
ciaz	2015	10.38	45000	Diesel	Dealer	Manual	0	7.45
ritz	2012	5.98	51439	Diesel	Dealer	Manual	0	3.1
ritz	2011	4.89	54200	Petrol	Dealer	Manual	0	2.35
swift	2014	7.49	39000	Diesel	Dealer	Manual	0	4.95
ertiga	2014	9.95	45000	Diesel	Dealer	Manual	0	6

```
pd.get_dummies(df,columns=["Fuel_Type"])
```

Fuel_Type_CNG	Fuel_Type_Diesel	Fuel_Type_Petrol
False	False	True
False	True	False
False	False	True
False	False	True
False	True	False
...	...	...
False	True	False
False	False	True
False	False	True
False	True	False
False	False	True

Why One-Hot Encoding is Needed

- Machine learning algorithms cannot understand text data directly.
- Categorical values must be converted into numbers without introducing false order.
- In One-Hot Encoding unique category in a categorical variable,A new binary column is created.
- The column corresponding to the category is assigned as 1 and 0.

- One-hot encoding is suitable for algorithms that cannot handle categorical variables directly, such as linear regression, SVM, and KNN.
- It works well with tree-based algorithms (like decision trees, random forests, or gradient boosting), but these algorithms can sometimes handle categorical variables in other ways.

- Problem with One-hot encoding is encoded columns become dependent
- One column can be predicted using the others
- Fuel_Petrol + Fuel_Diesel + Fuel_CNG = 1
- If values of two columns are known,
- The value of the third column is automatically known
- If Fuel_Diesel = 0 and Fuel_CNG = 0 Then: Fuel_Petrol = 1 (no need to store it)

Dummy Variable Trap

- Same information is stored multiple times
- It Causes multicollinearity(Multicollinearity occurs when two or more independent variables are highly correlated)
- One feature can be predicted using another feature
- Confuses models that assume independent features
- Makes it hard to decide : Which feature is important
- Multicollinearity mainly affects:Linear Regression,Logistic Regression,Linear SVM
- Results:Coefficients change drastically
    - Interpretation becomes unreliable
    - Predictions may become unstable

```
dummies=pd.get_dummies(df,columns=["Fuel_Type"],drop_first=True)
dummies
```

Selling_Price	Fuel_Type_Diesel	Fuel_Type_Petrol
3.35	False	True
4.75	True	False
7.25	False	True
2.85	False	True
4.60	True	False
...	...	...
9.50	True	False
4.00	False	True
3.35	False	True
11.50	True	False
5.30	False	True

Your dataset has a Country feature with the values USA, UK, and India. You want to use this feature in a machine learning model. What are the different encoding techniques you can apply to this categorical feature? Explain the differences between one-hot encoding and label encoding and when each method is appropriate.

- When a dataset contains a categorical feature like Country = {USA, UK, India}, it must be converted into numerical form before using it in a machine learning model. This process is called categorical encoding.

Techniques to Encode Categorical Data

- Label Encoding
- One-Hot Encoding

Label Encoding assigns a unique integer to each category.

Example:

USA → 0
UK → 1
India → 2

- For ordinal data (ranked categories)
- For tree-based models (Decision Tree, Random Forest)

Dimensionality Reduction

- It is a key preprocessing step in machine learning and data analysis that focuses on reducing the number of features in a dataset while retaining as much information as possible.
- The greater the number of features of a machine learning model, the lower its performance.
- complex high-dimensional data often leads to overfitting.
- The dimensionality reduction provides the solution to this problem by reducing the complexity of the model and improving its generalization performance.

Dimensionality Reduction

Benefits

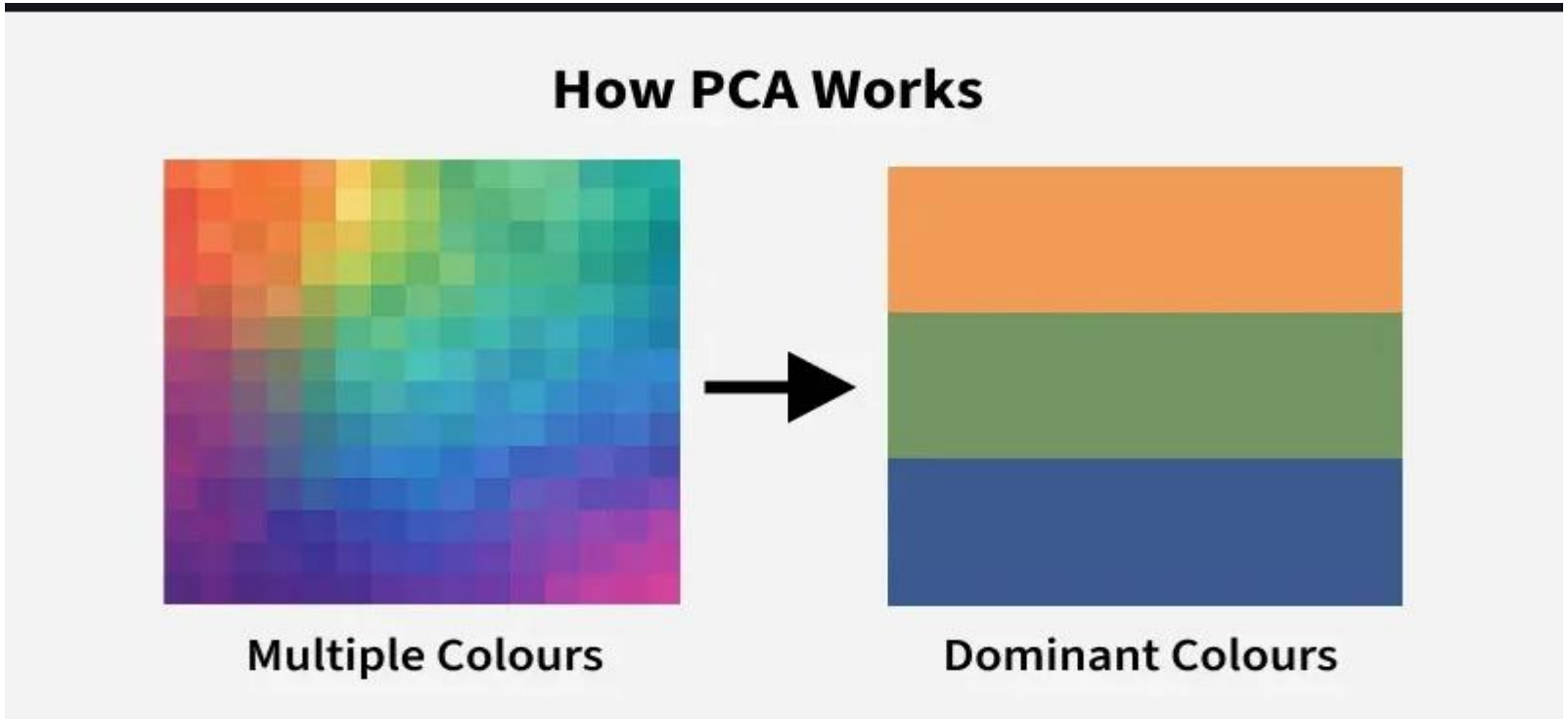
1. Reduces Overfitting: By simplifying the model and removing noise, the model generalizes better to new, unseen data.
2. Improves Model Performance: With fewer features, training times are reduced, and the model becomes less complex.
3. Mitigates the Curse of Dimensionality: In high-dimensional spaces, data points become sparse, and distance measures become less meaningful, affecting the model's performance. Dimensionality reduction addresses this problem.
4. Easier Visualization: It helps visualize high-dimensional data in 2D or 3D, which aids in gaining insights from the data

Disadvantages

1. **Loss of Information**
2. **Hard to interpret**
3. **Can reduce model accuracy**
4. **May not work for complex data**
5. **Computationally Expensive**
6. **Deciding how many features to keep**
7. **Results vary across the dataset**

PCA

- PCA (Principal Component Analysis) is a dimensionality reduction technique and helps us to reduce the number of features in a dataset while keeping the most important information.
- It changes complex datasets by transforming correlated features into a smaller set of uncorrelated components.



- It helps us to remove redundancy, improve computational efficiency and make data easier to visualize and analyze.

How Principal Component Analysis Works

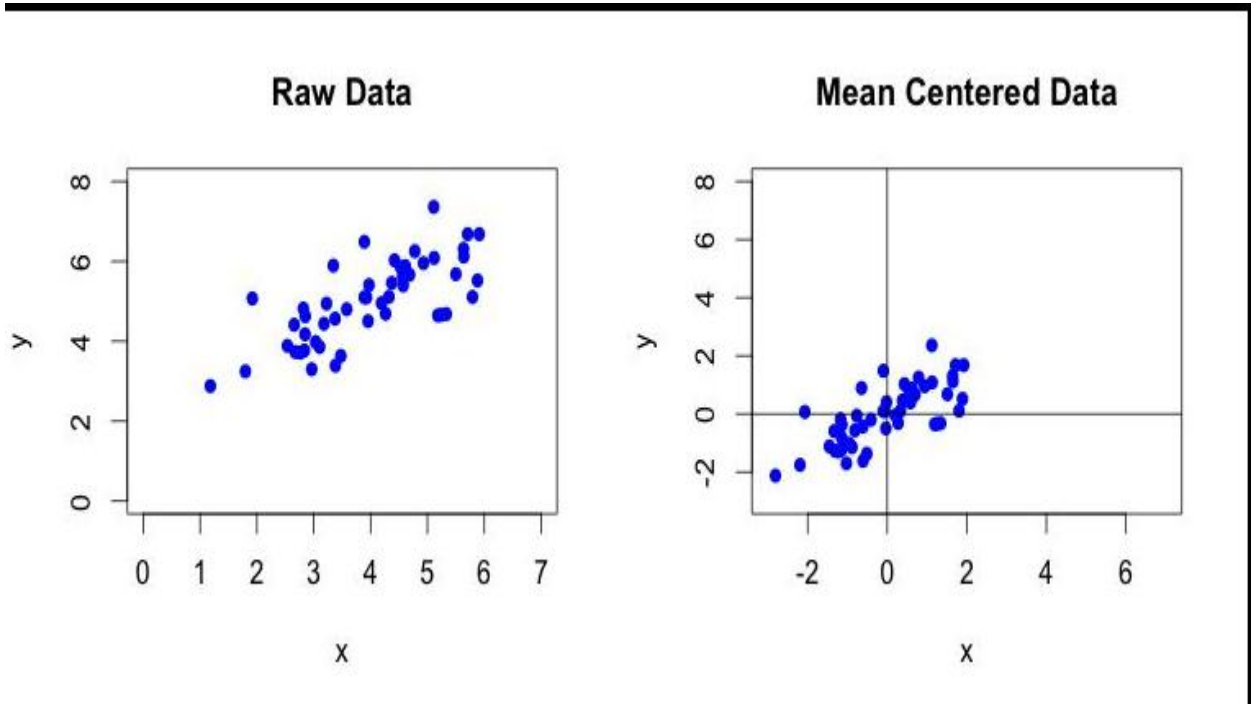
- PCA uses linear algebra to transform data into new features called principal components.
- It finds these by calculating eigenvectors (directions) and eigenvalues (importance) from the covariance matrix.
- PCA selects the top components with the highest eigenvalues and projects the data onto them simplify the dataset.

Steps of Principal Component Analysis (PCA)

Step-1: Calculate Mean of Every Feature

$$\text{Mean} = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

- This step called Mean centering which shifts the data to the origin without changing its shape



Step-2: Calculate the Covariance Matrix

This is the most important step in PCA.

Covariance tells ,How two features change together

$$Cov(R, A) = \frac{\sum(R_i - \bar{R})(A_i - \bar{A})}{n-1}$$

Covariance Matrix : $\text{Cov} = \begin{matrix} \\ X1 \\ X2 \end{matrix} \begin{matrix} \begin{matrix} X1 & X2 \end{matrix} \\ \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) \end{bmatrix} \end{matrix}$

Sum of products:

$$30.62 + 5.50 + 2.36 + 40.03 = 78.51$$

$$Cov(R, A) = \frac{78.51}{3} = 26.17$$

	f_1	f_2	f_3
f_1	$V(f_1)$	$C(f_1, f_2)$	$C(f_1, f_3)$
f_2	$C(f_2, f_1)$	$V(f_2)$	$C(f_2, f_3)$
f_3	$C(f_3, f_1)$	$C(f_3, f_2)$	$V(f_3)$

R−R̄	A−Ā	Product
-1.5	-20.41	30.62
-0.5	-10.99	5.50
0.5	4.71	2.36
1.5	26.69	40.03

Covariance Matrix

$$Cov = \begin{bmatrix} Var(R) & Cov(R, A) \\ Cov(A, R) & Var(A) \end{bmatrix}$$

$$= \begin{bmatrix} 1.67 & 26.17 \\ 26.17 & 424.19 \end{bmatrix}$$

Example

Radius	Area
1	3.14
2	12.56
3	28.26
4	50.24

Step 1: Calculate Mean of Each Feature

Mean of Radius

$$\bar{R} = \frac{1+2+3+4}{4} = \frac{10}{4} = 2.5$$

Mean of Area

$$\bar{A} = \frac{3.14+12.56+28.26+50.24}{4}$$

$$= \frac{94.2}{4} = 23.55$$

Step 2: Calculate Variance $Var(X) = \frac{\sum(x_i - \bar{x})^2}{n-1}$

Variance (Radius)=$(1-2.5)^2 + (2-2.5)^2 + (3-2.5)^2 + (4-2.5)^2$

$$= 2.25 + 0.25 + 0.25 + 2.25 = 5$$

$$Var(R) = \frac{5}{3} = 1.67$$

Variance (Area)= $(3.14-23.55)^2 + (12.56-23.55)^2 + (28.26-23.55)^2 + (50.24-23.55)^2$

$$= 416.7 + 120.78 + 22.2 + 712.9$$

$$= 1272.58$$

$$Var(A) = \frac{1272.58}{3} = 424.19$$

Find Eigenvalues

We solve:

$$| C - \lambda I | = 0$$

$$\begin{vmatrix} 1.67 - \lambda & 26.17 \\ 26.17 & 424.19 - \lambda \end{vmatrix} = 0$$

$$\lambda_1 \approx 425.8$$
$$\lambda_2 \approx 0.06$$

Interpretation of Eigenvalues

- $\lambda_1 = 425.8 \rightarrow$ Very large
- $\lambda_2 = 0.06 \rightarrow$ Very small

Total variance:

$$425.8 + 0.06 = 425.86$$

Explained variance ratio:

$$\frac{425.8}{425.86} \approx 99.98\%$$

This means PC1 alone captures almost all the information.

So we select:

$$k = 1$$

Step 4

Find Eigenvectors

Eigenvector corresponding to largest eigenvalue (PC1):

$$v_1 \approx \begin{bmatrix} 0.06 \\ 0.998 \end{bmatrix}$$

Eigenvector for small eigenvalue (PC2):

$$v_2 \approx \begin{bmatrix} -0.998 \\ 0.06 \end{bmatrix}$$

Interpretation of Eigenvectors

PC1 direction:

$$PC1 = 0.06R + 0.998A$$

Mostly aligned with Area axis,Because Area variance is much larger.

Transformation

Original centered data:

$$Z = X - \bar{X}$$

Now project:

$$Y = Z \cdot v_1$$

Each 2D point (Radius, Area) becomes:

$$PC1 = 0.06(R - \bar{R}) + 0.998(A - \bar{A})$$

Now data becomes **1D instead of 2D**.

A financial analyst develops a stock price prediction model but faces high-dimensional data, increasing complexity and overfitting risks. How can Principal Component Analysis (PCA) help? Explain PCA's role in dimensionality reduction, its ability to preserve variance, its impact on model performance, and the potential trade-offs of applying PCA.	3	2	An	C

Confusion Matrix

- Confusion matrix is a simple table used to measure how well a classification model is performing.
- It compares the predictions made by the model with the actual results and shows where the model was right or wrong.
- This helps you understand where the model is making mistakes so you can improve it. It breaks down the predictions into four categories:

True Positive (TP): The model correctly predicted a positive outcome i.e the actual outcome was positive.

True Negative (TN): The model correctly predicted a negative outcome i.e the actual outcome was negative.

False Positive (FP): The model incorrectly predicted a positive outcome i.e the actual outcome was negative.

False Negative (FN): The model incorrectly predicted a negative outcome i.e the actual outcome was positive.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Confusion Matrix

It also helps calculate key measures like accuracy, precision and recall which give a better idea of performance especially when the data is imbalanced.

Metrics based on Confusion Matrix Data

Accuracy

- Accuracy shows how many predictions the model got right out of all the predictions.
- It gives idea of overall performance but it can be misleading when one class is more dominant over the other.
- For example a model that predicts the majority class correctly most of the time might have high accuracy but still fail to capture important details about other classes.
- It can be calculated using the formula:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

- Precision focus on the quality of the model's positive predictions.
- It tells us how many of the "positive" predictions were actually correct.
- It is important in situations where false positives need to be minimized such as detecting spam emails or fraud.
- The formula of precision is:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

- measures how good the model is at predicting positives.
- It shows the proportion of true positives detected out of all the actual positive instances.
- High recall is essential when missing positive cases has significant consequences like in medical tests.

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score

- combines precision and recall into a single metric to balance their trade-off.
- It provides a better sense of a model's overall performance particularly for imbalanced datasets.
- It is helpful when both false positives and false negatives are important though it assumes precision and recall are equally important but in some situations, one might matter more than the other.

$$\text{F1-Score} = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

Specificity

- is another important metric in the evaluation of classification models particularly in binary classification.
- It measures the ability of a model to correctly identify negative instances.
- Specificity is also known as the True Negative Rate Formula is given by:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Example: Confusion Matrix for Dog Image Recognition with Numbers

Index	1	2	3	4	5	6	7	8	9	10
Actual	Dog	Dog	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Not Dog	Not Dog
Predicted	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Dog	Dog	Not Dog	Not Dog
Result	TP	FN	TP	TN	TP	FP	TP	TP	TN	TN

- Actual Dog Counts = 6
- Actual Not Dog Counts = 4
- True Positive Counts = 5
- False Positive Counts = 1
- True Negative Counts = 3
- False Negative Counts = 1

```
import numpy as np
from sklearn.metrics import confusion_matrix,classification_report
import seaborn as sns
import matplotlib.pyplot as plt
```

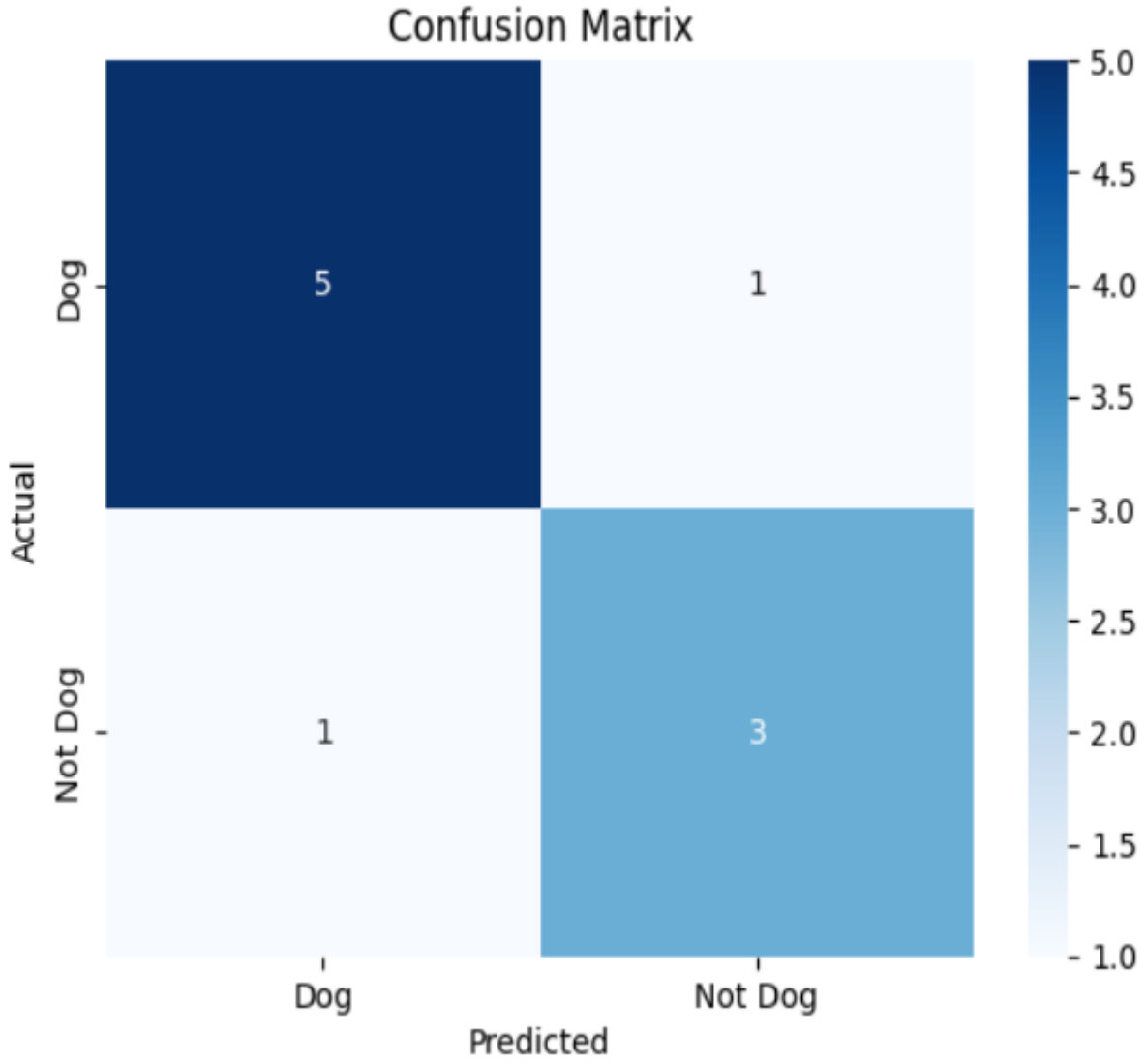
```
actual    = np.array(
  ['Dog','Dog','Dog','Not Dog','Dog','Not Dog','Dog','Dog','Not Dog','Not Dog'])
predicted = np.array(
  ['Dog','Not Dog','Dog','Not Dog','Dog','Dog','Dog','Dog','Not Dog','Not Dog'])
```

```
cm = confusion_matrix(actual,predicted)
```

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(cm,
                annot=True,
                fmt='g',
                cmap='Blues',
                xticklabels=['Dog', 'Not Dog'],
                yticklabels=['Dog', 'Not Dog'])

plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```




```
print(classification_report(actual, predicted))
```

	precision	recall	f1-score	support
Dog	0.83	0.83	0.83	6
Not Dog	0.75	0.75	0.75	4
accuracy			0.80	10
macro avg	0.79	0.79	0.79	10
weighted avg	0.80	0.80	0.80	10

A machine learning model is developed to predict whether patients have a certain disease. The confusion matrix obtained from the test dataset is given below:

	Predicted: Positive	Predicted: Negative
Actual: Positive	70	30
Actual: Negative	10	90

Based on this confusion matrix: Calculate the following evaluation metrics:

(1) Accuracy, Precision, Recall (Sensitivity), TNR (Specificity), and F1-score.

(2) If the model is to be used in a critical healthcare setting, which metric would be more important Precision or Recall? Justify your answer.

5 1 E C

- True Positive (TP) = 70
- False Negative (FN) = 30
- False Positive (FP) = 10
- True Negative (TN) = 90

Total samples:

$$70 + 30 + 10 + 90 = 200$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Accuracy = \frac{70 + 90}{200} = \frac{160}{200} = 0.80$$

Accuracy = 80%

$$\text{Specificity} = \frac{TN}{TN + FP}$$

$$Specificity = \frac{90}{90 + 10} = \frac{90}{100} = 0.90$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$Precision = \frac{70}{70 + 10} = \frac{70}{80} = 0.875$$

Precision = 87.5%

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$Recall = \frac{70}{70 + 30} = \frac{70}{100} = 0.70$$

Recall = 70%

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

$$F1 - \text{Score} = 2 \times \frac{0.875 \times 0.70}{0.875 + 0.70}$$

$$F1 = 2 \times \frac{0.6125}{1.575} = \frac{1.225}{1.575} \approx 0.778$$

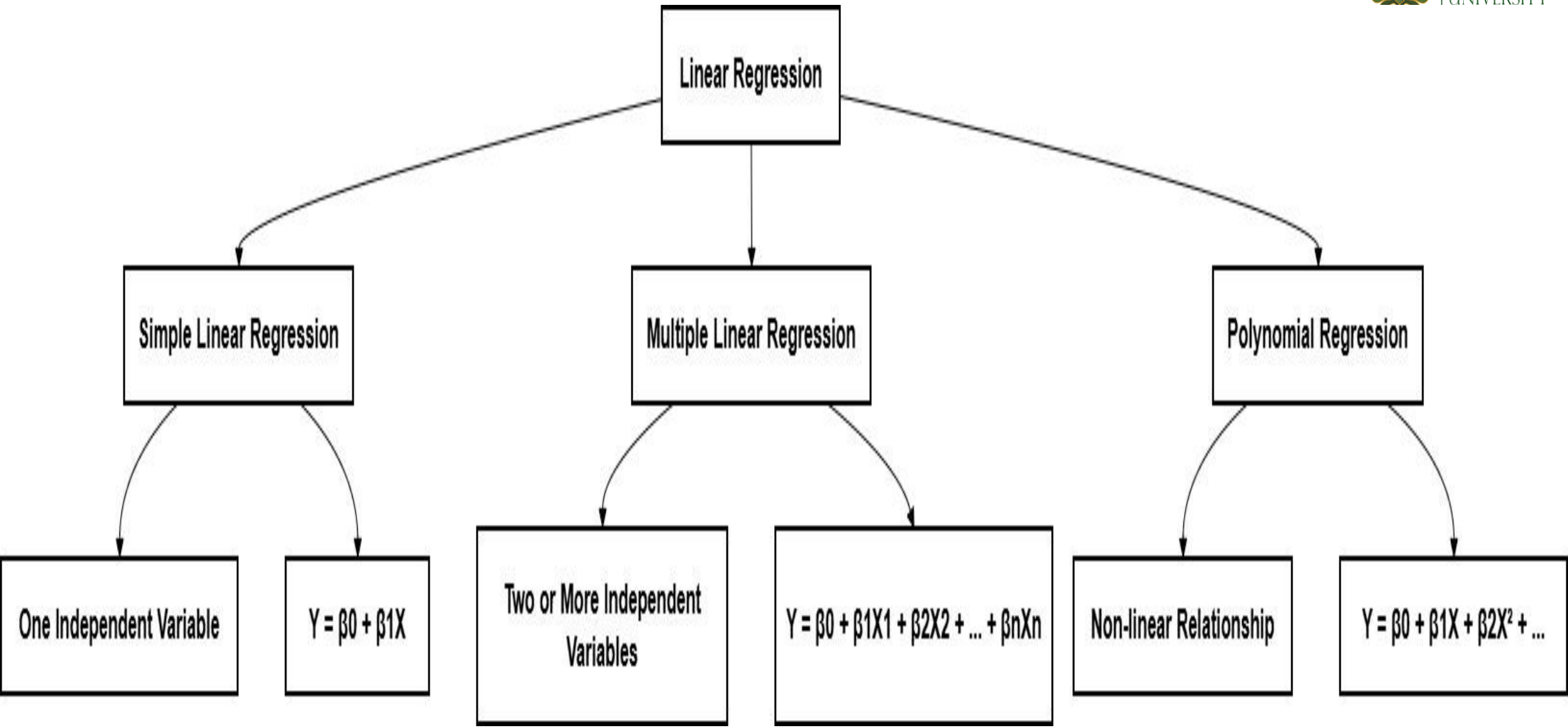
F1-Score ≈ 77.8%

Recall is MORE important

- In healthcare, False Negatives are dangerous
- A False Negative means:
- *Patient actually has the disease but the model predicts "No disease"*
- This can delay treatment and may cause serious harm or death
- Therefore:
- High Recall ensures maximum diseased patients are detected
- Some False Positives are acceptable, but missing a disease is not

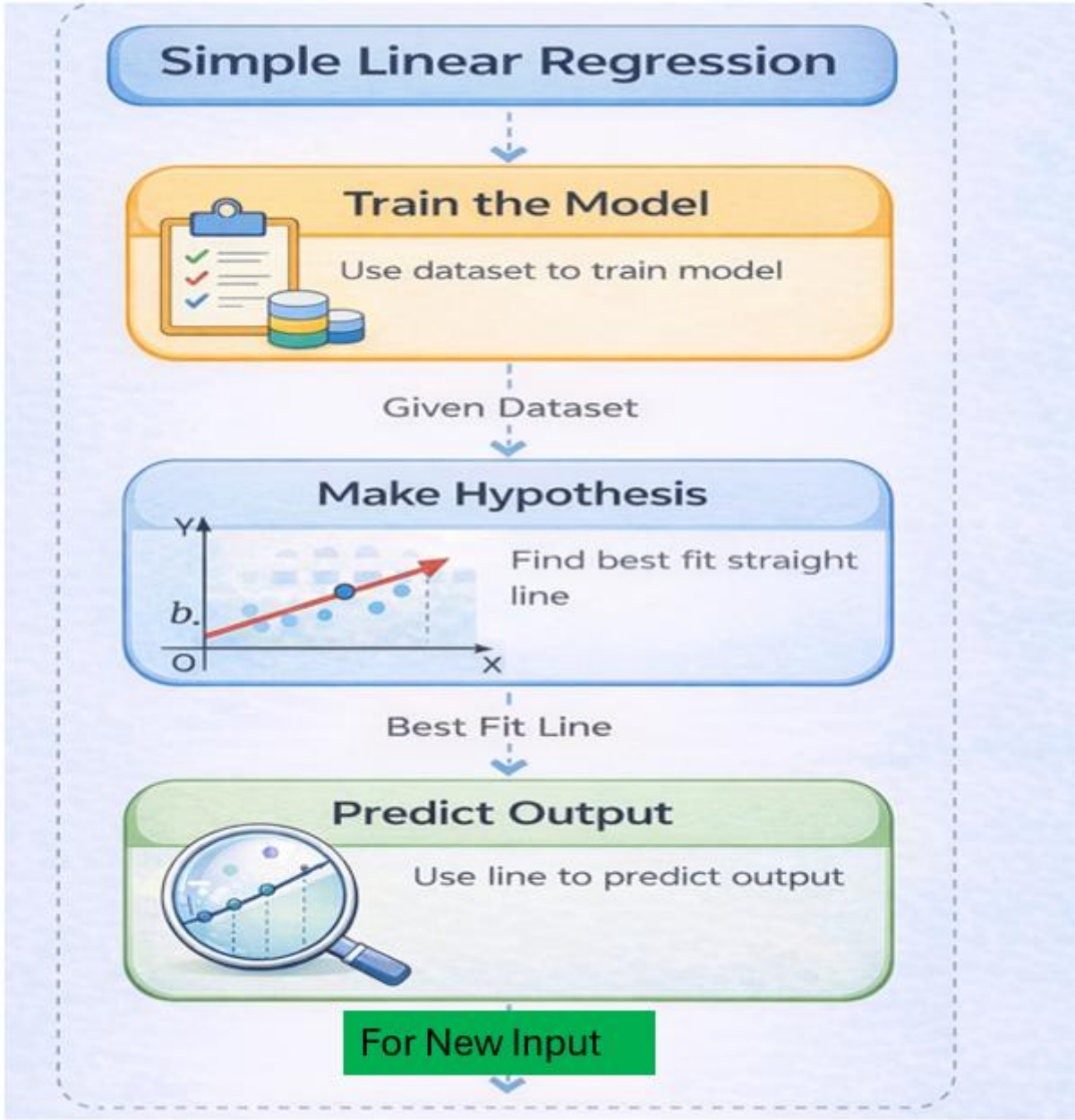
Linear Regression

- Linear Regression is Relationship between Input and output feature or independent and dependent variable.
- means that the output changes depending on the input.
- Regression Model predict the value of target variable based on new instances for example predict house price prediction based on feature like size.


Linear Regression
Simple Linear Regression
Multiple Linear Regression
Polynomial Regression
One Independent Variable
Y = β0 + β1X
Two or More Independent Variables
Y = β0 + β1X1 + β2X2 + ... + βnXn
Non-linear Relationship
Y = β0 + β1X + β2X² + ...

Simple Linear Regression Model

- simple linear Regression model used with dataset which have two feature .
- One is Input feature another is output feature.
- In simple linear regression, we first train the model using the given dataset.
- During training, the model learns a hypothesis in the form of a best fit line.
- This line represents the relationship between input and output,
- and it is then used to predict the output for new input values.


Simple Linear Regression
Train the Model
Use dataset to train model
Given Dataset
Make Hypothesis
Y
b.
O
X
Find best fit straight line
Best Fit Line
Predict Output
Use line to predict output
For New Input

Step 1: take a dataset which have two feature one is input and another is output

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

Double-click (or enter) to edit

```
df = pd.read_csv('/content/placement.csv')
df.head()
```

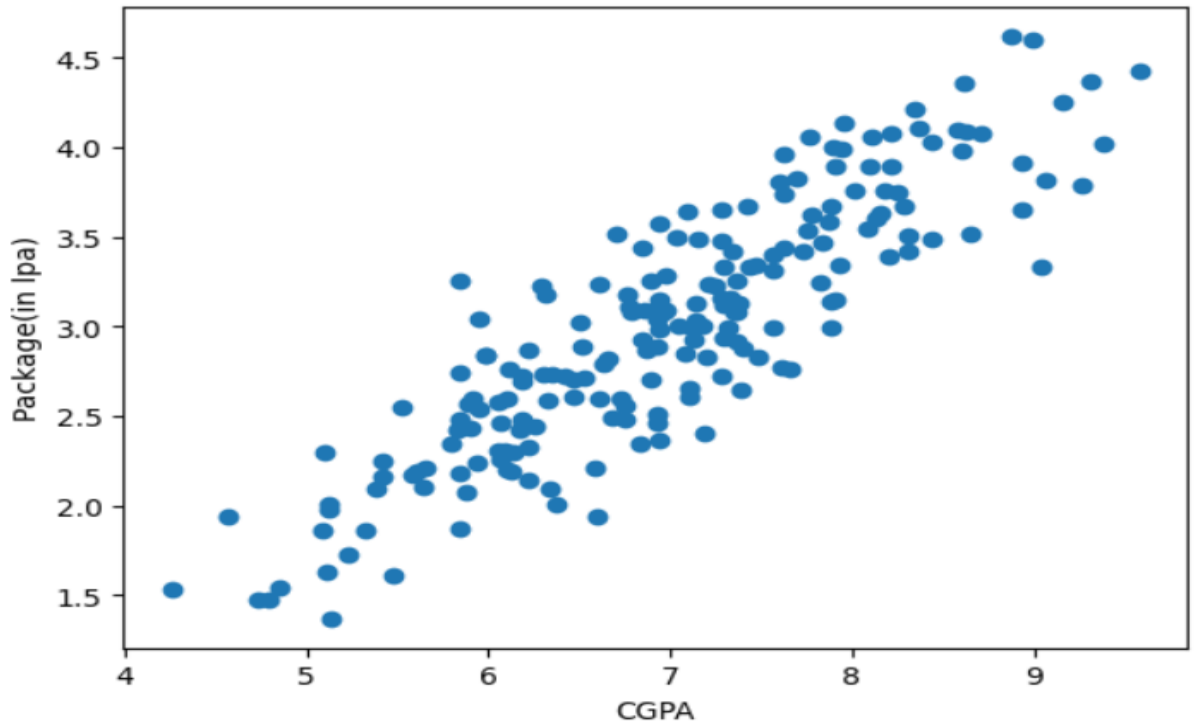
	cgpa	package
0	6.89	3.26
1	5.12	1.98
2	7.82	3.25
3	7.42	3.67
4	6.94	3.57

Step 2: Check Whether Data is Linearly Distributed

To apply Simple Linear Regression, we must check whether the relationship between input (X) and output (Y) is linear.

```
plt.scatter(df['cgpa'], df['package'])
plt.xlabel('CGPA')
plt.ylabel('Package(in lpa)')
```

```
Text(0, 0.5, 'Package(in lpa)')
```



Step 3: Split Dataset into Training Set and Testing Set

```
X = df.iloc[:, 0:1]
y = df.iloc[:, -1]
```

```
# Import library
from sklearn.model_selection import train_test_split

# Splitting the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,       # 20% data for testing
    random_state=42      # For reproducibility
)

# Check shapes
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

Step 4: Train the Model

```
from sklearn.linear_model import LinearRegression
```

```
lr = LinearRegression()
```

```
lr.fit(X_train, y_train)
```

```
LinearRegression()
```

LinearRegression

LinearRegression()

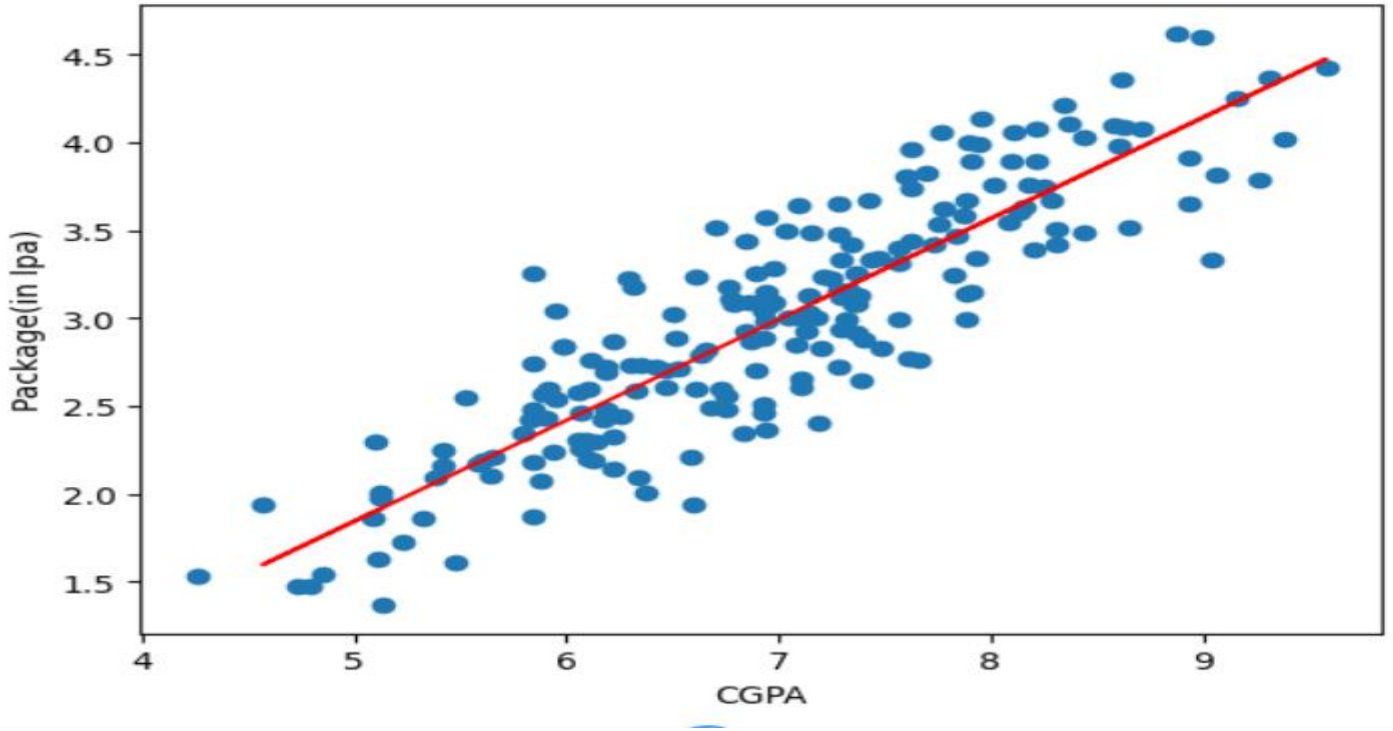
Best fit line for predicted output

```
lr.predict(X_test.iloc[1].values.reshape(1,1))
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: U
  warnings.warn(
array([3.13635249])
```

```
plt.scatter(df['cgpa'], df['package'])
plt.plot(X_train, lr.predict(X_train), color='red')
plt.xlabel('CGPA')
plt.ylabel('Package(in lpa)')
```

```
Text(0, 0.5, 'Package(in lpa)')
```




```
X_test.head()
```

	cgpa
95	6.63
15	7.25
30	7.36
158	5.95
128	7.93

```
y_train.head()
```

	package
79	3.00
197	3.24
38	4.36
24	2.71
122	2.01

dtype: float64

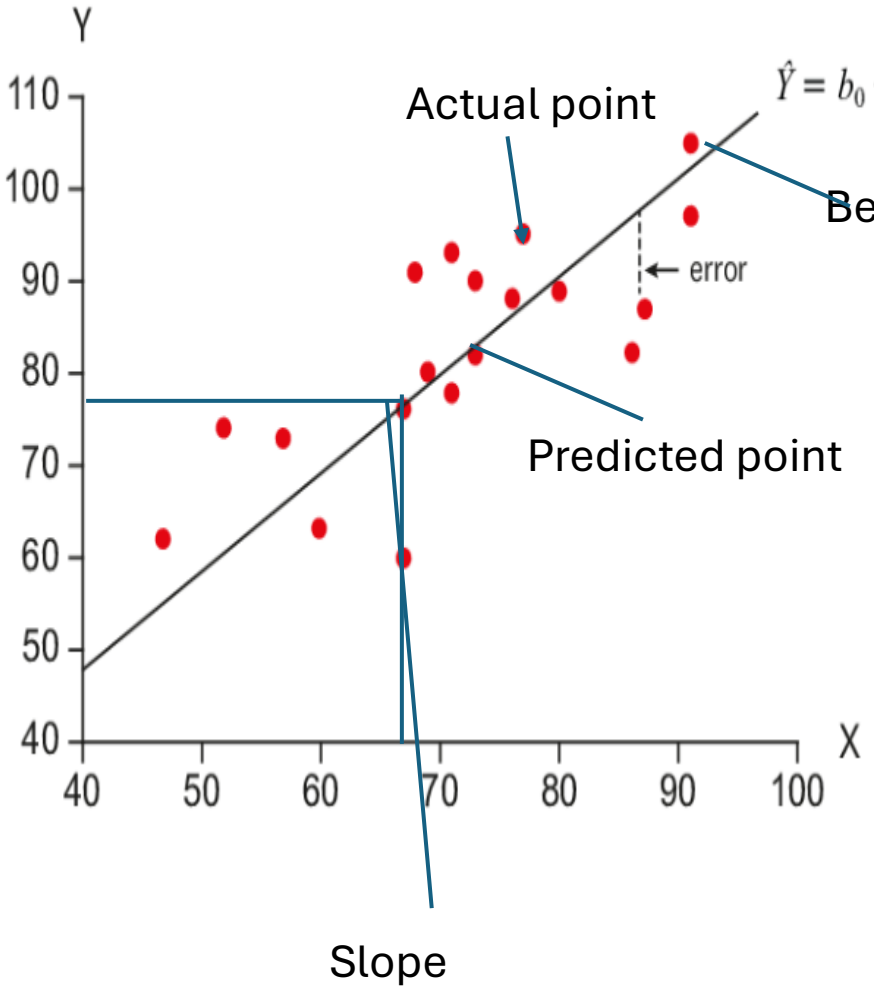
This best fit line is calculated using line equation Y=mx +C

```
m = lr.coef_
b = lr.intercept_

# y = mx + b
m * 7.25 + b
```

```
array([3.13635249])
```

➢ The main aim of Linear Regression makes best fit line



➢ Equation of the line is: Y=mx+c

➢ Or in regression algorithm term we can write :$Y=b_0 +b_1X$

Intercept

Slope or Coefficient

➢ Slope or coefficient tells if X increases by 1 unit, how much Y increases?
- If m positive → Positive relationship
  If m negative → Negative relationship

➢ Intercept tells when X is 0 where best fit line intercept Y

y=mx+c

Where:

m = slope (rate of change)

c = intercept (starting value)

Residual (Error): The vertical distance between : Actual value and Predicted value

That distance is called Residual (Error).

$$Residual\ Error = Actual\ \ point - Predicted\ point$$

The main Aim of the Regression algorithm is to find best fit line with minimum error.

That means we choose a line that minimizes:

$\sum(y - \hat{y})^2$ (where Y is Actual point and $\hat{y}$ is predicted point)

This is called Mean Squared Error (MSE).

Cost Function

A **cost function** is a mathematical function that measures how much error a model makes while predicting.
It calculates the difference between:

- **Actual value (y)**
- **Predicted value (ŷ)**

In Linear Regression The cost function used is **Mean Squared Error (MSE)**.

$$J(\theta_0, \theta_1) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

Where:

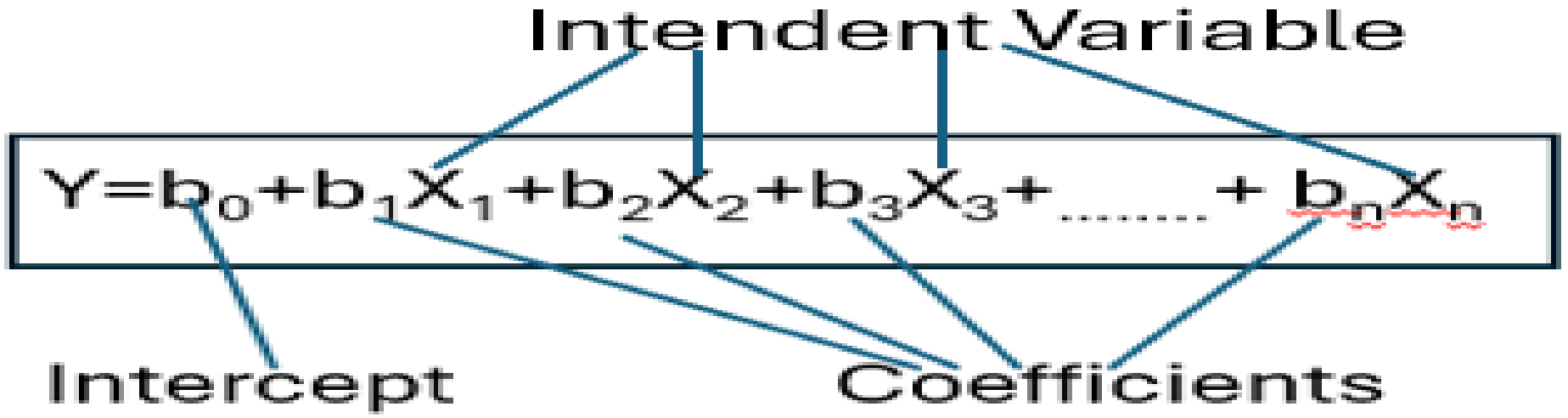
- m= number of training examples
- $h_\theta(x)$ =predicted value
- y= actual value

Advantage

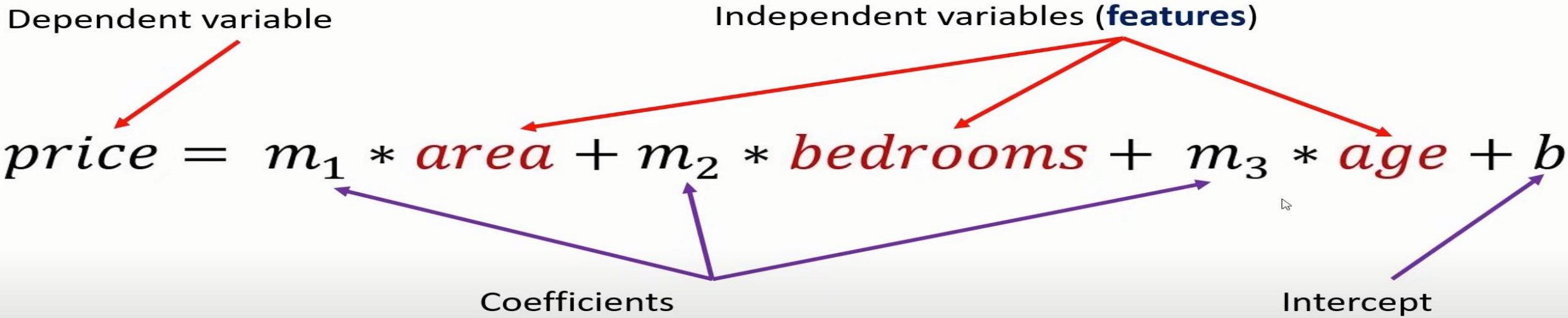
- It tells how well the model is performing.
- Training aims to minimize the cost function.
- Lower cost → Better model.

Multiple Linear Regression

- Multiple Linear Regression is an extension of simple linear regression.
- It models the relationship between one dependent variable and two or more independent variable.



Example: House price prediction



```
from sklearn import linear_model
import pandas as pd
import numpy as np
```

```
data = {
    'area': [2600, 3000, 3200, 3600, 4000],
    'bedrooms': [3, 4, 3, 3, 5],
    'age': [20, 15, 18, 30, 8],
    'price': [550000, 565000, 610000, 595000, 760000]
}

df = pd.DataFrame(data)

print(df)
```

```
   area  bedrooms  age   price
0  2600         3   20  550000
1  3000         4   15  565000
2  3200         3   18  610000
3  3600         3   30  595000
4  4000         5    8  760000
```

```
reg_mod = linear_model.LinearRegression()
```

```
df = pd.DataFrame(data)
reg_mod.fit(df[['area','bedrooms','age']], df.price)
```

LinearRegression

LinearRegression()

```
#coffecient
reg_mod.coef_
```

```
array([   137.25, -26025.  ,  -6825.  ])
```

```
#Intercept
reg_mod.intercept_
```

```
np.float64(383724.9999999998)
```

```
reg_mod.predict([[3600,3,30]])
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation
  warnings.warn(
array([595000.])
```

```
#Price=b0+b1(Area)+b2(Bedrooms)+b3(Age)
383724+3600*137.25+3*-26025+30*-6825
```

```
594999.0
```

You are using a linear regression model to predict house prices based on the following data:
Size of house (in square feet): [1000, 1500, 2000, 2500]
Price of house (in $1000): [300, 400, 500, 600]
Fit a linear regression model to this data and find the equation of the line (i.e., determine the slope and intercept).

Solution:

Linear Regression Equation

$$Y = b_0 + b_1 X$$

X	Y	X^2	XY
1000	300	1,000,000	300,000
1500	400	2,250,000	600,000
2000	500	4,000,000	1,000,000
2500	600	6,250,000	1,500,000
ΣX = 7000	**ΣY = 1800**	**ΣX^2 = 13,500,000**	**ΣXY = 3,400,000**

n=4

$$\sum X = 7000$$

$$\sum Y = 1800$$

$$\sum X^2 = 13,500,000$$

$$\sum XY = 3,400,000$$

Calculate Slope (b_1)

$$b_1 = \frac{n\sum XY - (\sum X)(\sum Y)}{n\sum X^2 - (\sum X)^2}$$

$$b_1 = \frac{4(3400000) - (7000)(1800)}{4(13500000) - (7000)^2}$$

$$= \frac{13600000 - 12600000}{54000000 - 49000000}$$

$$= \frac{1000000}{5000000}$$

$$= 0.2$$

Calculate intercept

$$b_0 = \frac{(\sum Y)(\sum X^2) - (\sum X)(\sum XY)}{n\sum X^2 - (\sum X)^2}$$

$$b_0 = \frac{(1800)(13500000) - (7000)(3400000)}{4(13500000) - (7000)^2}$$

$$= \frac{500000000}{54000000 - 49000000}$$

$$= \frac{500000000}{5000000}$$

$$= 100$$

Regression Equation Y=mx+C

$$\boxed{Y = 100 + 0.2X}$$

Prediction

If house size = 2000:

$$Y = 100 + 0.2(2000)$$
$$= 100 + 400$$
$$= 500$$

If house size = 1500:

$$Y = 100 + 0.2(1500)$$
$$= 100 + 300$$
$$= 400$$

Question: Find linear regression equation for the following two sets of data:

x	2	4	6	8
y	3	7	5	10

Construct the following table:

x	y	x^2	xy
2	3	4	6
4	7	16	28
6	5	36	30
8	10	64	80
$\sum x$ = 20	$\sum y$ = 25	$\sum x^2$ = 120	$\sum xy$ = 144

$$a\,(intercept) = \frac{\sum y \sum x^2 - \sum x \sum xy}{n(\sum x^2) - (\sum x)^2}$$

$$b\,(slope) = \frac{n\sum xy - (\sum x)(\sum y)}{n\sum x^2 - (\sum x)^2}$$

$$b = \frac{n\sum xy - (\sum x)(\sum y)}{n\sum x^2 - (\sum x)^2}$$

$$b = \frac{4\times 144 - 20\times 25}{4\times 120 - 400}$$

b = 0.95

$$a = \frac{\sum y \sum x^2 - \sum x \sum xy}{n(\sum x^2) - (\sum x)^2}$$

$$a = \frac{25\times 120 - 20\times 144}{4(120) - 400}$$

a = 1.5

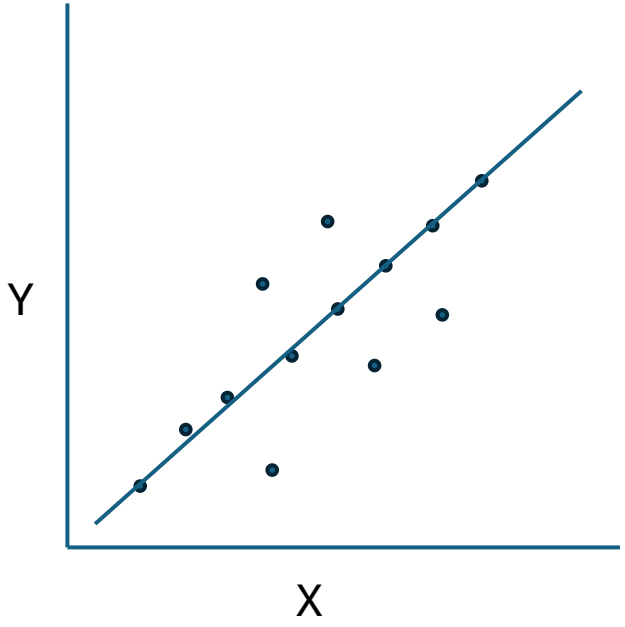
Linear regression is given by:

y = a + bx

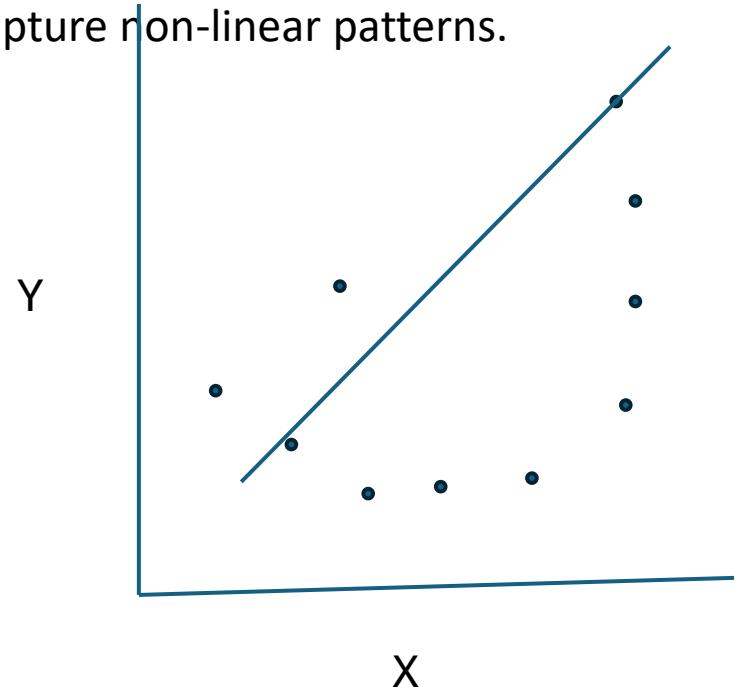
y = 1.5 + 0.95 x

Polynomial Linear Regression

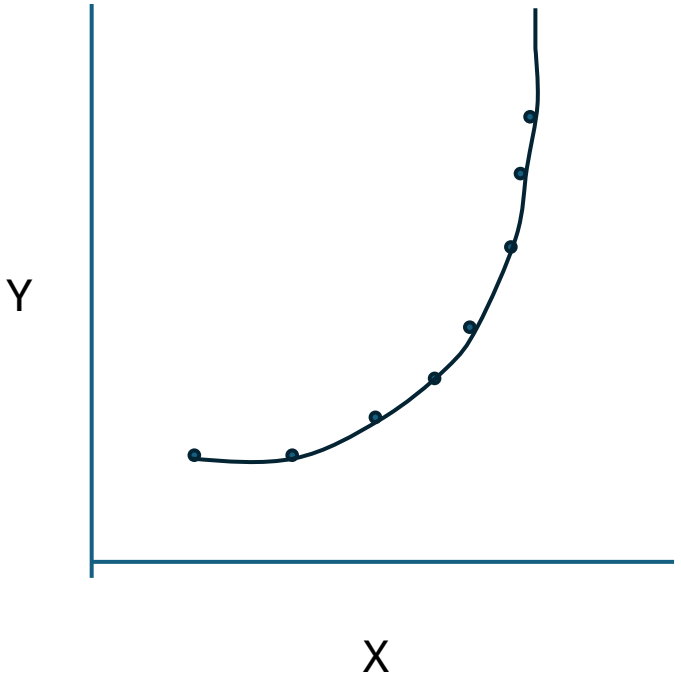
- In Simple Linear Regression, we use a straight line with formula Y = mX + C.
- It was used to best fit the line to our dataset.
- But in some datasets, it is hard to fit a straight line. Therefore, we use a polynomial function of a curve to fit all the data points.
- When data is curved (U-shape, accelerating growth), a straight line underfits.
- Polynomial Regression fits a curve to capture non-linear patterns.



Linear Relationship with best fit line



Predicted result difference is vey high , if relationship between X,Y is non linear



Best fitted model on non linear dataset

Linear Regression Equation: **Y=mX+c**

If Degree 0: $y = \text{constant}$ =Flat line

If Degree 1: $y = mx + c$ = Straight line

If Degree 2: $y = b_0 + b_1x + b_2x^2$ = U-shape curve

Degree 3+ : More flexible curves $y = b_0 + b_1x + b_2x^2 + b_3x^3$

General Polynomial Equation (Degree n)

$$y = b_0 + b_1x + b_2x^2 + b_3x^3 + \cdots + b_nx^n$$

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
```

```
data = {
    "Level": [1,2,3,4,5,6,7,8,9,10],
    "Salary": [45000,50000,60000,80000,110000,
               150000,200000,300000,500000,1000000]
}
df = pd.DataFrame(data)
X = df[["Level"]]
y = df["Salary"]
```

```
lin_model = LinearRegression()
lin_model.fit(X, y)
```

LinearRegression

LinearRegression()

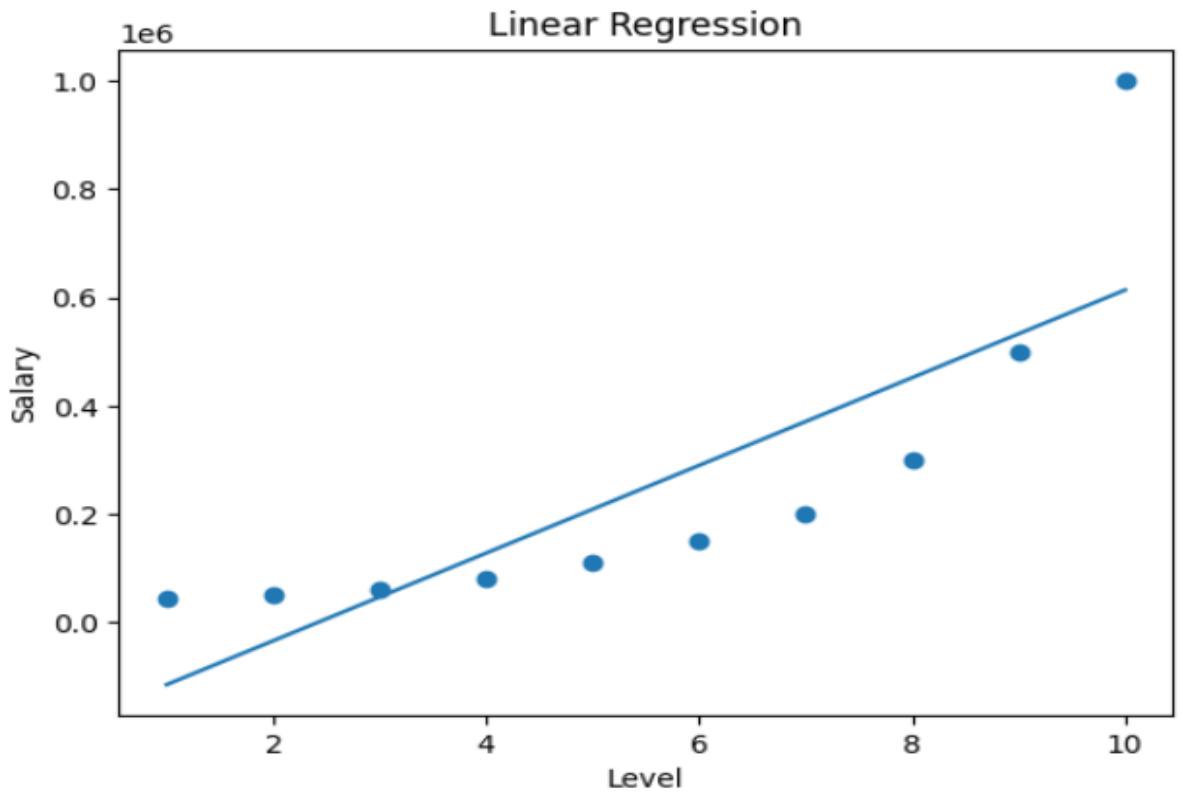
```
#Polynomial Regression Model
poly = PolynomialFeatures(degree=4)
X_poly = poly.fit_transform(X)
poly_model = LinearRegression()
poly_model.fit(X_poly, y)
```

LinearRegression

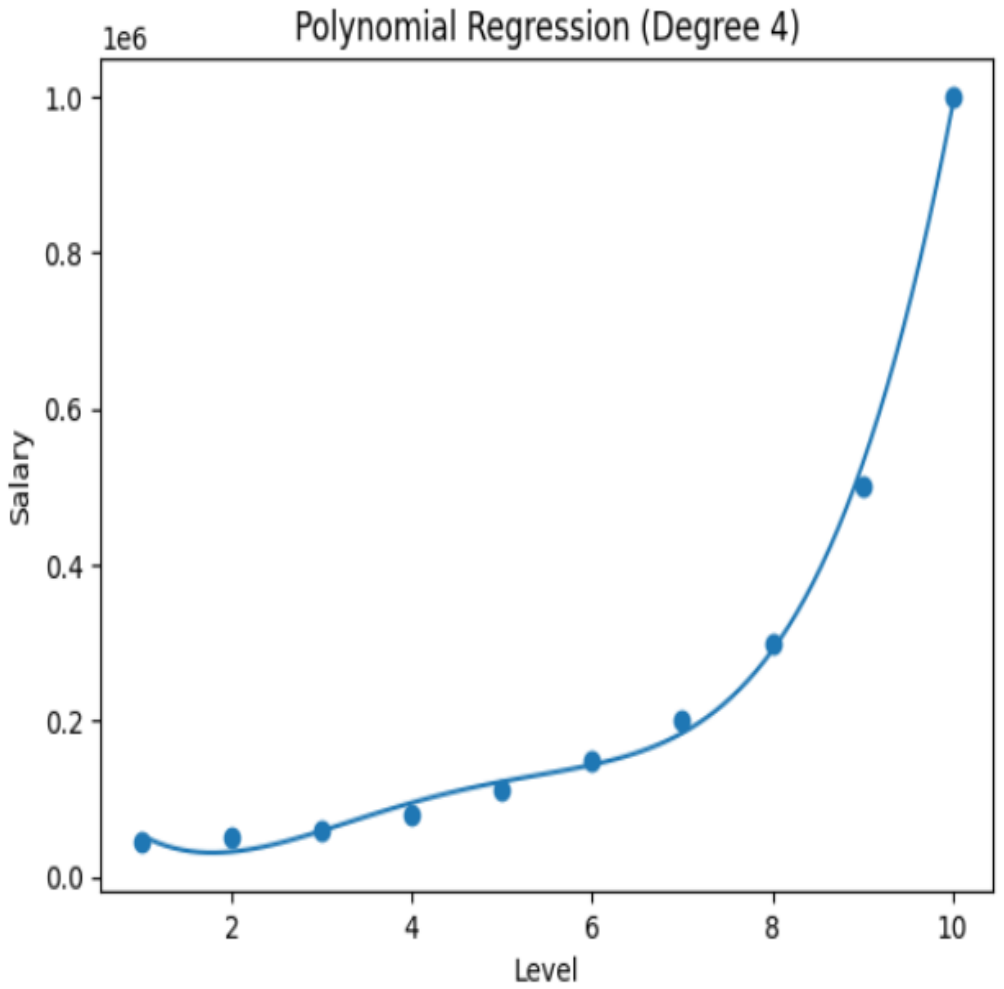
LinearRegression()

```
X_range = np.linspace(1, 10, 100).reshape(-1,1)
# Linear prediction
y_lin_pred = lin_model.predict(X_range)
# Polynomial prediction
X_range_poly = poly.transform(X_range)
y_poly_pred = poly_model.predict(X_range_poly)
```

```
plt.figure()
plt.scatter(X, y)
plt.plot(X_range, y_lin_pred)
plt.title("Linear Regression")
plt.xlabel("Level")
plt.ylabel("Salary")
plt.show()
```




```
plt.figure()
plt.scatter(X, y)
plt.plot(X_range, y_poly_pred)
plt.title("Polynomial Regression (Degree 4)")
plt.xlabel("Level")
plt.ylabel("Salary")
plt.show()
```



Logistic Regression

- Logistic Regression is a supervised classification algorithm .
- Logistic regression predicts the output of a categorical dependent variable.
- Therefore, the outcome must be a categorical or discrete value.
- It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, it gives the probabilistic values which lie between 0 and 1.
- In Logistic regression, instead of fitting a regression line, we fit an “S” shaped logistic function, which predicts two maximum values (0 or 1).

```python
import numpy as np
from sklearn.model_selection import train_test_split

data = {
    'age': [22,25,28,30,32,35,38,40,42,45,47,50,52,55,58],
    'bought_insurance': [0,0,0,0,0,1,0,1,1,1,1,1,1,1,1]
}
df = pd.DataFrame(data)
df
```

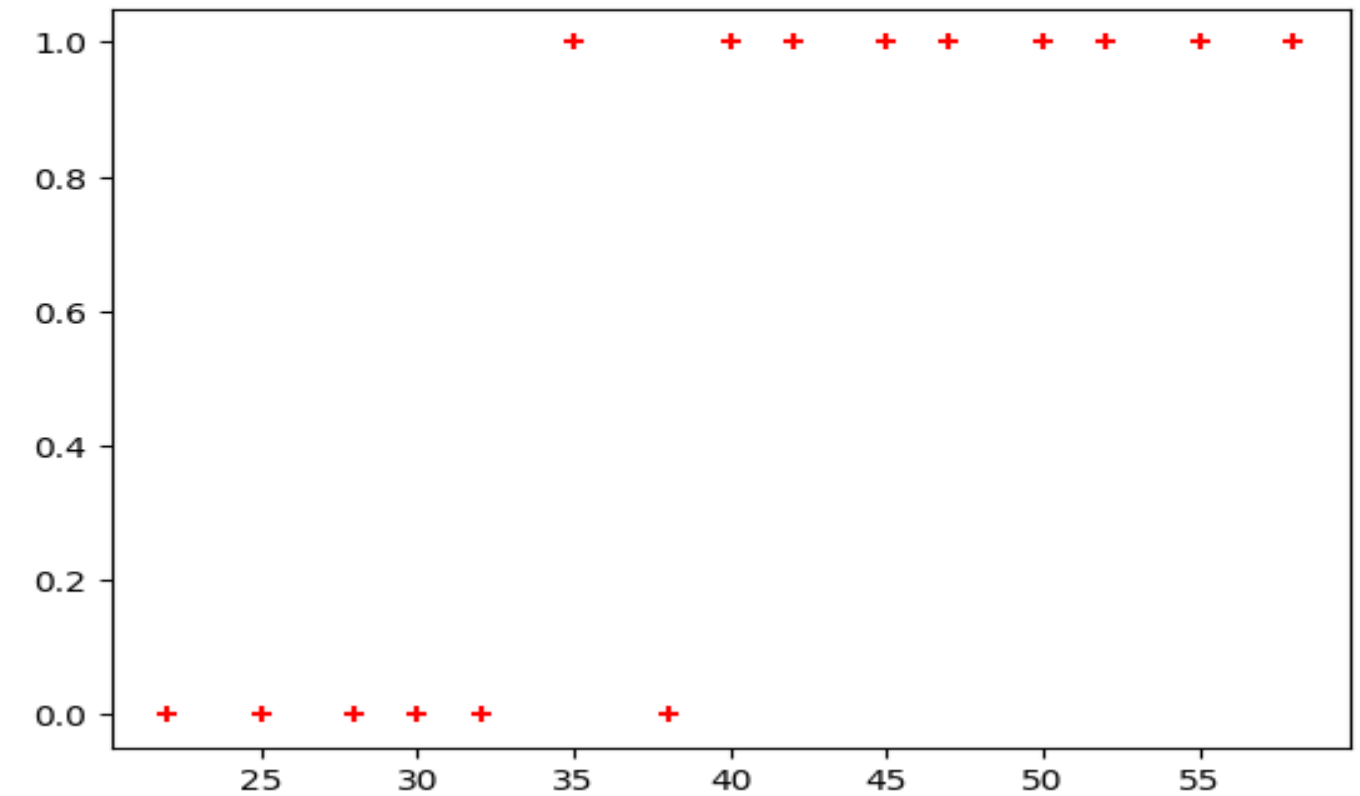
In this dataset:
0 → No Insurance
1 → Has Insurance

- Ages below 30 Mostly 0 → No insurance
- Ages above 45 Mostly 1 → Has insurance

	age	bought_insurance
0	22	0
1	25	0
2	28	0
3	30	0
4	32	0
5	35	1
6	38	0
7	40	1
8	42	1
9	45	1
10	47	1
11	50	1
12	52	1
13	55	1
14	58	1

```python
plt.scatter(df.age, df.bought_insurance, marker='+', color='red')
```

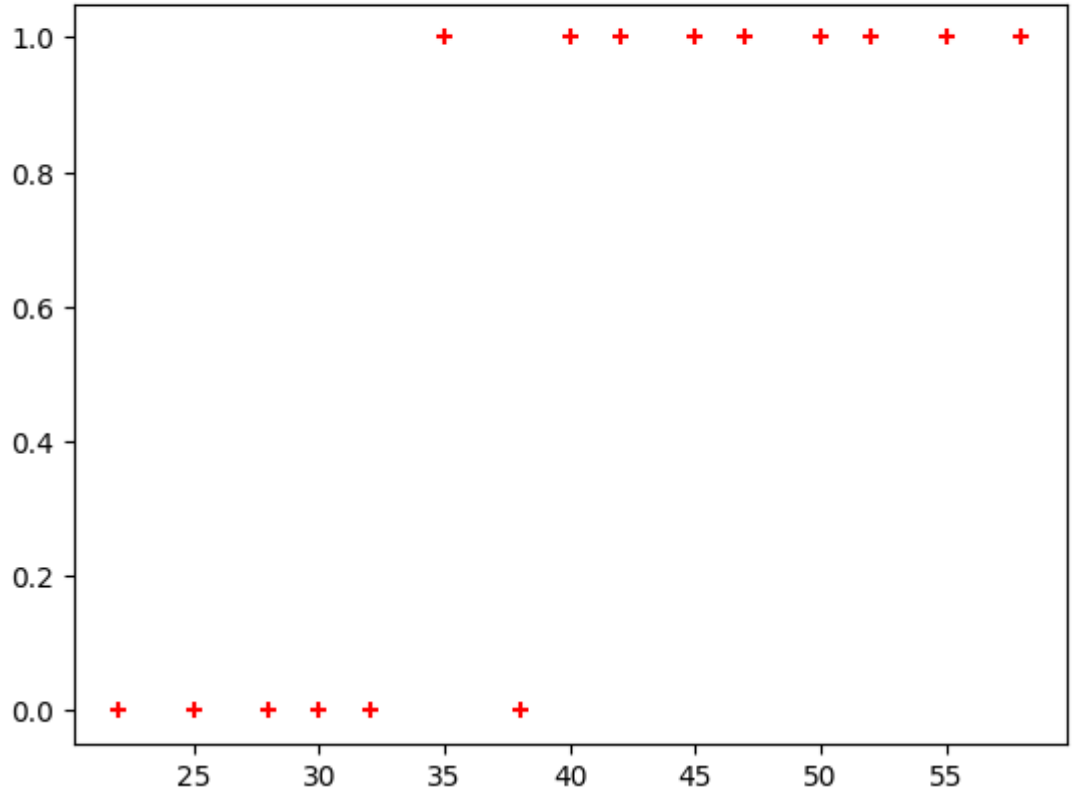
```
<matplotlib.collections.PathCollection at 0x7b05ef18f680>
```

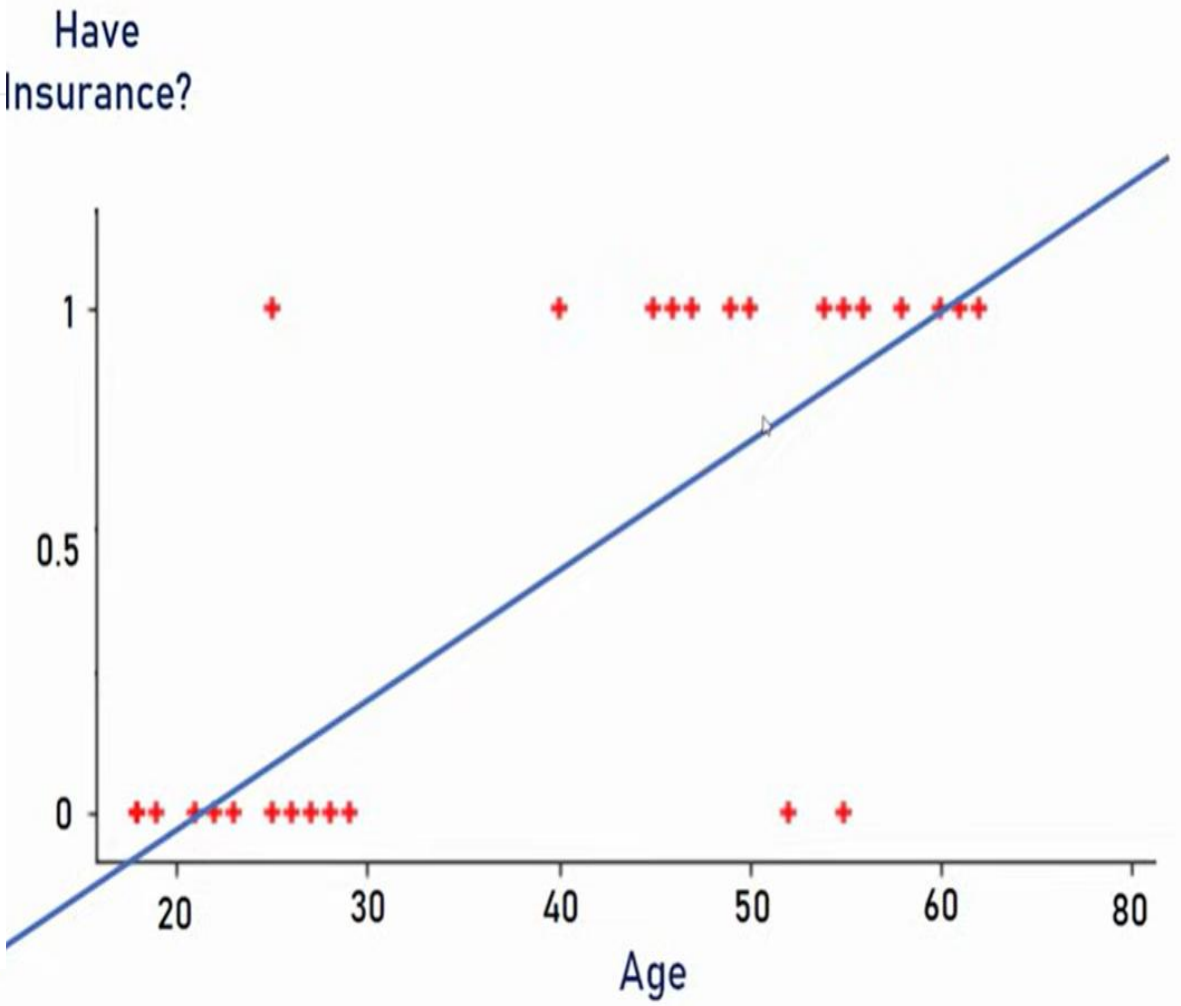


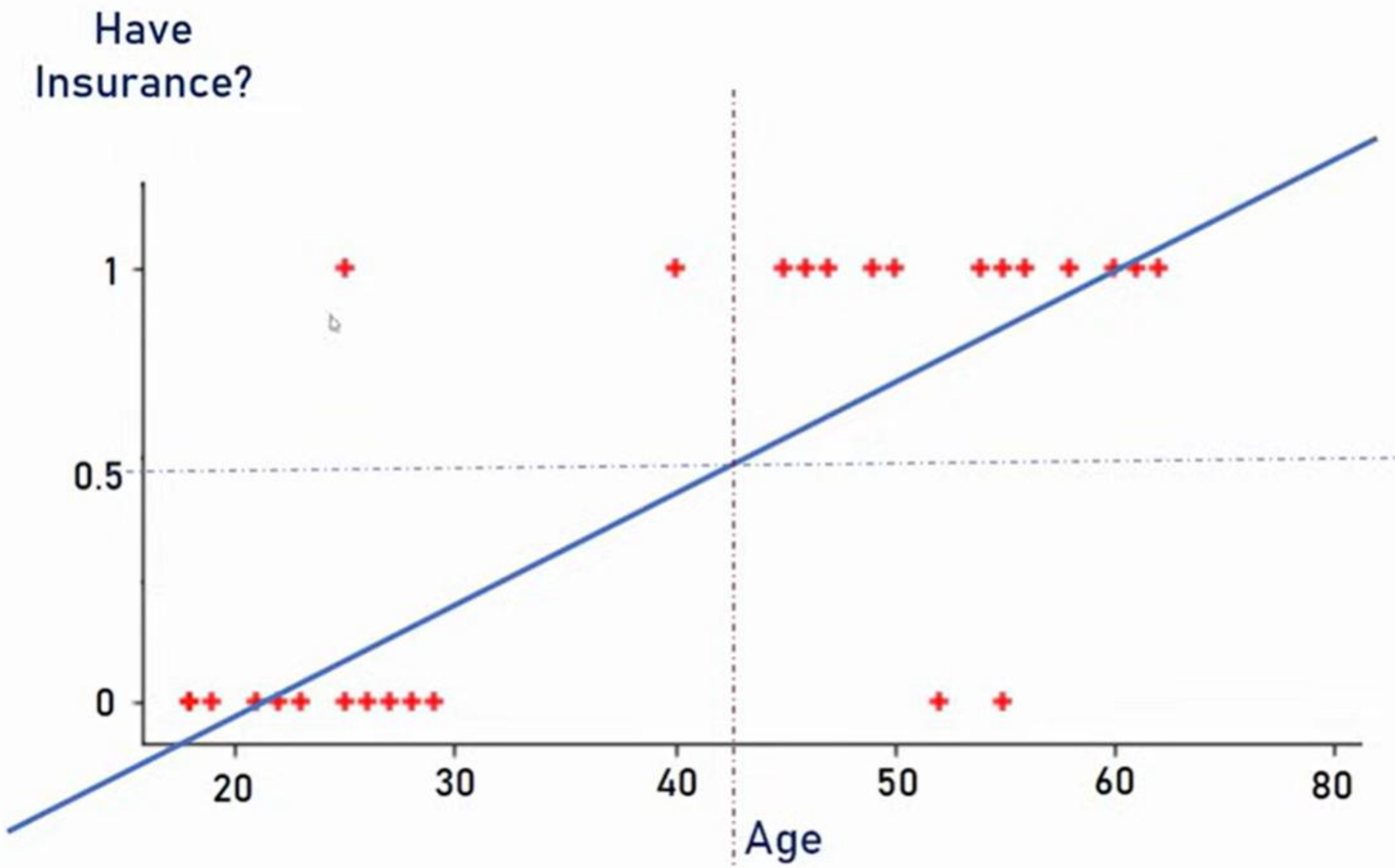
When we observe the data distribution, it is clearly not linear. Therefore, if we try to solve this problem using Linear Regression, we do not obtain a suitable best-fit line.

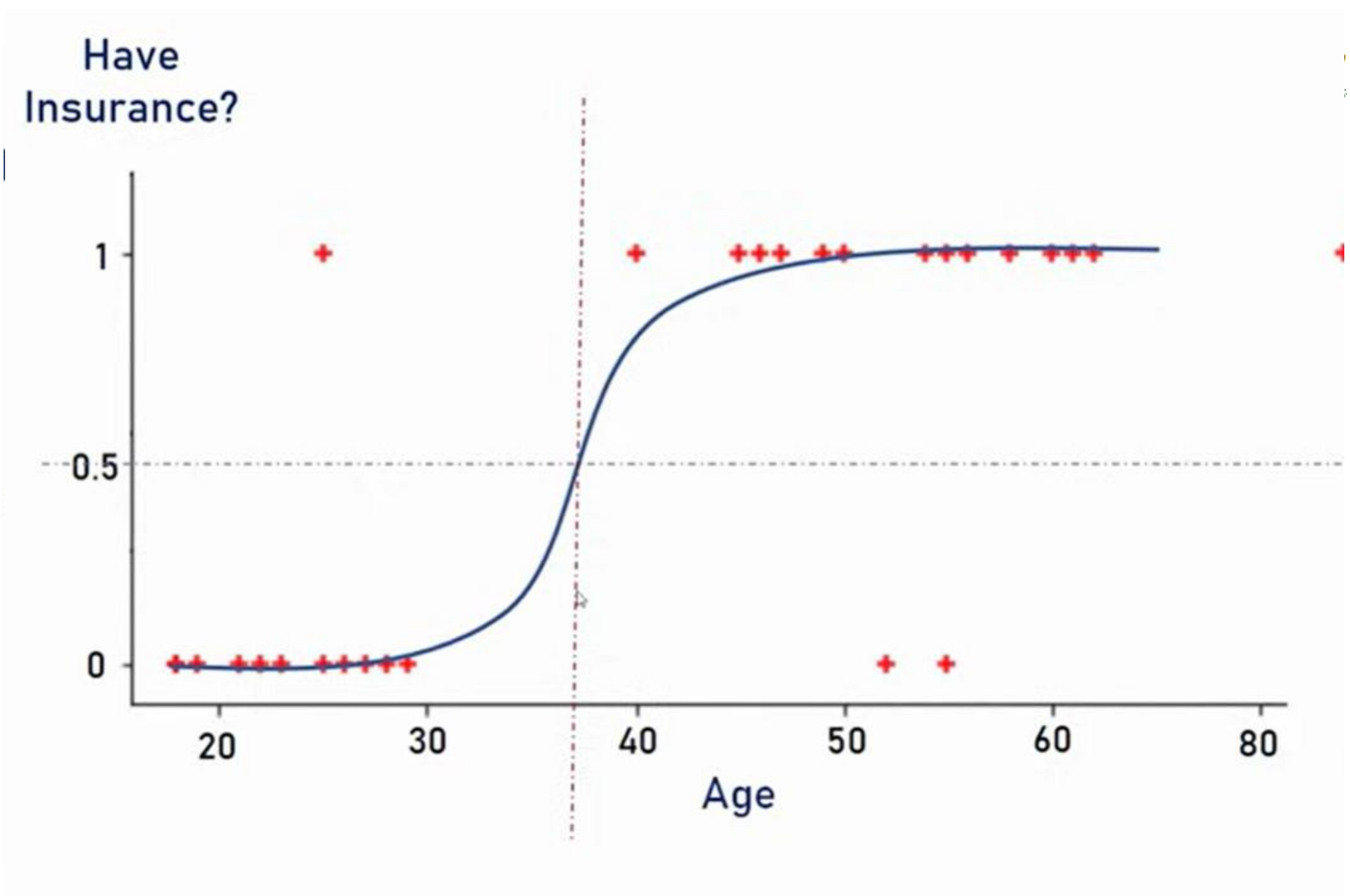
```
plt.scatter(df.age, df.bought_insurance, marker='+', color='red')
```

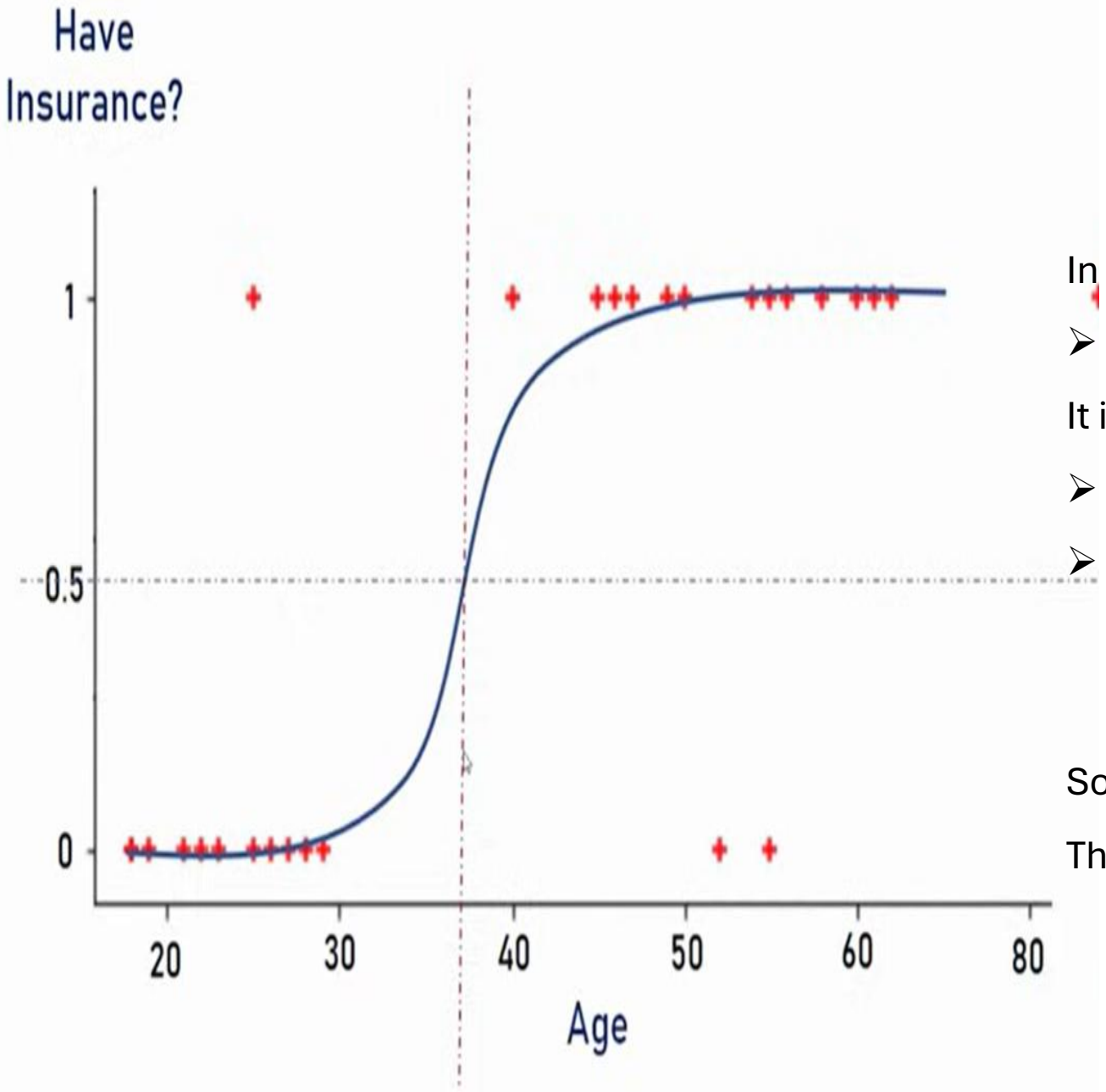
```
<matplotlib.collections.PathCollection at 0x7b05ef18f680>
```






Have
Insurance?
1
0.5
0
20
30
40
50
60
80
Age


Have
Insurance?
1
0.5
0
20
30
40
50
60
80
Age



This S shape line is sigmoid function

$$\text{sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

In Logistic Regression:

- $z = \theta_0 + \theta_1 x$ or y=mx+c

It is the linear combination of inputs.

- e is Euler's number ,Approximate value:$e \approx 2.71828$
- It converts any real number (from $-\infty$ to $+\infty$) into a value between:

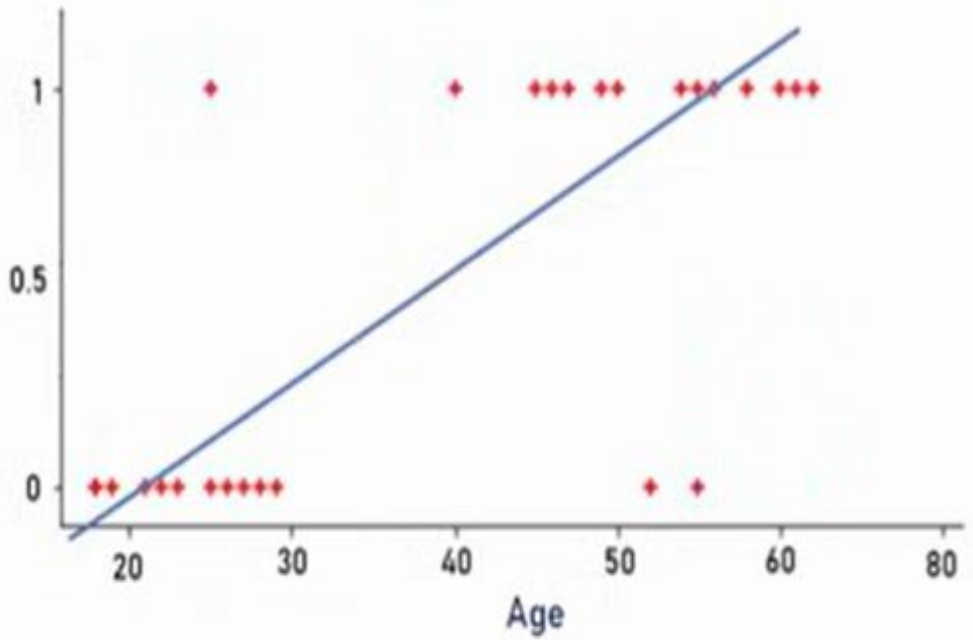
$$0 \text{ and } 1$$

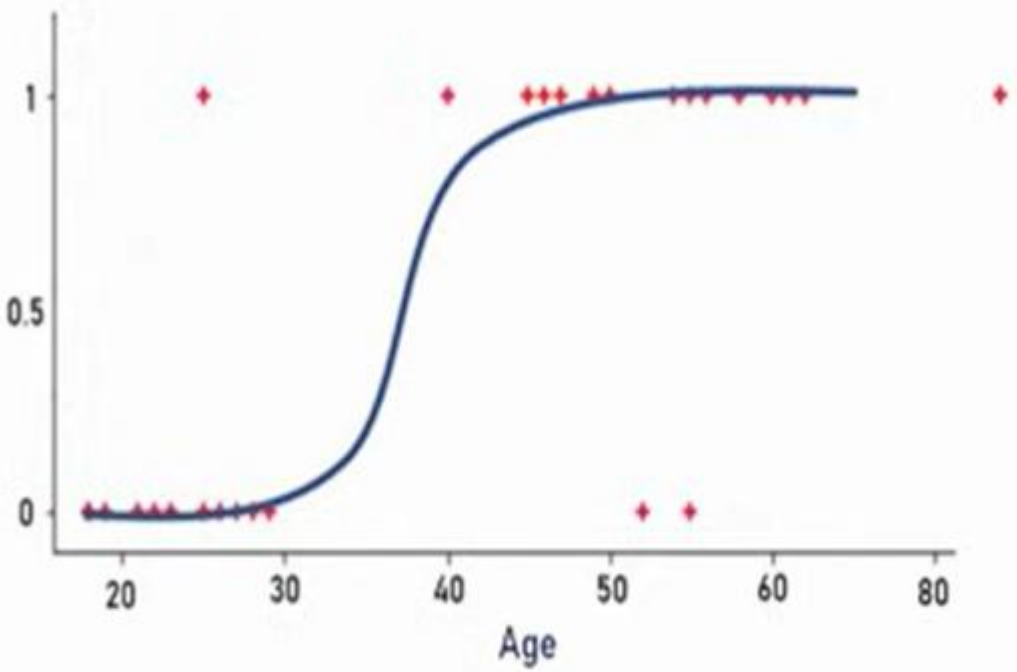
So: $0 < \text{sigmoid}(z) < 1$

That is why it is perfect for probability.

$$y = m * x + b$$

$$y = \frac{1}{1 + e^{-(m*x+b)}}$$





```
X = df[['age']]          # Feature (must be 2D)
y = df['bought_insurance']   # Target
X_train, X_test, Y_train, y_test = train_test_split(
    X, y,
    test_size=0.1,      # 20% test data
    random_state=42     # For reproducibility
)
```

X_test

	age
9	45
11	50

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
```

```
model.fit(X_train,Y_train)
```

LogisticRegression

LogisticRegression()

```
model.predict(X_test)
```

```
array([1, 1])
```

```
model.score(X_test,y_test)
```

```
1.0
```

```
model.predict_proba(X_test)
```

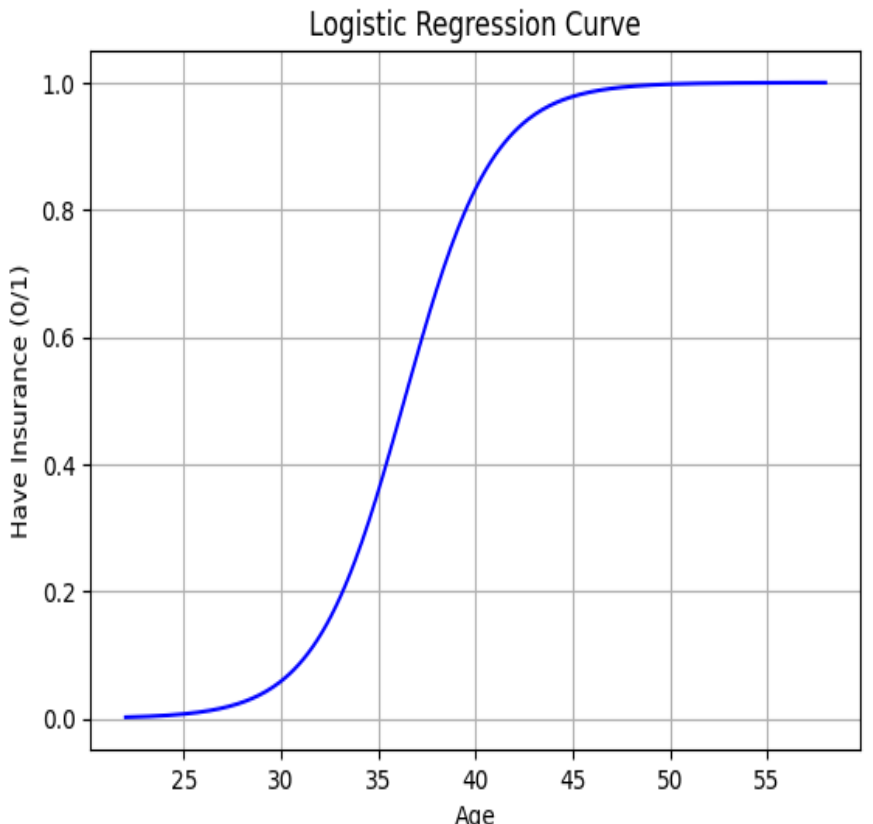
```
array([[0.02191085, 0.97808915],
       [0.00249532, 0.99750468]])
```

```
# Step 7: Plot Sigmoid Curve
age_range = np.linspace(df.age.min(), df.age.max(), 300).reshape(-1,1)
probabilities = model.predict_proba(age_range)[:,1]

plt.plot(age_range, probabilities, color='blue')

# Labels
plt.xlabel("Age")
plt.ylabel("Have Insurance (0/1)")
plt.title("Logistic Regression Curve")
plt.grid(True)
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid fe
  warnings.warn(
```



Logistic Regression - Example

- The dataset of pass or fail in an exam of 5 students is given in the table.
- Use logistic regression as classifier to answer the following questions.

1. Calculate the probability of pass for the student who studied 33 hours.
2. At least how many hours student should study that makes he will pass the course with the probability of more than 95%.

Hours Study	Pass (1) / Fail (0)
29	0
15	0
33	1
28	1
39	1

Assume the model suggested by the optimizer for odds of passing the course is,

$$\log(odds) = -64 + 2 * hours$$

Calculate the probability of pass for the student who studied 33 hours.

Model Given:

$$\log(\text{odds}) = -64 + 2 \times \text{hours}$$

Find z $\quad z = -64 + 2 \times 33$

$$z = -64 + 66 = 2$$

Z=2 then

Find Probability

$$p = \frac{1}{1 + e^{-2}}$$

$$e^{-2} \approx 0.135$$

$$p = \frac{1}{1.135}$$

$$p \approx 0.88$$

88% chance student pass the exam

$$\text{sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

At least how many hours student should study that makes he will pass the course with the probability of more than 95%.

Step 1: Start from Logistic Formula
When Probability = 0.95

$$p = \frac{1}{1 + e^{-z}}$$

$$0.95 = \frac{1}{1 + e^{-z}}$$

Multiply both sides:

$$0.95(1 + e^{-z}) = 1$$
$$0.95 + 0.95e^{-z} = 1$$
$$0.95e^{-z} = 0.05$$
$$e^{-z} = \frac{0.05}{0.95}$$
$$e^{-z} = 0.0526$$

Step 2: Take Natural Log

$$\ln(e^{-z}) = \ln(0.0526)$$
$$\ln(e^{-z}) = -z$$
$$-z = -2.94$$
$$z = 2.94$$

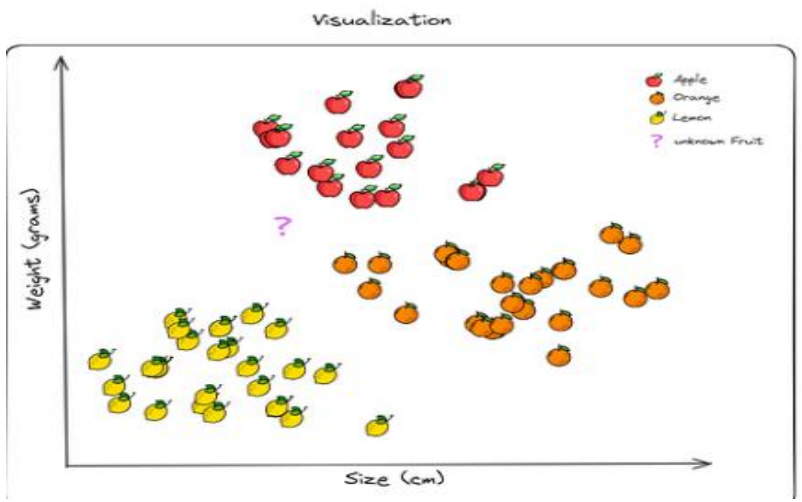
Step 3: Substitute in z Equation

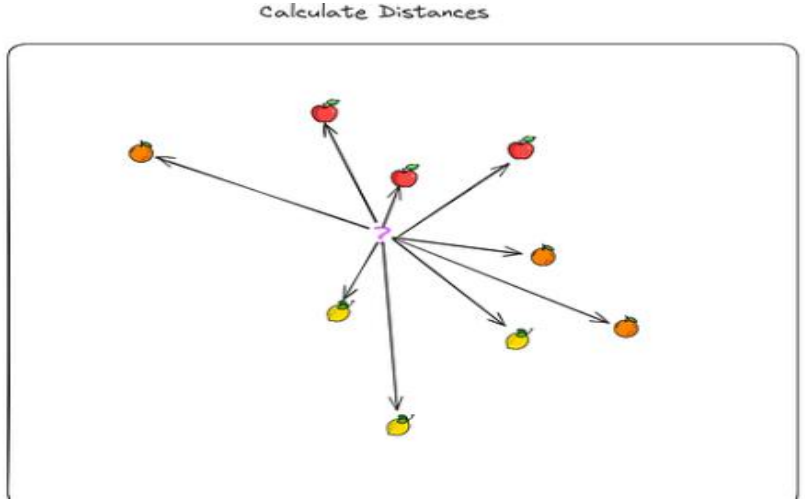
$$z = -64 + 2(\text{hours})$$
$$2.94 = -64 + 2(\text{hours})$$
$$2(\text{hours}) = 66.94$$
$$\text{hours} = 33.47$$

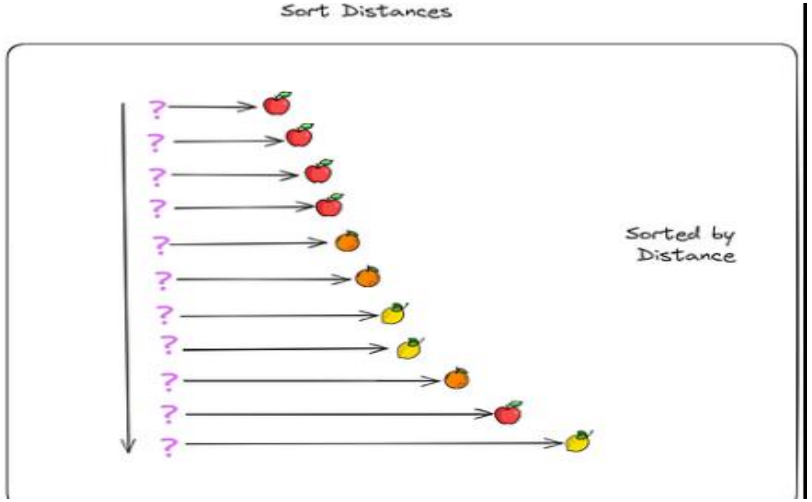
Student must study approximately: $\boxed{33.47 \text{ hours}}$
to have more than 95% probability of passing.

K-Nearest Neighbour

- KNN (K-Nearest Neighbors) is:A Supervised Machine Learning Algorithm
- Used for Classification and Regression but mostly used for classification
- For Example :Suppose we are predicting a fruit is an apple or orange based on weight and size , if k=3 and among 3 the nearest fruits 2 are apple and 1 is orange then kNN will classify new fruit as apple.







- No model is built during training.
- Training phase only stores the dataset .Actual computation happens during prediction time.
- It uses stored training instances directly. So it is called : Instance-based or Memory-based Learning
- It is also called Lazy Learning Algorithm
- If the model is not learning during training is called Instance-Based Learning Method
- It predicts the class of a new data point based on the majority class of its nearest neighbors.

How KNN works

Step 1: Store Training Data

- During training:
- KNN does NOT build a model.
- It simply stores all training data in memory.
- No equation is formed.
- This is why it is called Lazy Learning.

Step 2: Choose K

- Select number of neighbors.
- For Example: $K = 3$

Step 3: New Data Point Comes

- Suppose new fruit: Weight = 160g ,Size = 7cm
- We want to classify it.

Step 4: Calculate Distance

- We calculate distance between new fruit and all training fruits.

Most common distance formula is :**Euclidean Distance**

$$d = \sqrt{\left(x_2 - x_1\right)^2 + \left(y_2 - y_1\right)^2}$$

Step 5: Sort Distances

- Arrange distances in ascending order.
- Closest fruits come first.

Weight	Size	Label
150g	7cm	Apple
180g	8cm	Apple
120g	5cm	Orange
130g	6cm	Orange

Step 6: Select K Nearest Neighbors

If K = 3 ,Take first 3 closest fruits.

Example:

- Apple
- Apple
- Orange

Step 7: Majority Voting (Classification)

Count classes:

- Apple = 2
- Orange = 1
- Prediction = Apple

For Regression Case

Instead of voting:

Take average of K nearest values.

$$\hat{y} = \frac{1}{K}\sum y_i$$

KNN Algo


Store Data
Choose K
New Point
Calculate Distance
Sort Distances
Select K Nearest Neighbors
Majority Vote
Predict Class

Advantages of KNN Algorithm:

- It is simple to implement.
- It is robust to the noisy training data
- It can be more effective if the training data is large.

Disadvantages of KNN Algorithm:

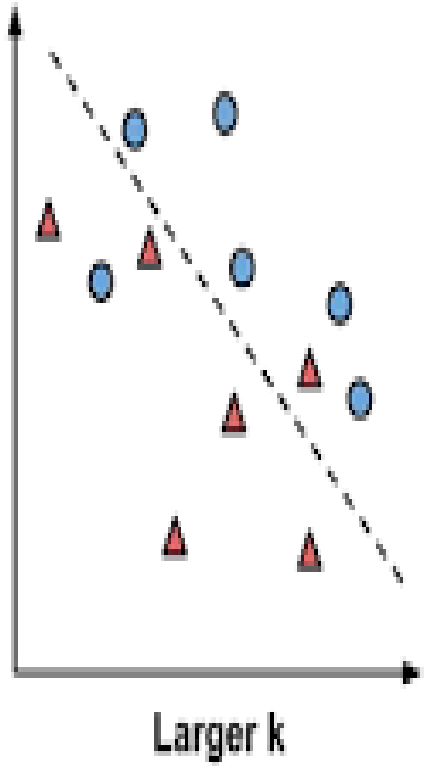
- Always needs to determine the value of K which may be complex some time.
- The computation cost is high because of calculating the distance between the data points for all the training samples.

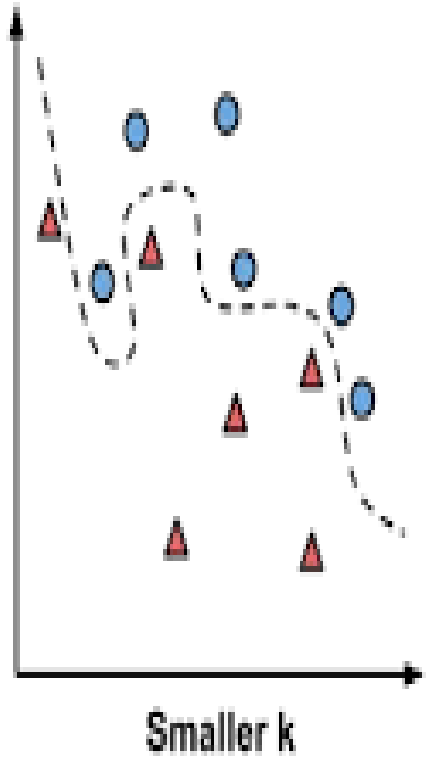
Larger K value:

The case of underfitting occurs when the value of k is increased. In this case, the model would be unable to correctly learn on the training data.

Smaller k value:

The condition of overfitting occurs when the value of k is smaller. The model will capture all of the training data, including noise. The model will perform poorly for the test data in this scenario.





KNN in Regression

- **Purpose**: Predicts a continuous value based on the average of the target values of its nearest neighbors.
- **Distance Metric**: Typically uses Euclidean distance to find the nearest neighbors.
- **Prediction**: The predicted value is the mean (or weighted average) of the values of the k nearest neighbors.
- **Sensitivity to k**: Small values of k can lead to high variance, while large values can result in high bias.
- **Outlier Sensitivity:** KNN regression can be sensitive to outliers as they may skew the average.
- **Non-parametric**: No assumptions about the underlying data distribution.
- **Scaling**: Input features should be scaled, as KNN is distance-based and sensitive to feature magnitudes.

KNN in Classification

- **Purpose**: Classifies a data point based on the majority class of its nearest neighbors.
- **Distance Metric**: Most commonly uses Euclidean distance, but can also use other distance metrics (e.g., Manhattan, Minkowski).
- **Prediction**: The predicted class is determined by the **majority vote** among the k nearest neighbors.
- **Sensitivity to k**: Small values of k can lead to overfitting, while large values may lead to underfitting.
- **Decision Boundaries**: KNN creates non-linear decision boundaries depending on the distribution of the data.
- **Outlier Handling**: More robust to outliers compared to KNN regression due to majority voting.
- **Non-parametric**: No need for assumptions about the distribution of data.
- **Feature Scaling**: Distance-based, so features need to be normalized or standardized for accurate predictions.

predict the category of "Barbie" movie with the IMDB rating 7.4 & duration 114 minutes where K=3

Training Data:

IMDb Rating	Duration	Genre
8.0 (Mission Impossible)	160	Action
6.2 (Gadar 2)	170	Action
7.2 (Rocky & Rani)	168	Comedy
8.2 (OMG 2)	155	Comedy

Solution:

It simply stores all movies as feature vectors:

$X = (\text{Rating}, \text{Duration})$ Example:Mission Impossible → (8.0, 160)

K=3 is given and new data point is(7.4,114)

Calculate Euclidean Distance $\mathrm{d} = \sqrt{\left(x_2 - x_1\right)^2 + \left(y_2 - y_1\right)^2}$

Distance to Mission Impossible

$$\sqrt{\left(7.4 - 8.0\right)^2 + \left(114 - 160\right)^2}$$

$$= \sqrt{0.36 + 2116}$$

$$= \sqrt{2116.36} \approx 46$$

Distance to Gadar 2

$$\sqrt{(7.4-6.2)^2+(114-170)^2}$$
$$=\sqrt{1.44+3136}$$
$$=\sqrt{3137.44}\approx 56$$

Distance to Rocky & Rani

$$\sqrt{(7.4-7.2)^2+(114-168)^2}$$
$$=\sqrt{0.04+2916}$$
$$=\sqrt{2916.04}\approx 54$$

Distance to OMG 2

$$\sqrt{(7.4-8.2)^2+(114-155)^2}$$
$$=\sqrt{0.64+1681}$$
$$=\sqrt{1681.64}\approx 41$$

Sort the distance

Movie	Distance	Genre
OMG 2	41	Comedy
Mission Impossible	46	Action
Rocky & Rani	54	Comedy
Gadar 2	56	Action

Select K = 3 Nearest

Nearest 3 are :

- OMG 2 → Comedy
- Mission Impossible → Action
- Rocky & Rani → Comedy

Majority Voting

Comedy = 2

Action = 1

Majority = Comedy

Final Prediction

Barbie → Comedy

A retailer uses a k-Nearest Neighbors (k-NN) classifier to predict whether a customer will make a purchase (1 for purchase, 0 for no purchase) based on features like age and spending score. The model uses k=3 neighbors. Given the following data points: • Customer A: (Age: 25, Spending: 80) ⟶ Purchase: 1 • Customer B: (Age: 30, Spending: 60) ⟶ Purchase: 0 • Customer C: (Age: 28, Spending: 70) ⟶ Purchase: 1 What will the prediction be for a new customer with age = 27 and spending = 75?	5	3	A

Customer	Age	Spending	Purchase
A	25	80	1
B	30	60	0
C	28	70	1

K=3
Age = 27
Spending = 75
We must predict Purchase (1 or 0).

Distance to Customer A (25, 80)

$$d_A = \sqrt{(27-25)^2 + (75-80)^2}$$
$$= \sqrt{2^2 + (-5)^2}$$
$$= \sqrt{4+25}$$
$$= \sqrt{29}$$
$$\approx 5.39$$

Distance to Customer B (30, 60)

$$d_B = \sqrt{(27-30)^2 + (75-60)^2}$$
$$= \sqrt{(-3)^2 + 15^2}$$
$$= \sqrt{9+225}$$
$$= \sqrt{234}$$
$$\approx 15.30$$

Distance to Customer C (28, 70)

$$d_C = \sqrt{(27-28)^2 + (75-70)^2}$$
$$= \sqrt{(-1)^2 + 5^2}$$
$$= \sqrt{1+25}$$
$$= \sqrt{26}$$
$$\approx 5.10$$

Since K = 3, we take all three:
C → 1
A → 1
B → 0

Customer	Distance	Purchase
C	5.10	1
A	5.39	1
B	15.30	0

Sort the distance

Majority Voting
Purchase = 1 appears 2 time
Purchase = 0 appears 1 time
Majority = 1

New customer will purchase